

HTL Maturatrainer

DIPLOMARBEIT

verfasst im Rahmen der

Reife- und Diplomprüfung

an der

Höheren Abteilung für Medientechnik

Eingereicht von:

Thomas Gossenreiter
Stefanie Steinmair

Betreuer:

Birgit Schröder

Leonding, 6. April 2026

Ich erkläre an Eides statt, dass ich die vorliegende Diplomarbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt bzw. die wörtlich oder sinngemäß entnommenen Stellen als solche kenntlich gemacht habe.

Die Arbeit wurde bisher in gleicher oder ähnlicher Weise keiner anderen Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

Die vorliegende Diplomarbeit ist mit dem elektronisch übermittelten Textdokument identisch.

Leonding, 6. April 2026

Thomas Gossenreiter & Stefanie Steinmair

Abstract

This diploma thesis focuses on the development of a web-based learning platform designed to support students in preparing for their final school-leaving examinations (Matura). The objective of this work is to provide students with individualized learning support and to assist them in building knowledge through a structured learning strategy and realistic exam simulations.



The technical implementation is realized as a modern web application based on Angular for the frontend and a Quarkus-based REST API for the backend, with data management handled by a MySQL database.

The core focus lies on the clear separation of learning and testing phases. This is achieved through a practice mode featuring automated repetition logic as well as an exam mode with time limits and delayed feedback. Furthermore, the application is enhanced by statistical evaluations, which provide transparent tracking of learning progress and a targeted analysis of errors through integrated solution paths.

Zusammenfassung

Diese Diplomarbeit beschäftigt sich mit der Entwicklung einer webbasierten Lernplattform zur Vorbereitung auf die Reifeprüfung (Matura). Ziel dieser Arbeit ist, Schüler:innen eine individuelle Lernbegleitung zu bieten und durch eine strukturierte Lernstrategie und durch realitätsnahe Prüfungssimulationen beim Wissensaufbau zu unterstützen.



Die technische Umsetzung erfolgt als moderne Web-Applikation auf Basis von Angular im Frontend und einem Quarkus REST-API-Backend, wobei die Datenhaltung über eine MySQL-Datenbank realisiert wird.

Der Fokus liegt dabei auf der Trennung von Lern- und Testphasen. Dies wird durch einen Übungsmodus mit automatisierter Wiederholungslogik sowie einen Prüfungsmodus mit Zeitlimit und verzögertem Feedback umgesetzt. Die Anwendung wird außerdem durch statistische Auswertungen ergänzt, welche eine transparente Verfolgung des Lernfortschritts und eine gezielte Analyse von Fehlern durch hinterlegte Lösungswege ermöglichen.

Inhaltsverzeichnis

1	Einleitung	1
1.1	Ausgangslage	1
1.1.1	Die HTL-Matura in Österreich	1
1.2	Motivation und Problemstellung	3
1.3	Grundlagen und Analyse	4
1.3.1	Marktanalyse	4
1.3.2	Maturatraining digitalisieren	6
1.4	Lernstrategien	6
1.4.1	Spaced Repetition	6
1.4.2	Das Leitner-System	7
1.4.3	Microlearning	8
1.4.4	Kombination aus Leitner und Spaced Repetition	9
1.5	Zielsetzung	9
2	Technologische Entscheidungen und Architektur	10
2.1	Backend	10
2.1.1	Spring Boot – Die Evolution des Java-Enterprise-Standards . . .	11
2.1.2	Node.js - Das ereignisgesteuerte Gegenmodell	15
2.1.3	Quarkus	20
2.1.4	Fazit und finale Technologieentscheidung	25
2.2	Frontend	26
2.2.1	Angular	28
2.2.2	Flutter	31
2.2.3	React	35
2.2.4	Fazit und finale Technologieentscheidung	38
3	Praktische Umsetzung	40
3.1	Logische Programmstruktur und Use Cases	40

3.2	Startansicht	42
3.2.1	Aufbau und Funktionen der Startseite	42
3.3	Fragenpool erstellen und verwalten	44
3.3.1	Die „Fragen browsen“-Ansicht als digitaler Katalog	44
3.3.2	Auswahl und individuelle Anpassung des Fragenpools	46
3.4	Use Case: Üben	47
3.4.1	Übungsmodus mit direktem Feedback	47
3.4.2	Lösungswege ansehen und erstellen	53
3.5	Use Case: Prüfungssimulation	56
3.5.1	Konfiguration einer Prüfung	56
3.5.2	Prüfungsmodus mit Zeitlimit	58
3.5.3	Bewertung und Ergebnisausgabe	60
3.6	Use Case: Fortschritt einsehen und analysieren	63
3.6.1	Prüfungsverlauf und tiefergehende Detailanalyse	63
3.6.2	Zentrale Lernstatistiken und Motivationsmechanismen	65
3.6.3	Selektive Auswertung nach Themenpools	66
3.7	Implementierungsdetails	66
3.7.1	Datenmodell	66
3.7.2	Überblick über Komponenten	70
3.7.3	Custom CSS-Klassen und eingebundene Bibliotheken	73
3.8	Zukunftsansichten und Erweiterungsmöglichkeiten	78
3.8.1	KI-gestützte Inhaltsgenerierung und Feedback	78
3.8.2	Mobile Applikation (Cross-Platform)	79
4	Zusammenfassung	80
	Literaturverzeichnis	V
	Abbildungsverzeichnis	VII
	Tabellenverzeichnis	IX

1 Einleitung

1.1 Ausgangslage

Die Vorbereitung auf Tests, Schularbeiten und schließlich die Matura erfordern ein hohes Maß an Eigenverantwortung und Organisation. Im Moment erfolgt das Lernen größtenteils mithilfe von Skripten, Arbeitsblättern und Übungsbeispielen aus dem Unterricht. Diese bewährten Methoden stoßen allerdings dort an ihre Grenzen, wo es um die Dynamik des Lernprozess geht.

Zwar ist eine erfolgreiche Vorbereitung mit den vorhandenen Mitteln durchaus möglich, sie ist allerdings häufig mit einem hohen organisatorischen Aufwand verbunden. Wesentlich dabei ist das fehlende unmittelbare Feedback beim eigenständigen Üben. Schüler und Schülerinnen können ihre Ergebnisse mit Musterlösungen abgleichen, dabei bleibt jedoch der Weg zur Lösung und die Analyse spezifischer Fehlerquellen oft ungeklärt. Weiters ist es ohne digitale Unterstützung schwierig, den individuellen Lernfortschritt über einen längeren Zeitraum präzise zu verfolgen oder gezielt Schwachstellen zu identifizieren, die mehr Übung erfordern.

1.1.1 Die HTL-Matura in Österreich

Mit der Reife- und Diplomprüfung wird die Ausbildung an einer Höheren Technischen Lehranstalt (HTL) in Österreich abgeschlossen [1]. Diese ist modular aufgebaut und in ein 3-Säulen-Modell (Abbildung 1) unterteilt. Während dabei die erste Säule, die Diplomarbeit, ein in einem Team fertiggestelltes Projekt darstellt, konzentriert sich die Vorbereitung der Schüler:innen vor allem auf die weiteren zwei Säulen, die den Kern der klassischen Prüfungssituation bilden.

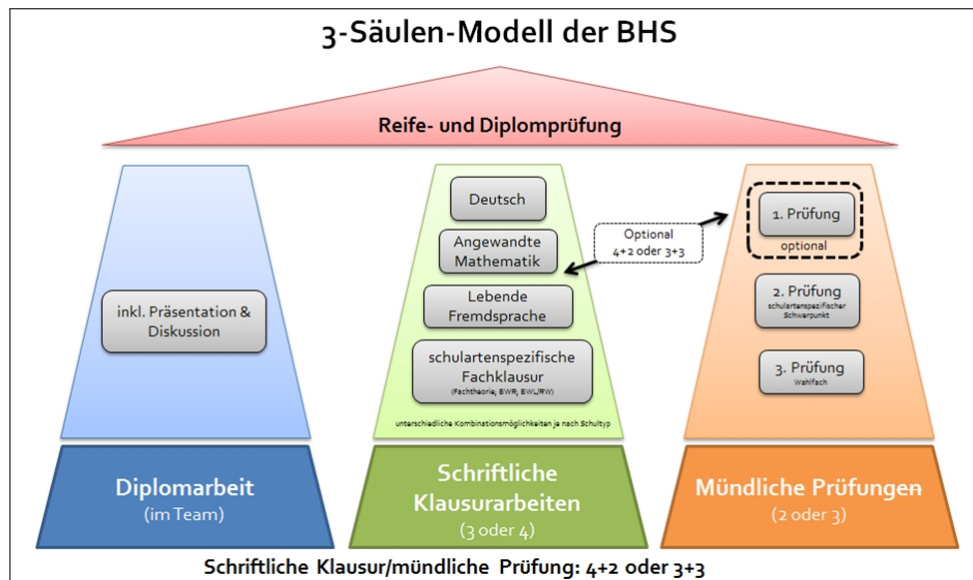


Abbildung 1: Das 3-Säulen-Modell der Reife- und Diplomprüfung ([1])

Die schriftliche Klausurprüfung (Säule 2)

Im schriftlichen Teil der Matura müssen Schüler:innen entweder drei oder vier Klausurarbeiten absolvieren, je nachdem, ob sie alle vier Fächer schriftlich oder eine der Sprachen mündlich ablegen möchten. Hierbei gibt es zwei verschiedene Arten von Aufgaben:

- **Zentralisierte Standardisierung:**

In den Hauptfächern Deutsch, Englisch und Angewandte Mathematik werden die Aufgabenstellungen vom Bundesministerium zentral vorgegeben. Das bedeutet, dass für diese Fächer in jeder Berufsbildenden Höheren Schule (BHS), wie es auch bei der HTL der Fall ist, die selben Aufgaben pro Jahrgang zur Matura kommen.

- **Fachspezifische Klausur:**

Zusätzlich zu den Hauptfächern muss eine schriftliche Prüfung in einem schwerpunktspezifischen Fach wie beispielsweise Softwareentwicklung abgelegt werden. Diese Aufgaben erstellen die Lehrkräfte an den jeweiligen Schulstandorten. Das heißt also, dass diese Aufgaben schulspezifisch erstellt werden und sich auch bei Schulen mit selbem Schwerpunkt unterscheiden können.

Die mündliche Prüfung (Säule 3)

Bei der mündlichen Säule gibt es zwei oder drei Teilprüfungen, je nachdem für welche Variante sich die Schüler:innen entschieden haben. Hierbei müssen sie aus einem Pool an Themenbereichen ziehen, die im Vorfeld angekündigt wurden. Vor allem an technischen

Schulen ist dabei vorgeschrieben, dass mindestens eine Prüfung aus der Fachtheorie stammen muss.

Bereits bestehende Lernplattformen fokussieren sich oft nur auf die zentralisierten Fächer und erfüllen die spezifischen Anforderungen des technischen Lehrplans nur bedingt. Die geplante Matura-Lernplattform setzt genau hier an. Sie soll nicht als Ersatz für den Unterricht, sondern als ergänzendes Werkzeug dienen und den Lernvorgang für Schüler:innen durch direktes Feedback und eine strukturierte Fortschrittskontrolle effizienter und transparenter gestalten.

1.2 Motivation und Problemstellung

Die im vorherigen Abschnitt beschriebene Struktur der HTL-Matura zeigt, dass der Lernprozess bei der Vorbereitung für die Matura insbesondere bei den standortspezifischen Anforderungen stark von eigenständigem Üben und individueller Organisation geprägt ist. Die Motivation für diese Diplomarbeit ergibt sich aus der Überlegung, wie moderne Webtechnologien genutzt werden können, um diesen Prozess von einer passiven Kontrolle, wie es das reine Abgleichen mit Musterlösungen ist, hin zu einer aktiven Lernbegleitung zu transformieren. Daraus ergibt sich eine zentrale Fragestellung: **Wie kann eine digitale Plattform gestaltet sein, sodass sie den Lernprozess strukturierter, transparenter und effizienter macht?**

Ein wesentlicher Aspekt liegt hierbei in der Handhabung von Fehlern. Im aktuellen Lernvorgang werden Fehler oft nur zur Kenntnis genommen, nicht aber systematisch für den weiteren Lernpfad genutzt. Es bleibt also oft unklar, ob ein Fehler auf einem Flüchtigkeitsfehler oder einem grundlegenden Verständnisproblem basiert. Die Motivation dieser Arbeit liegt darin, ein System zu entwickeln, das genau solche Schwachstellen identifiziert und durch gezielte Wiederholungsintervalle den Aufbau eines nachhaltigen Wissensfundaments unterstützt.

Zusätzlich stellt auch der Mangel an Transparenz über den eigenen Wissensstand eine Motivation für die Entwicklung an diesem Projekt dar. Während klassische Lernmaterialien statisch sind, bietet eine digitale Plattform die Chance, den eigenen Lernfortschritt über Monate hinweg zu sammeln und zu verfolgen. Ziel ist es, den Schüler:innen ein Werkzeug zur Verfügung zu stellen, welches durch Visualisierung von Lernstatistiken

die Selbsteinschätzung unterstützt und somit die organisatorische Last der Prüfungsvorbereitung reduziert.

1.3 Grundlagen und Analyse

1.3.1 Marktanalyse

In der Analyse bestehender digitaler Angebote liegt der Fokus auf deren Eignung für die spezifischen Anforderungen der Matura-Vorbereitung. Dabei spielen die authentische Prüfungssimulation, die Bewältigung des enormen Stoffumfangs sowie die fachliche Komplexität der Aufgabenstellungen eine entscheidende Rolle.

StudySmarter

StudySmarter wurde ursprünglich als Tool für das *Microlearning* und zum Auswendiglernen von isoliertem Faktenwissen konzipiert. Diese historische Ausrichtung auf eine „häppchenweise“ Wissensvermittlung macht die Plattform zwar effizient für Vokabeltraining oder kurze Definitionen, führt jedoch bei der komplexen Matura-Vorbereitung an deutliche Grenzen.

Hinsichtlich der **Lern- und Quizfunktionen** setzt StudySmarter primär auf das Prinzip der digitalen Karteikarten. Benutzer können Sätze von Karten durcharbeiten, wobei ein Algorithmus (Spaced Repetition) die Abstände der Wiederholungen steuert. Die Quizzes bestehen meist aus einfachen Zuordnungen oder Single-Choice-Fragen, die auf ein schnelles Erfolgserlebnis abzielen. Eine gezielte Auswahl spezifischer Themenpools, wie sie für die systematische Vorbereitung auf einzelne Matura-Kapitel nötig wäre, ist oft nur durch das manuelle Sortieren hunderter Einzelkarten möglich, was bei dem großen Stoffumfang der Matura sehr zeitintensiv ist.

Im Bereich des **Prüfungsmodus** zeigt sich ein deutliches Defizit: Die Quiz-Funktionen liefern meist ein unmittelbares Feedback nach jeder einzelnen Frage. Zwar bietet StudySmarter mittlerweile Funktionen an, die als „Prüfung“ betitelt werden, diese sind jedoch oft nur eine Aneinanderreihung der bekannten Karteikarten ohne echte Prüfungssimulation. Für die Matura ist hingegen die Simulation einer kompletten, mehrstündigen Klausur ohne Unterbrechung entscheidend. Das sofortige Anzeigen der Lösung stört den Konzentrationsfluss und verhindert das Training der mentalen Ausdauer sowie des Zeitmanagements, welche für eine reale Prüfungssituation essenziell sind.

Serlo / Anton

Diese Plattformen sind historisch im Bereich der Primar- und Sekundarstufe 1 verwurzelt, wo spielerisches Lernen (*Gamification*) im Vordergrund steht. Das **Lernkonzept** basiert hier auf kleinen Lerneinheiten, die mit Belohnungen wie Sternen oder Punkten verknüpft sind.

Die Komplexität der Aufgaben beschränkt sich meist auf einfache Multiple-Choice-Fragen oder Lückentexte, die per Drag-and-Drop gelöst werden. Während dies für die Festigung von Basiswissen in unteren Schulstufen ideal ist, verlangt die Matura – insbesondere an technischen Schulen – das aktive Erarbeiten und Nachvollziehen mehrschrittiger Lösungswege sowie Freitext-Antworten. Ein **Prüfungsmodus unter Zeitdruck** oder eine Simulation, die über das spielerische Sammeln von Punkten hinausgeht, ist hier kaum vorhanden. Die **Statistiken** dienen eher der Motivation als der harten datengestützten Analyse von Wissenslücken, da sie nicht präzise aufzeigen, an welcher Stelle eines komplexen Lösungsweges ein Schüler gescheitert ist.

Zusammenfassender Vergleich und Alleinstellungsmerkmale

Im Gegensatz zu diesen allgemein gehaltenen Apps ist das vorliegende Projekt gezielt auf die strukturelle Tiefe der Matura ausgelegt. Während herkömmliche Tools oft generische Inhalte und einfache „Richtig/Falsch“-Statistiken nutzen, setzt dieses System auf folgende Alleinstellungsmerkmale:

- **Detaillierte Lösungswege:** Diese wurden von Schülern für Schüler aufbereitet, um komplexe Fehlerquellen verständlich zu machen und Erklärungen auf Augenhöhe zu bieten.
- **Gezielte Themenpools:** Eine flexible Auswahl spezifischer Themengebiete ermöglicht es, den Fokus exakt auf individuelle Schwachstellen zu legen, anstatt starren Lernpfaden folgen zu müssen.
- **Authentischer Prüfungsmodus:** Durch einstellbare Zeitlimits, freie Navigation zwischen den Fragen und verzögertes Feedback wird die reale Matura-Situation realitätsnah simuliert.
- **Übungsmodus mit direktem Feedback:** Dieser Modus ermöglicht es, sofortige Rückmeldung zu erhalten, um gezielt an Fehlern zu arbeiten, ohne den Druck einer kompletten Prüfungssimulation.

1.3.2 Maturatraining digitalisieren

Die Digitalisierung des Maturatrainings zielt darauf ab, fragmentierte Lernprozesse in eine strukturierte, interaktive Umgebung zu überführen. Ein zentrales Problem in der Vorbereitung war bisher die mangelnde Konsistenz der Unterlagen: Schüler:innen arbeiteten oft mit unvollständigen Mitschriften, die teils analog auf Papier und teils digital als unsortierte Fragmente vorlagen.

Das Partnerprojekt hat hierfür die notwendige Grundlage geschaffen, indem verstreute Mitschriften zentralisiert und in digitale Fragenkataloge überführt wurden. Da zu einer vollständigen Lernplattform ein Übungs- und Prüfungsmodus inkludiert sein sollte, konzentriert sich die vorliegende Arbeit auf die methodische Nutzbarmachung dieser Inhalte durch die Entwicklung eines Übungs- und Prüfungsmodus.

1.4 Lernstrategien

Im Rahmen der Entwicklung einer digitalen Lernplattform für HTL-Maturanten müssen Lernstrategien gewählt werden, die über reines Auswendiglernen hinausgehen. Die Matura zeichnet sich durch einen enormen Stoffumfang und eine hohe Komplexität aus. Gefragt ist nicht nur isoliertes Faktenwissen, sondern vor allem das Verständnis von Zusammenhängen und die Anwendung von Wissen auf neue, vernetzte Problemstellungen.

1.4.1 Spaced Repetition

Spaced Repetition, oder verteiltes Lernen, ist eine kognitionswissenschaftliche Methode, die darauf abzielt, Informationen durch zeitlich versetzte Wiederholungen im Langzeitgedächtnis zu verankern. Sie stellt die wissenschaftliche Antwort auf die Ebbinghaus'sche Vergessenskurve dar.

Allgemeine Funktionsweise

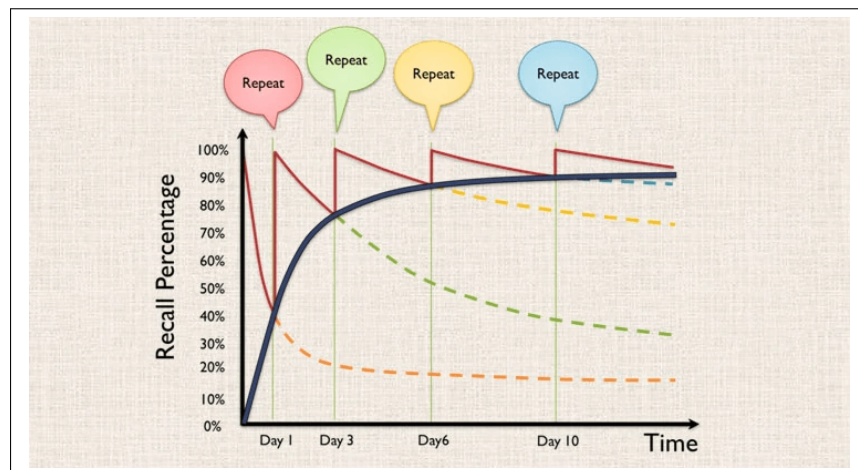


Abbildung 2: Die Ebbinghaus'sche Vergessenskurve und die Wirkung von Spaced Repetition

Diese Methode basiert auf dem Prinzip, dass Informationen genau dann wiederholt werden, wenn das Gehirn kurz davor ist, sie zu vergessen. Ein praxisnahes Beispiel ist das Lernen einer komplexen Formel aus der Nachrichtentechnik: Die erste Wiederholung findet nach 24 Stunden statt, die zweite nach drei Tagen und die dritte nach einer Woche. Mit jedem Erfolg vergrößert sich der zeitliche Abstand progressiv.

Kontext der Matura

Diese Methode erweist sich als extrem zeiteffizient, da sie das kurzfristige Auswendiglernen direkt vor der Prüfung durch eine nachhaltige Verankerung im Langzeitgedächtnis ersetzt. Für die Maturavorbereitung ist dies aufgrund des langen Zeitraums essenziell, um sicherzustellen, dass technische Grundlagen auch dann noch präsent sind, wenn am Ende des Jahres hochkomplexe Anwendungsbeispiele gelöst werden müssen.

1.4.2 Das Leitner-System

Das Leitner-System ist eine systematische Lernmethode, die die Organisation von Lerninhalten in verschiedenen Kategorien nutzt, um den Lernprozess zu strukturieren.

Allgemeine Funktionsweise

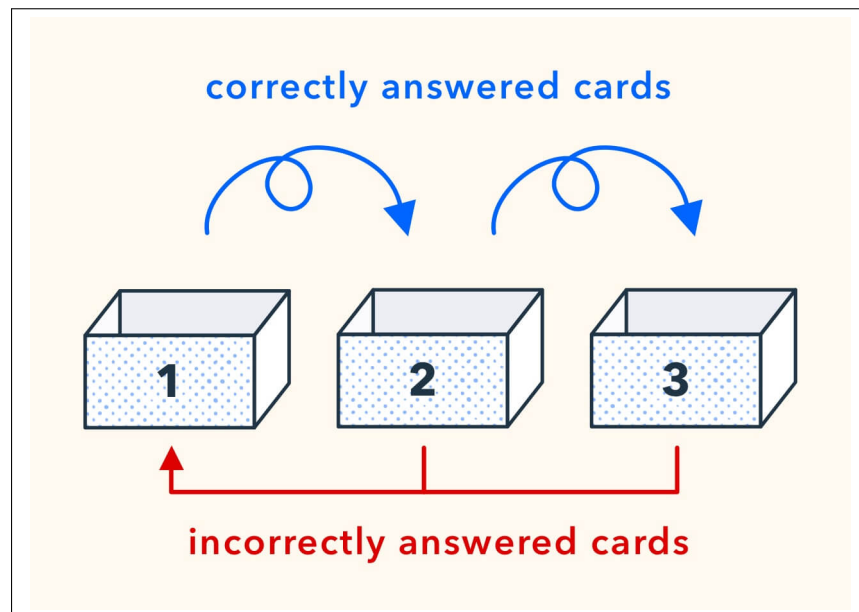


Abbildung 3: Das Leitner-System

Das System nutzt das Prinzip von physischen oder digitalen Lernkartei-Boxen, um den Lernstoff nach dem individuellen Beherrschungsgrad zu sortieren. In einer digitalen Anwendung wandern Karten bei einer richtigen Antwort in ein höheres Fach, das seltener abgefragt wird, während sie bei Fehlern unmittelbar zurück in das erste Fach verschoben werden.

Kontext der Matura

Obwohl das System eine gute Visualisierung des Fortschritts bietet, ist es in seiner klassischen Form oft zu starr. Für Maturanten ist es dennoch hilfreich, da die Einteilung in Fächer den Fortschritt bei großen Themengebieten greifbar macht.

1.4.3 Microlearning

Microlearning bezeichnet das Lernen in sehr kurzen, inhaltlich fokussierten Einheiten, um umfangreiche Wissensbereiche handhabbar zu machen.

Allgemeine Funktionsweise

Microlearning bricht Stoffmengen in kleinste Einheiten auf. Ziel ist es, Wissen in "mundgerechten" Häppchen zu vermitteln, was die Hemmschwelle senkt und schnelles Feedback ermöglicht.

Kontext der Matura

Während dies den Einstieg erleichtert, ist die Methode für die Matura nur bedingt geeignet. Die Gefahr besteht in einer Fragmentierung des Wissens: Die Matura verlangt

das Verständnis großer, vernetzter Systeme. Werden diese zu stark zerstückelt, verlieren Schüler den Blick für die Kausalzusammenhänge. Microlearning versagt daher oft bei der Vermittlung der für die Matura essenziellen Tiefe.

1.4.4 Kombination aus Leitner und Spaced Repetition

Für den Maturatrainer wird eine Zusammensetzung aus dem Leitner-System und Spaced Repetition implementiert. Diese Kombination löst die Schwächen der Einzelmethoden auf:

- **Struktur durch Leitner:** Die Einteilung in fünf digitale Fächer dient als intuitive Benutzeroberfläche. Der Schüler sieht jederzeit, wie viele Karten sich im Bereich *Beherrscht* befinden, was die Selbstwirksamkeit stärkt.
- **Wiederholung durch Spaced Repetition:** Durch die Einteilung in die Leitner-Fächer wird die Spaced Repetition automatisch gesteuert. Fragen, die falsch beantwortet wurden, bleiben im ersten Fach, die häufig richtig beantworteten Karten wandern in die höheren Fächer, wodurch die Intervalle für die Wiederholung automatisch angepasst werden. Je höher das Fach, desto seltener die Wiederholung, was die Effizienz maximiert.

Dieser Mix ist für die Matura ideal, er bietet die motivierende Struktur des Leitner-Systems, kombiniert mit der Effizienz der Spaced Repetition, um sicherzustellen, dass das Wissen nachhaltig verinnerlicht ist.

1.5 Zielsetzung

Das primäre Ziel dieser Diplomarbeit ist die Realisierung von „LearnHub“, einer spezialisierten Web-Plattform zur gezielten Matura-Vorbereitung an der HTL Leonding. Die Anwendung strukturiert den Lernprozess durch eine automatisierte Methodik, die auf dem Leitner-System und Spaced Repetition basiert, um den langfristigen Wissensaufbau zu maximieren.

Nutzer:innen können dabei ohne großen Aufwand individualisierte Fragenpools erstellen und ihren Lernstand durch dynamische Statistiken objektiv bewerten. Eine konfigurierbare Prüfungssimulation mit variablen Zeitlimits und Themenkomplexen ergänzt das System, um eine realitätsnahe Vorbereitung auf die Abschlussprüfung zu gewährleisten und die Eigenverantwortung der Schüler:innen digital zu unterstützen.

2 Technologische Entscheidungen und Architektur

2.1 Backend

Das Backend fungiert als zentrale Logik- und Datenschicht für beide Teilprojekte der Diplomarbeit. Es stellt die notwendigen Ressourcen für die Verwaltung der Lernmaterialien als auch für das interaktive Lernsystem über eine einheitliche REST-Schnittstelle zur Verfügung.

Dabei muss das Backend folgende projektspezifische Kernaufgaben erfüllen:

- **Zentrale Datenhaltung:** Verwaltung und Persistierung von Fragen, Lösungswegen und Nutzerdaten in einer gemeinsamen Datenbank, um Konsistenz zwischen den Teilprojekten zu gewährleisten.
- **Materialverwaltung:** Abwicklung von Datei-Uploads (Mitschriften, Skripte) und deren strukturierte Zuordnung zu Fachbereichen.
- **Validierung und Business-Logik:** Prüfung von Nutzerantworten im interaktiven Modus und die serverseitige Interpretation von Lernfortschritten.
- **Schnittstellenfunktion:** Bereitstellung von Endpunkten für das Frontend, wobei das Backend die endgültige Entscheidungshoheit über die Datenintegrität besitzt.

Die technologische Wahl ist aufgrund dieser zentralen Rolle also entscheidend für die Performance und Wartbarkeit des Gesamtsystems. Im folgenden Vergleich werden daher die infrage kommenden Frameworks gegenübergestellt.

Analyse technischer Anforderungen

Die Auswahl des Backend-Frameworks ist eine kritische Entscheidung, die Auswirkungen auf die gesamte Lebensdauer der Software hat. Dabei geht es nicht nur um die Syntax, sondern vor allem darum, wie die Anwendung mit der Infrastruktur (Datenbanken, Container) interagiert und wie sie konfiguriert wird.

Folgende Punkte wurden dabei kritisch untersucht:

- **Technische Basis und Ressourceneffizienz:** Untersuchung, wie das Framework intern arbeitet (Optimierung beim Starten vs. Optimierung beim Bauen). Hierbei ist wichtig, wie viel Arbeitsspeicher benötigt wird und wie schnell das Backend in einer Container-Umgebung einsatzbereit ist.
- **Datenbankanbindung und Datenverwaltung:** Analyse, wie einfach sich Datenbanken einbinden lassen und wie gut das Framework dabei hilft, diese als Objekte abzubilden und als solche dann zu persistieren (ORM). Ein wichtiger Punkt ist hier die Flexibilität bei verschiedenen Datenbanksystemen.
- **Einrichtung und automatisierte Abläufe:** Bewertung der Mechanismen für das *Database Seeding*. Es wird geprüft, wie Stammdaten automatisch beim Starten in die Datenbank kommen und wie gut das Framework die nötige Infrastruktur (wie Docker-Container) während der Entwicklung selbst verwaltet.
- **Konfiguration und Sicherheit:** Vergleich, wie Einstellungen über die zentrale Einstellungsdatei gesteuert werden. Dabei geht es vor allem um die Trennung von Entwicklungs- und Live-Umgebungen sowie den sicheren Umgang mit Passwörtern und Zugangsdaten.
- **Zukunftssicherheit und Zusammenarbeit:** Bewertung, wie gut die Dokumentation ist und wie einfach es ist, Hilfe bei Problemen zu finden. Da zwei Teams am selben Backend arbeiten, spielt auch die Erweiterbarkeit für beide Teilprojekte eine große Rolle.

2.1.1 Spring Boot – Die Evolution des Java-Enterprise-Standards



Abbildung 4: Logo von Spring Boot

Historischer Kontext

Spring Boot wurde 2014 veröffentlicht und hatte als Ziel, die massive Komplexität der

damaligen Java-Enterprise-Entwicklung (J2EE/Jakarta EE) zu lösen. Zuvor verbrachten Entwickler einen erheblichen Teil ihrer Zeit mit der Konfiguration von XML-Dateien, wobei man hunderte Zeilen XML-Code schreiben musste, um einfache Datenbankverbindungen zu realisieren. Das wurde schnell zu einer redundanten Arbeit und bekannt als „*Configuration-Hell*“, da man Fehler oft unglaublich schwer finden und ausbessern konnte. Mit Spring Boot mussten die Entwickler:innen nicht mehr wissen, wie ein Webserver startet, sie mussten ihn nur noch benutzen. Es basiert dabei auf dem bewährten Spring Framework und nutzt dessen *Inversion of Control* Container, um die Abhängigkeiten der Anwendung zu verwalten. [2] [3]

Technologischer Durchbruch: Convention over Configuration

Entscheidend für den technologischen Durchbruch war das Prinzip „Convention over Configuration“. Anstatt alles explizit konfigurieren zu müssen, geht Spring Boot dabei einfach davon aus, dass Entwickler in 90% der Fälle dieselben Standardeinstellungen für Datenbanken oder Webserver benötigen. Das Framework erkennt beim Start, welche Bibliotheken im Klassenpfad vorhanden sind und konfiguriert diese automatisch (*Auto-Configuration*). Wenn zum Beispiel eine H2-Datenbank im Klassenpfad liegt, dann möchte der:die Entwickler:in vermutlich eine In-Memory-Datenbank nutzen. Durch diese Automatisierung wurde Java wieder wettbewerbsfähig gegenüber schnellen Frameworks in Ruby- und Python-Umgebungen. [4]

Das führte dazu, dass Unternehmen wie Netflix, Uber oder Zalando Spring Boot als Basis für ihre Microservice-Architekturen wählten.

Ineffizienz in modernen Container-Umgebungen [5]

Obwohl Spring Boot die Entwicklung um ein Vielfaches beschleunigt hat, kommt es in modernen, containerisierten und ressourcenbeschränkten Umgebungen zu einer messbaren Ineffizienz. Das Kernproblem liegt in der hohen Dynamik des Startvorgangs. Während moderne Cloud-Native-Ansätze versuchen, so viel Wissen und Entscheidungen wie möglich in die Kompilierzeit zu verlagern, trifft Spring Boot fast alle architektonischen Entscheidungen erst zur Laufzeit.

Sobald der `main`-Befehl ausgeführt wird, startet das Framework eine Kette von ressourcenintensiven Operationen:

1. Iteratives Classpath Scanning und Bytecode-Analyse:

Da zur Kompilierzeit nicht feststeht, welche Komponenten aktiv sein sollen, muss die Java Virtual Machine beim Start das gesamte Dateiarchiv rekursiv durchsuchen. Spring scannt dabei den rohen Bytecode tausender Klassen nach Annotationen (z. B. `@Service`). Dieser Vorgang skaliert negativ mit der Projektgröße. Je mehr Funktionen die Lernplattform erhält, desto träger und zeitaufwendiger wird dieser Suchvorgang bei jedem Start.

2. Dynamisches Bean-Wiring und Condition-Check:

Nach der Analyse muss Spring Boot entscheiden, welche Komponenten wie miteinander verknüpft werden (Dependency Injection). Dabei werden hunderte Bedingungen geprüft, wie beispielsweise ob eine Datenbank-Library vorhanden ist. Das Framework berechnet erst jetzt einen komplexen Abhängigkeits-Graphen. Ein Konfigurationsfehler wird oft erst am Ende dieser Kette bemerkt, was die Entwicklungszyklen im Vergleich zu proaktiveren Ansätzen verlängert.

3. Runtime Reflection und Proxy-Generierung:

Um essentielle Funktionen wie das Transaktionsmanagement zu ermöglichen, erzeugt Spring Boot zur Laufzeit mittels Reflection künstliche Stellvertreter-Klassen, auch *Proxies* genannt. Das ist nicht nur CPU-intensiv, sondern belegt auch im Arbeitsspeicher permanent Platz. Dies erklärt auch den hohen Memory-Footprint, der bereits im Leerlauf der Anwendung vorhanden ist. (ca. 250-400 MB).

Datenverwaltung und Entwicklerproduktivität im Projektkontext

Für den Erfolg der Lernplattform ist allerdings nicht nur die rein technische Laufzeit-performance ausschlaggebend, sondern auch die Effizienz während der Implementierungsphase. Spring Boot ist Teil des Spring-Ökosystems, und dieses bietet mit *Spring Data JPA* eine der ausgereiftesten Abstraktionen für den Datenbankzugriff. Komplexe Relationen lassen sich durch dieses Object-Relational-Mapping sehr stabil modellieren. Dies stellt einen wesentlichen Vorteil für die Datenintegrität dar, da die Geschäftslogik schon durch die strikte Typisierung von Java geschützt wird, bevor sie überhaupt die Persistenzschicht erreicht.

Trotz dieser Erleichterungen erfolgt bei der praktischen Einrichtung der Infrastruktur ein deutlicher administrativer Aufwand. Dabei folgt Spring Boot einem sehr statischen Konfigurationsansatz über die zentrale Datei `application.properties`. Um das in

einem konkreten Beispiel zu veranschaulichen: Das Framework kann zwar automatisch Objekte mit Tabellen verknüpfen, es kann allerdings nicht die benötigte Infrastruktur wie einen MySQL-Container selbständig verwalten. Daher muss jedes Teammitglied eine passende Datenbankumgebung starten und die Verbindungsparameter exakt denen in der Konfigurationsdatei angleichen.

Natürlich lässt sich der manuelle Setup-Aufwand durch den Einsatz von *Docker-Compose* standardisieren, indem die benötigte Datenbank-Infrastruktur in einer separaten Konfigurationsdatei definiert wird. Während der Entwicklung jedoch müssen die Zugangsdaten und Ports in der `docker-compose.yml` stets manuell mit jenen in `application.properties` synchronisiert werden.

Durch diese Redundanz zwischen Infrastruktur-Definition und Applikations-Konfiguration entsteht eine potenzielle Fehlerquelle. Da zwei Teams gleichzeitig am Code arbeiten, führen Änderungen an der Datenbankstruktur oder den Verbindungsparametern häufig zu „Connection-Refused“-Fehlern bei anderen Teammitgliedern, wenn bei diesen die lokale Umgebung nicht exakt synchronisiert wurde. Spring Boot bietet dafür keine native Integration, die den Lebenszyklus der Docker-Container während der Entwicklung steuert oder auf Strukturänderung im Code reagiert. [3]

Database Seeding via `import.sql` [6]

Ein wesentliches Kriterium für das Entwickeln eines Backends im Zusammenhang mit einer Datenbank ist das *Database Seeding*, also das automatisierte Befüllen der Datenbank mit Stammdaten wie Nutzerdaten und Themenbereiche. Spring Boot bietet hierfür primär den Mechanismus der `import.sql`. Beim Anwendungsstart liest das Framework diese Datei automatisch ein und führt deren SQL-Statements direkt gegen die Datenbank aus, sofern die Property `ddl-auto` auf `create` oder `create-drop` gesetzt ist.

In der praktischen Umsetzung für die Lernplattform offenbaren sich jedoch deutliche konzeptionelle Schwächen:

- **Mangelnde Refactoring-Sicherheit:** Es besteht keine Kopplung zum Java-Quellcode, wodurch eine Umbenennung eines Attributs im Code nicht zu einem Compiler-Fehler im SQL-Skript führt. Der Fehler tritt also erst zur Laufzeit auf, was die Fehlersuche erschweren kann.

- **Eingeschränkte Ausdrucksstärke:** Komplexe Anforderungen, wie das korrekte Hashen von Test-Passwörtern, sind über das SQL-Skript nicht möglich.
- **Kopplung an den Schema-Lebenszyklus:** Die `import.sql` ist strikt an das Neuerstellen der Tabellen gebunden. Das bedeutet, dass bei jeder Änderung an den Testdaten die gesamte Datenbankstruktur gelöscht und neu aufgebaut werden muss. In Kombination mit den bereits beschriebenen langen Startzeiten von Spring Boot führt dies zu einem noch trägeren Entwicklungszyklus, da kein inkrementelles Update der Daten möglich ist.

Fazit

Insgesamt zeigt sich also, dass die Flexibilität von Spring Boot durch einen hohen Preis erkauft wird. Das Framework benötigt einen signifikanten Teil des verfügbaren Arbeitsspeichers und der CPU-Zyklen für die eigene Verwaltung. Auf der oft begrenzten Infrastruktur einer Schule oder in kleinen Cloud-Instanzen wird dieser Overhead zum Problem, da Ressourcen für die Framework-Logik statt für die eigentliche Business-Logik der Lernplattform reserviert werden, wobei es auf solchen Container-Umgebungen wichtig ist, wirklich nur so viele Ressourcen zu verwenden, wie es auch notwendig ist.

Aufgrund dieses Preises für Flexibilität und des tatsächlichen Ressourcenverbrauchs müssen Alternativen evaluiert werden, die entweder ein schlankeres Laufzeitmodell verfolgen oder die Analyseprozesse komplett anders organisieren.

2.1.2 Node.js - Das ereignisgesteuerte Gegenmodell



Abbildung 5: Logo von Node.js

Historischer Kontext

Node.js wurde schon im Jahr 2009 von Ryan Dahl vorgestellt und gilt als Wendepunkt

in der Server-Programmierung. Zu dieser Zeit dominierten Frameworks wie das Spring Framework oder der Apache HTTP Server den Markt. Diese verwendeten das klassische „Thread-per-Request“-Modell. Das heißt für jeden Besucher, der eine Webseite aufrief, wurde ein eigener Thread im Arbeitsspeicher reserviert. Bei einer geringen Nutzerzahl funktionierte dieses Modell auch reibungslos, wenn man aber dann tausende gleichzeitige Verbindungen laufen hatte, verbrauchten diese Systeme den Großteil ihres Arbeitsspeichers nur für die Verwaltung der Threads, anstatt die eigentliche Logik auszuführen. [7]

Dabei erkannte Dahl, dass das Problem nicht bei der CPU lag, sondern bei dem Warten auf Ein- und Ausgabeprozesse wie Datenbankabfragen oder das Lesen von Dateien. Deshalb ersetzte er das blockierende Modell mit einer ereignisgesteuerten Architektur, welche er mit der extrem schnellen V8-JavaScript-Engine von Google Chrome kombinierte. Anstatt also auf eine Antwort der Datenbank zu warten und dabei einen Thread blockieren zu müssen, verwendet Node.js eine *Event-Loop*. Durch diese Architektur war es plötzlich möglich, tausende Verbindungen zu verarbeiten, und dabei nur einen Bruchteil des Arbeitsspeichers zu verwenden, den eine vergleichbare Java-Anwendung benötigt hatte. Das machte JavaScript, das zuvor fast nur im Browser für Animationen genutzt wurde, zu einer wettbewerbsfähigen Technologie für hochperformante Backends. [8]

Risiken des Single Threaded Ansatzes

Die enorme Effizienz von Node.js beruht auf der Annahme, dass der Server hauptsächlich mit „I/O-bound-tasks“, also mit dem Warten auf Lese- und Schreibvorgänge, beschäftigt ist. Da aber die gesamte Anwendung in einem einzigen Thread agiert, ergibt sich im Gegenzug ein Risiko bei rechenintensiven Aufgaben.

Anders als bei Spring Boot, bei welchem rechenintensive Operationen einfach auf einen freien Thread verlagert werden, würde bei Node.js die gesamte Event-Loop einfrieren. Diesen Zustand nennt man „Starvation“ und bei diesem kann der Server keine weiteren Anfragen mehr verarbeiten. Einfachste Funktionen, wie zum Beispiel der Login, blieben in diesem Fall unbeantwortet, bis die CPU-intensive Aufgabe abgeschlossen wurde. [9]

Obwohl Node.js oft als rein „single-threaded“ bezeichnet wird, agiert im Hintergrund eigentlich die C++ Bibliothek *libuv*, welche die zeitintensiven I/O-Operationen in einen internen Worker-Pool auslagert. Das ändert allerdings nichts am fundamentalen Risiko

für die Lernplattform: Da der eigentliche JavaScript-Code trotzdem strikt auf einem einzigen Haupt-Thread läuft, können komplexe Berechnungen innerhalb der Business-Logik nicht automatisch auf diesen Worker-Pool verteilt werden. Dafür benötigt es den expliziten Einsatz von *Worker-Threads* durch die Entwickler:innen, ohne welchen eine hohe CPU-Last im JavaScript-Code zu einer Blockade der Event-Loop führt, da *libuv* nur systemnahe Aufgaben, aber keine benutzerdefinierten Rechenoperationen automatisch parallelisiert. [10] [11]

Für die Lernplattform heißt das, dass eine strikte Trennung zwischen leichtgewichtigen API-Abfragen und schweren Hintergrundprozessen vorherrschen muss. Das wäre zum Beispiel durch Worker-Threads oder durch Microservices möglich, stellt aber auch einen zusätzlichen Architekturaufwand dar. Es ist also notwendig, diesen Aufwand gegen die Vorteile der schnellen Startzeiten gegenüber Spring Boot abzuwägen.

Datenverwaltung und Infrastruktur-Steuerung

Bei der Datenhaltung ist die Philosophie von Node.js in seiner Modularität deutlich erkennbar [12]. Während Java-Ökosysteme durch den JPA-Standard eine formalisierte und herstellerunabhängige Schicht erzwingen, ist der Datenbankzugriff in Node.js geprägt von spezialisierten Bibliotheken und vor allem für JSON-basierte Datenbanken entwickelt. Im Gegensatz dazu gibt es zwar moderne ORMs wie *Prisma* [13] oder *TypeORM* [14], jedoch fehlt diesen die jahrzehntelange Entwicklung und die daraus resultierende Reife und tiefe Integration, die Spring Boot durch die *Auto-Configuration* bietet.

Das bedeutet für die Lernplattform konkret, dass Node.js eine exzellente Anbindung an JSON-basierte Datenbanken, wie zum Beispiel MongoDB, ermöglicht, was für unstrukturierte Lernmaterialien durchaus vorteilhaft sein kann. Auf der anderen Seite erfordert die Nutzung von relationalen SQL-Systemen allerdings eine wesentlich stärkere Absicherung, da Node.js keine native Integration für komplexe Enterprise-Transaktionsmuster bietet.

Automatisierung und Entwicklungszyklus

Für die Effizienz im täglichen Entwicklungsprozesses ist vor allem der Grad der Automatisierung, den die gewählte Technologie bietet, ausschlaggebend. Dabei verfolgt Node.js allerdings einen sehr minimalistischen Ansatz. Während Spring Boot durch seine *Auto-Configuration* versucht, so viele Entscheidungen über die Infrastruktur als

möglich vorab zu treffen, liegt bei Node.js die Verantwortung fast vollständig bei den Entwickler:innen.

Das zeigt sich besonders in der Infrastruktur-Steuerung, wobei keine Mechanismen in Node.js eingebaut sind, um externe Abhängigkeiten, wie Datenbank-Container oder Mock-Services, zu koordinieren. Bei der parallelen Entwicklung von zwei verschiedenen Teams bedeutet das, dass die Synchronisation der lokalen Entwicklungsumgebungen manuell über externe Werkzeuge wie *Docker-Compose* erfolgen muss. Somit erfordert jede Änderung am Datenmodell oder an den Verbindungsparametern eine explizite Kommunikation zwischen den beiden Entwicklungsteams, da das Framework strukturelle Ungleichheiten erst zur Laufzeit bemerkt.

Ein wesentlicher Vorteil aber zeigt sich in einem erheblichen Gewinn an Agilität durch die Art und Weise, wie Node.js auf Code-Änderungen reagiert. Einer der größten Vorteile in der Verwendung dieses Frameworks ist die Schnelligkeit des Feedback-Zyklus. Durch den nativ eingebauten Watch-Modus entfallen die zeitintensiven Startzeiten, die bei Spring Boot durch das Neu-Kompilieren und Scannen des Classpaths entstehen und oft mehrere Sekunden in Anspruch nehmen. Änderungen am Quellcode werden nahezu in Echtzeit in der laufenden Anwendung reflektiert und es ist kein Neustart erforderlich. [15]

Dadurch können also in der Lernplattform iterative Anpassungen an der REST-API wesentlich schneller getestet werden und somit steigt die Effizienz des Entwicklungsprozesses durchaus. Diese Geschwindigkeit kommt allerdings trotzdem mit dem Nachteil, dass Node.js keine native, annotationsbasierte Validierung von Datenmodellen mitbringt. Deshalb müssen Entwickler:innen die repetitive Prüfungslogik manuell implementieren, was das Risiko für Inkonsistenzen in der Geschäftslogik erhöht.

Flexibilität beim Database Seeding

Im Database Seeding zeigt sich ein deutlicher funktionaler Unterschied. Während Spring Boot primär auf SQL-Dateien zum Importieren von Testdaten setzt, die unflexibel gegenüber Änderungen am Datenmodell sind und keine logischen Operationen erlauben, erfolgt das Befüllen der Datenbank in Node.js in der Regel programmatisch. Das hat den Vorteil, dass beispielsweise Passwörter für Test-User direkt während des Seedings gehasht werden können, was im statischen SQL-Ansatz nicht möglich ist. Außerdem nutzen diese Skripte das selbe Datenmodell wie die Anwendung selbst, wodurch strukturelle

Fehler in den Testdaten schon während der Entwicklung und nicht erst zur Laufzeit auffallen. [16]

Konfigurationsmanagement und Umgebungsvariablen

Bei der Konfiguration bietet Node.js kein direktes Äquivalent zur zentral verankerten `application.properties`-Datei von Spring Boot. Stattdessen setzt das Node.js-Ökosystem meist auf Umgebungsvariablen, die häufig über `.env`-Dateien verwaltet werden. [17]

Dieser Ansatz birgt allerdings wesentliche Nachteile im Vergleich zur strukturierten in Java für die Wartbarkeit. Während Spring Boot durch *Configuration Properties* die Werte aus der Einstellungsdatei direkt in typsichere Java-Objekte umwandelt und diese beim Start validiert, werden Umgebungsvariablen in Node.js als einfache Strings eingelesen. Dabei ist Spring Boot „Fail-Fast“, das heißt es stoppt sofort bei Fehlern, und Node.js ist oftmals „Lazy“, das heißt, dass Fehler erst auftreten, wenn der Code-Pfad erreicht wird. Dadurch werden Konfigurationsfehler, wie es zum Beispiel ein fehlender API-Schlüssel oder ein falsch formatierter Port wären, erst spät zur Laufzeit bemerkt und nicht schon beim Start der App abgefangen. Weiters fehlt Node.js ein eingebautes System für Profile wie `dev` oder `prod`. Somit müssen die Teams manuell sicherstellen, dass für jede Umgebung die passende `.env`-Datei geladen wird, was bei der parallelen Arbeit von zwei Teams die Fehleranfälligkeit bei der Bereitstellung der Teilprojekte erhöht.

Fazit

Node.js ist also ein hochperformantes Backend-Framework, allerdings nur spezifische Anwendungsfälle vollständig geeignet. Durch seine ereignisgesteuerte Architektur und die effiziente *Event-Loop* löst es das Ressourcen-Problem von klassischen Java-Anwendungen und bietet zugleich eine Agilität im Entwicklungsprozess, die sich durch nahezu sofortige Feedback-Zyklen auszeichnet. Vor allem beim Database-Seeding übertrifft Node.js durch die Flexibilität von programmatischen Skripten die statischen Ansätze von Spring Boot deutlich.

Dennoch ergeben sich für die Entwicklung der geplanten Lernplattform kritische Nachteile. Die Single-Threaded-Architektur stellt ein wesentliches Risiko für die zukünftige Erweiterbarkeit dar, da rechenintensive Operationen die gesamte Anwendung blockieren könnten. Ein schwerwiegenderes Problem ergibt sich allerdings im Mangel an Standardi-

sierung und nativer Typsicherheit innerhalb des Ökosystems. Da zwei Entwicklerteams an diesem Projekt arbeiten, würde die manuelle Verwaltung von Infrastruktur, Validierung und Konfiguration ein erhöhtes Fehlerrisiko bei der Integration bedeuten.

Somit stehen sich zwei Extreme gegenüber: Auf der einen Seite das robuste, aber ressourcenintensive Spring Boot und auf der anderen Seite das leichtgewichtige, aber architektonisch weniger gefestigte Node.js. Im Zuge dieser Analyse wird also ein weiteres Framework betrachtet. Im bestenfall bringt es die Stabilität, Typsicherheit und die bewährten Standards des Java-Ökosystems, aber auch die Effizienz und moderne Entwickler-Experience von Node.js zusammen. Dieses Anforderungsprofil führt zu **Quarkus**, einem Framework, das explizit als „Supersonic Subatomic Java“ entwickelt wurde, um diese Lücke zu schließen.

2.1.3 Quarkus



Abbildung 6: Logo von Quarkus

Historischer Kontext: Die Java-Krise in der Ära der Containerisierung

Um die Entstehung von Quarkus im Jahr 2019 zu verstehen, muss man zuerst die Geschichte von Java in Server-Umgebungen und den mit den Jahren zum Problem gewordenen Standards kennen. Über Jahrzehnte war Java die unangefochtene erste Wahl für Unternehmenssoftware. Frameworks wie Spring Boot in 2014 perfektionierten die Entwicklung von großen Microservices, die auf leistungsstarken Servern mit gigantischen Arbeitsspeicherkapazitäten liefen. Dabei war Java darauf optimiert, einmal zu starten und über mehrere Wochen hinweg durch den Just-In-Time Compiler [18] im laufenden Betrieb immer mehr an Schnelligkeit zu gewinnen.

Mit der Zeit verschob sich der Standard allerdings immer mehr in Richtung Containerisierung und damit kam der Aufstieg von Docker-Containern und Kubernetes. So änderten sich die Anforderungen fundamental: In modernen Cloud-Umgebungen werden Anwendungen nicht mehr monatelang betrieben, sondern wenn nötig im Bruchteil einer Sekunde hochgefahren und bei Nichtgebrauch sofort wieder abgeschaltet, um Kosten zu sparen. In dieser neuen Welt, in der Minimalverbrauch von Ressourcen eine entscheidende Anforderung wurde, verlor Java zunehmend seinen bisherigen Monopolstatus.

Für viele Expert:innen war dies ein Signal, dass Java nun seinem Ende zuneigte, wenn es um moderne Cloud-Anwendungen geht. Während also Entwicklerteams massenhaft zu Node.js oder Go wechselten, startete aber ein Team bei Red Hat ein ehrgeiziges Projekt mit dem Ziel, Java nicht durch eine neue Sprache zu ersetzen, sondern fundamental so umzubauen, dass es „Cloud-Native“ wird.

Im März 2019 wurde Quarkus als Open-Source-Projekt veröffentlicht. Dabei leitet sich der Name aus der Physik ab, wobei ein Quark der kleinste Baustein der Materie ist und „us“ für die Community steht. Mit dem Slogan „Supersonic Subatomic Java“ war es ihnen also wichtig, einen extrem geringen Speicherverbrauch zu erzielen, der es erlaubt, viel mehr Container auf der selben Hardware zu betreiben als zuvor möglich war, sowie die Startzeiten auf einen wettbewerbsfähigen Stand für moderne Cloud-Architekturen zu reduzieren. [19]

Die technische Revolution: Build-Time Augmentation

Einer der entscheidendsten Faktoren für die Schnelligkeit von Quarkus liegt in der Verschiebung des „Bootstrapping“-Prozesses. Wie bereits bei Spring Boot analysiert, verbringen klassische Frameworks beim Start viel Zeit damit, eine Art „Bestandsaufnahme“ zu machen. Dabei durchsuchen sie tausende Dateien nach Annotationen, bauen komplexe Abhängigkeitsbäume auf und generieren dynamisch Proxies im Arbeitsspeicher. Dieser Prozess findet bei jedem erneuten Starten statt, egal ob auf der Entwickler-Umgebung oder auf dem Cloud-Server.

Im Gegensatz dazu nutzt Quarkus einen Ansatz, den man als „*Shift-Left*“ der Infrastruktur-Logik bezeichnet. Das Framework verlagert also die Intelligenz von der Runtime in die Build-Time. Während also bei Spring Boot beim Start alle Parameter aufs Neue analysiert werden müssen, passiert dies bei Quarkus nur einmal, und zwar beim Kompilieren.[20]

In der Tiefe passiert dabei folgendes:

- **Vorausblickendes Metadaten-Parsing:**

Anstatt zur Laufzeit mühsam nach Annotationen wie `@Inject` oder `@Path` zu suchen, erledigt Quarkus dies einmalig während des Kompilierens. Die Ergebnisse werden in einem effizienten Format gespeichert, das beim Start direkt gelesen werden kann.

- **Vorbereitung der Reflection:**

Reflection, also das dynamische Untersuchen von Klassen zur Laufzeit, ist in Java extrem mächtig, aber auch extrem langsam und speicherintensiv. Quarkus minimiert den Einsatz von Reflection also, indem es den notwendigen Code für die Verknüpfung von Komponenten bereits während des Build-Vorgangs direkt in den Bytecode schreibt.

- **Pre-Bootig der Framework-Logik:**

Viele Framework-Komponenten wie beispielsweise Hibernate werden von Quarkus schon während des Baus „vor-initialisiert“. Alle Entscheidungen, die nicht von Laufzeit-Informationen wie Passwörtern abhängen, sind beim Start der Applikation bereits getroffen.

- **Dead-Code Elimination:**

Quarkus führt beim Bauen eine vollständige Analyse durch. Dadurch kann es präzise bestimmen, welche Teile der eingebunden Bibliotheken niemals aufgerufen werden. Dieser „Dead-Code“ wird entfernt, was die resultierende Datei verkleinert und Speicherbedarf reduziert.

Das Ergebnis dieser „Augmentation“ ist eine Anwendung, die vor dem Start schon fast alle Entscheidungen getroffen hat und somit lediglich den bereits vorbereiteten Plan ausführen muss. Dadurch wurden Startzeiten von unter zwei Sekunden auf der Standard-JVM möglich. [21]

Kritische Betrachtung des Build-Time-Ansatzes

Aus dieser radikalen Verschiebung der Prozesse entsteht allerdings folglich auch ein klassischer technologischer Kompromiss. Da Quarkus den Großteil der Analysearbeit, die normalerweise erst beim Starten der Applikation anfällt, in den Kompilierungsablauf vorzieht, erhöht sich die Build-Zeit auch spürbar. Die Anwendung ist im Betrieb zwar „subatomar“ und schnell, jedoch müssen Entwickler:innen beim finalen Bau des Projekts beispielsweise in einer CI/CD-Pipeline mit längeren Wartezeiten als bei Spring Boot oder Node.js rechnen.

Da Quarkus außerdem beim Bauen bereits den gesamten Code-Pfad kennen muss, um ihn optimieren zu können, ist zum Beispiel das Nachladen von Modulen zur Laufzeit stark eingeschränkt. Außerdem ist das Ökosystem trotz rasantem Wachstum nicht so umfangreich wie das von Spring Boot. Deshalb kann auch nicht jede beliebige

Java-Bibliothek ohne Weiteres effizient genutzt werden, sondern braucht eine spezielle *Quarkus-Extension* [22], die diese für den Build-Time-Prozess vorbereitet. [20]

Für die Entwicklung der Lernplattform bedeutet dieser technologische Kompromiss, dass die Priorität bewusst auf die Stabilität und Effizienz der Laufzeitumgebung gelegt wird. Da das interaktive Lernsystem auf schnelle Antwortzeiten angewiesen ist, um ein flüssiges Lernerlebnis zu gewährleisten, ist die investierte Zeit beim Kompilieren ein vorteilhafter Tausch. Die Einschränkungen bei der dynamischen Modulnachladung sind im Zuge dieser Diplomarbeit vernachlässigbar, da die Architektur der Lernplattform auf einem fest definierten Datenmodell basiert, das von der statischen Analyse von Quarkus profitiert.

Infrastruktur-Automatisierung durch Dev Services

Ein entscheidender Vorteil von Quarkus für die Zusammenarbeit im Team ist die Einführung der sogenannten *Dev Services*. In der vorherigen Analyse von Node.js und Spring Boot wurde die manuelle Koordination von Datenbank-Infrastruktur als potenzielle Fehlerquelle identifiziert, da Zugangsdaten und Container-Zustände zwischen den Teams manuell synchronisiert werden müssen.

Dieses Problem löst Quarkus durch eine tiefe Integration von *Testcontainers*. Wenn das Framework während der Entwicklung feststellt, dass eine Datenbank-Erweiterung vorhanden, aber keine Verbindung konfiguriert ist, so startet es automatisch den entsprechenden Docker-Container im Hintergrund. Dabei übernimmt Quarkus die vollständige Konfiguration der Verbindungsparameter. Das bedeutet für die zwei Teams der Diplomarbeit eine „Zero-Configuration“-Umgebung, also eine Umgebung, in der jedes Teammitglied das Backend ohne vorheriges manuelles Aufsetzen der Datenbank und Abgleichen der Passwörter starten kann. Dies eliminiert die bei Node.js kritisierten „Connection-Refused“-Fehler und sorgt dafür, dass beide Teilprojekte stets auf einer identischen und automatisierten Infrastruktur arbeiten. [23]

Datenbankanbindung und automatisierte Datenverwaltung

Quarkus setzt bei der Handhabung mit Daten auf *Hibernate ORM* mit der möglichen Erweiterung von *Panache*. Während klassische Java-Frameworks oft sehr umfangreichen Code für einfache Datenbankoperationen benötigen, reduziert Panache diesen Aufwand erheblich. [24]

- **Aktive Validierung und Konsistenz:**

Anders als bei Node.js, wo Datenbankänderungen oft erst zur Laufzeit zu Fehlern führen, ist in Quarkus eine strikte Typsicherheit gegeben. Ändert ein Team das Datenmodell, so prüft Quarkus bereits beim Kompilieren, ob die Datenbankstruktur noch zum Java-Code passt. Dies verringert Inkonsistenzen zwischen den Teilprojekten.

- **Automatisiertes Database Seeding:**

Für die Lernplattform ist wichtig, dass nach dem Start sofort Testdaten zur Verfügung stehen. Quarkus unterstützt dies auf zwei Ebenen:

1. **SQL-basiert:** Durch eine einfache `import.sql`-Datei werden beim Start automatisch Tabellen befüllt.
2. **Programmatisch:** Quarkus ermöglicht es, Seeding-Logik direkt in Java zu schreiben. So können beispielsweise verschlüsselte Passwörter für Test-User oder komplexe Lerninhalte dynamisch beim ersten Start generiert werden.

Die zuvor beschriebenen Dev Services gemeinsam mit dieser automatisierten Datenverwaltung schaffen eine „Ready-to-Code“-Umgebung, bei der Entwickler:innen lediglich den Entwicklungsmodus starten und innerhalb von Sekunden eine laufende Datenbank inklusive aller benötigten Testdaten erhält, ohne eine einzige Zeile SQL manuell ausführen zu müssen.

Effizienz im Entwicklungszyklus: Live Coding und Hot Reload

Einer der zentralen Kritikpunkte an klassischen Java-Frameworks wie Spring Boot war in der Vergangenheit der langsame Feedback-Zyklus. Nach fast jeder Code-Änderung musste die Anwendung manuell neu gestartet werden, wodurch häufig Wartezeiten von zehn bis zwanzig Sekunden entstanden.

Quarkus eliminiert diesen Nachteil von Java durch ein hocheffizientes Live Code-System. Wird eine Datei im Quellcode verändert und die Änderung gespeichert, so erkennt Quarkus dies und tauscht nur die betroffenen Klassen im laufenden Betrieb aus. Bei vielen Node.js Konfigurationen wird hierfür der gesamte Server-Prozess neu gestartet. Bei Quarkus allerdings bleibt die JVM stabil, es wird nur die geänderte Geschäftslogik neu in den Speicher geladen. [25]

Dieser Vorgang geschieht in der Regel in weniger als einer Sekunde. Das bedeutet für die Entwicklung der Lernplattform, dass Änderungen an der REST-Schnittstelle oder

der Validierungslogik sofort getestet werden können, ohne dabei den Arbeitsfluss zu unterbrechen.

Zentrale Konfiguration und Sicherheitsaspekte Ein weiterer Faktor, der im Zuge der Analyse behandelt wird, ist die Verwaltung von Umgebungsvariablen und sensitiven Daten. Dabei nutzt Quarkus ähnlich wie Spring Boot eine zentrale Datei namen `application.properties`, die aber eine Besonderheit aufweist: das native Profil-Management. [26]

- **Trennung von Umgebungen:**

Anders als bei Node.js, wo man für unterschiedliche Umgebungen verschiedene Dateien nutzt, erlaubt Quarkus die Definition von Profilen innerhalb einer Datei mit Präfixen (`%dev.db.url` oder `%prod.db.url`). Dadurch wird also niemals versehentlich die Konfiguration für die lokale Entwicklung die Live-Datenbank mit den echten Materialdaten überschreibt.

- **Typsichere Konfiguration:**

Bei Node.js werden Konfigurationen meist als einfache Zeichenketten eingelesen. Da hingegen bietet Quarkus typsichere Konfigurationsobjekte und bricht den Startvorgang sofort mit klaren Fehlermeldungen ab, sollte ein wichtiger Parameter fehlen oder im falschen Format stehen. Somit fährt die Lernplattform nie in einem instabilen Zustand hoch.

- **Sicherheit und Credentials:** Für den Umgang mit Passwörtern und API-Keys unterstützt Quarkus die Einbindung von externen Quellen wie Umgebungsvariablen oder Key-Vaults. Das bedeutet, dass sensible Daten nicht im Quellcode stehen müssen, sondern sicher beim Start der Container-Umgebung injiziert werden können.

Dadurch ergibt sich ein robustes Konfigurationsmodell, das besonders bei der Arbeit in zwei Teams die Fehleranfälligkeit senkt, da die Konfigurationsstruktur durch das Java-Backend fest vorgegeben und validiert wird.

2.1.4 Fazit und finale Technologieentscheidung

Die Analyse verdeutlicht, dass die Wahl des Backend-Frameworks ein Abwägen zwischen Reife, Agilität und Ressourceneffizienz darstellt.

- **Spring Boot** weist zwar eine besondere technologische Reife auf, scheidet aber trotzdem aufgrund des hohen Overheads und der fehlenden nativen Infrastruktur-Automatisierung aus, da dies die parallele Arbeit der beiden Teams unnötig erschweren würde.
- **Node.js** löst zwar das Problem des langsamen Feedback-Zyklus in Spring Boot, birgt jedoch durch die fehlende Striktheit der Standardisierung und nativer Typsicherheit ein zu hohes Risiko für die Datenkonsistenz zwischen den Teilprojekten.
- **Quarkus** erweist sich als die optimale Lösung für die Lernplattform. Es vereint die Typsicherheit und Robustheit des Java-Ökosystems mit einer modernen Entwickler-Experience. Die administrativen Hürden, die bei den anderen Frameworks als kritisch identifiziert wurden, werden durch Features wie *Dev Services* und den *Hot Reload* eliminiert.

Somit ist die Entscheidung nicht nur aufgrund einer performanten Laufzeitumgebung auf Quarkus gefallen, sondern war strategisch für eine effiziente und fehlerresistente Zusammenarbeit im Team.

2.2 Frontend

Das Frontend bildet die visuelle und interaktive Ebene der Anwendung. Es ist dafür verantwortlich, komplexe Datenstrukturen benutzerfreundlich aufzubereiten und eine performante Bedienung zu ermöglichen. Im Fokus steht hierbei die Umsetzung einer modernen Oberfläche, die durch kurze Reaktionszeiten und eine klare Benutzerführung überzeugt.

Dabei muss das Frontend folgende projektspezifische Kernaufgaben erfüllen:

- **Dynamische Datenvisualisierung:** Aufbereitung von Lernfortschritten und Fehleranalysen in Form von interaktiven Statistiken, um den Lernerfolg visuell darzustellen.
- **Zustandsmanagement:** Effiziente clientseitige Verwaltung des Anwendungszustands, um einen flüssigen Wechsel zwischen verschiedenen Ansichten und Datenpools ohne spürbare Ladezeiten zu garantieren.

- **Schnittstellen-Konsumation:** Integration der REST-API zur asynchronen Datenverarbeitung, wobei die Kommunikation mit dem Server so optimiert wird, dass eine hohe Reaktivität der Anwendung erhalten bleibt.

Die technologische Wahl des Frameworks ist entscheidend für die Performance und die langfristige Wartbarkeit der Benutzeroberfläche. Im folgenden Vergleich werden die infrage kommenden Technologien gegenübergestellt.

Analyse technischer Anforderungen

Die Auswahl des Frontend-Stacks ist eine strategische Entscheidung, die bestimmt, wie effizient die Benutzeroberfläche gerendert wird und wie strukturiert der Code aufgebaut ist. Dabei wurden vor allem die Wartbarkeit und die Tooling-Unterstützung bewertet.

Folgende Punkte wurden dabei kritisch untersucht:

- **Rendering-Verfahren und DOM-Manipulation:** Untersuchung, wie das Framework Änderungen am UI vornimmt (z. B. Virtual DOM). Ziel ist eine hohe Performance, damit die Anwendung auch bei einer großen Anzahl an darzustellenden Elementen flüssig reagiert.
- **Ökosystem und Community-Support:** Bewertung der Verfügbarkeit von etablierten Bibliotheken (z. B. für Routing oder UI-Komponenten).
- **Entwickler-Produktivität und Tooling:** Analyse der verfügbaren Entwicklerwerkzeuge. Es wurde geprüft, wie gut Debugging-Möglichkeiten und Hot-Reload-Funktionen unterstützt werden, um den Entwicklungsprozess zu beschleunigen.
- **Styling und Design-System-Integration:** Vergleich der Möglichkeiten zur Gestaltung der Oberfläche. Hierbei steht die einfache Integration von modernen CSS-Lösungen im Vordergrund, um ein konsistentes Design-System umzusetzen.
- **Lernkurve und Syntax:** Bewertung der Komplexität des Frameworks. Da die Effizienz in der Umsetzung entscheidend ist, wurde untersucht, wie schnell die Konzepte des Frameworks sicher beherrscht werden können.

2.2.1 Angular



Abbildung 7: Logo von Angular

Historischer Kontext: Die Evolution vom Pionier zum Industriestandard

Um die heutige Rolle von Angular zu verstehen, muss man die fundamentale Krise der Web-Entwicklung um das Jahr 2010 betrachten. Damals dominierten statische Seiten oder mit jQuery „zusammengeflickte“ Interaktionen das Web. Google veröffentlichte 2010 *AngularJS* (Version 1.x), um Ordnung in dieses Chaos zu bringen. Die Vision war es, HTML um dynamische Attribute zu erweitern und das *Two-Way-Data-Binding* zu etablieren. Dies ermöglichte es erstmals, komplexe Single-Page-Applications (SPAs) mit vergleichsweise wenig Code zu erstellen.

Mit dem rasanten Wachstum des Webs und der Komplexität von Enterprise-Anwendungen stieß AngularJS jedoch an seine Grenzen. Die Performance litt unter dem massiven Datenabgleich, und die fehlende Typisierung von JavaScript führte in großen Teams zu schwer wartbarem Code. Anstatt das alte System nur zu flicken, traf Google 2016 eine radikale Entscheidung: Der vollständige Rewrite unter dem Namen Angular (Version 2+). Man wechselte von JavaScript zu *TypeScript*, implementierte eine strikte komponentenbasierte Architektur und ersetzte das langsame Data-Binding durch einen hocheffizienten Datenfluss. Dieser Bruch war so gewaltig, dass er keine Abwärtskompatibilität bot, aber er legte den Grundstein für ein hochstrukturiertes Enterprise-Framework, das heute den Goldstandard für komplexe Web-Plattformen darstellt.

Architektur-Paradigma: „Batteries Included“

Angular verfolgt einen fundamental anderen Ansatz als modulare Bibliotheken wie React. Es versteht sich als vollumfängliches Framework, das bereits im Kern Lösungen für Routing, Formularvalidierung und HTTP-Kommunikation mitliefert. Für die Lernplattform bedeutet dies eine hohe Konsistenz: Da zwei Teams an unterschiedlichen Teilbereichen arbeiten, erzwingt Angular durch seine strikte, komponentenbasierte Architektur eine einheitliche Struktur. Jedes UI-Element wird in einer fest definierten Trias aus TypeScript-Logik, HTML-Template und CSS-Styling gekapselt. Dies verhin-

dert zwar individuelle Gestaltungsfreiheiten in der Codestruktur, garantiert aber, dass das Projekt auch bei steigender Anzahl an Lernmodulen wartbar bleibt.

Technisches Herzstück: RxJS und das reaktive State-Management

Ein wesentliches Kriterium für die Lernplattform ist die effiziente Verwaltung des Anwendungszustands (State-Management), um flüssige Übergänge zwischen Fragenkatalogen zu ermöglichen. Angular setzt hierbei konsequent auf *RxJS* (Reactive Extensions).

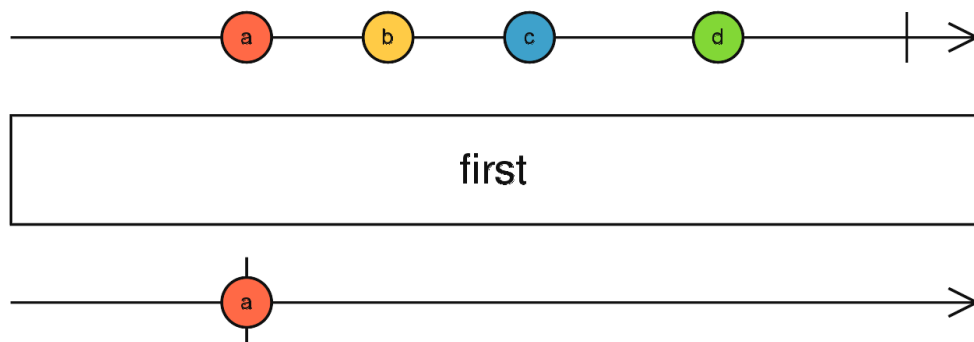


Abbildung 8: Funktionsweise von RxJS-Observables in der Datenverarbeitung

Abbildung 8 verdeutlicht das Prinzip der asynchronen Datenströme, auf denen Angular basiert. Anstatt Daten einfach als statische Variablen zu speichern, werden sie als asynchrone Datenströme (*Observables*) behandelt. Die Grafik zeigt die Transformation eines Eingabestroms durch verschiedene Operatoren (wie `Filter` oder `Map`), bevor das Ergebnis an den Abonnenten (*Subscriber*) – in diesem Fall die Benutzeroberfläche – weitergegeben wird. Dies führt zu einer hohen Reaktivität der Oberfläche:

1. **Asynchrone Schnittstellen-Verarbeitung:** Wenn die Lernplattform Statistiken oder neue Fragen vom Backend lädt, sorgt RxJS dafür, dass die UI nicht blockiert. Daten werden gestreamt und die Ansicht aktualisiert sich automatisch, sobald die Antwort eintrifft.
2. **Vermeidung von Inkonsistenzen:** Durch Operatoren wie `switchMap` oder `debounceTime` lassen sich komplexe Nutzerinteraktionen so steuern, dass alte, noch laufende Anfragen automatisch verworfen werden. Dies schont die Ressourcen des Clients und verhindert die Anzeige veralteter Datenstände.

Performance-Analyse: Change Detection und Rendering

Im Vergleich zu anderen Frameworks nutzt Angular eine „Dirty Checking“-Strategie für die Change Detection. Das Framework überwacht den Zustand der Anwendung und prüft bei Ereignissen, ob sich Daten geändert haben, die ein Re-Rendering erfordern.

In einer komplexen Lernumgebung mit vielen interaktiven Elementen (z. B. Wissensstand-Indikatoren) kann dies jedoch zu Performance-Einbußen führen:

- **Rechenintensität:** Standardmäßig prüft Angular den gesamten Komponentenbaum. Je mehr Funktionen die Plattform erhält, desto mehr CPU-Zyklen werden verbraucht.
- **Optimierung durch OnPush:** Um eine flüssige Bedienung auf schwächeren Endgeräten sicherzustellen, kann auf die `OnPush`-Strategie umgestellt werden. Dies erfordert jedoch ein tiefes Verständnis der Objekt-Immutabilität, was die Lernkurve für das Team erhöht.

Entwickler-Produktivität: Angular CLI und Material

Wie bei Quarkus im Backend spielt auch im Frontend das Tooling eine entscheidende Rolle. Das *Angular CLI* automatisiert das Scaffolding (Erstellen von Komponenten, Services) und stellt sicher, dass alle Teammitglieder dieselbe Projektstruktur verwenden.

Für die visuelle Umsetzung bietet *Angular Material* eine tief integrierte Komponentenbibliothek. Während dies die Entwicklung von Standard-Oberflächen (Buttons, Navigation) extrem beschleunigt, entsteht eine Abhängigkeit vom Material Design. Individuelle Design-Anpassungen sind oft komplex und erfordern tiefe Eingriffe in die CSS-Spezifität.

Laufzeit-Overhead: AOT-Compilation vs. Bundle-Größe

Ein kritischer Punkt ist das resultierende „Bundle“, also die Dateigröße, die der Browser beim ersten Aufruf laden muss. Da Angular ein „Vollsortimenter“ ist, bringt es viel Code für Funktionen mit, die eventuell gar nicht überall benötigt werden.

Zwar optimiert der *Ahead-of-Time (AOT) Compiler* den Code bereits während des Build-Prozesses, dennoch bleibt Angular im Vergleich zu leichtgewichtigeren Alternativen massiver. Für die Lernplattform bedeutet dies:

- **Längere Initial-Ladezeiten:** Besonders bei schlechten Mobilfunkverbindungen kann der erste Start der Anwendung träge wirken.
- **Komplexes Debugging:** Der optimierte Code ist im Browser schwerer zu lesen, was die Fehlersuche in der Produktionsumgebung erschwert.

Fazit

Angular bietet für ein Projekt mit zwei Teams eine enorme Stabilität durch seine strikten Vorgaben. Die „Batteries Included“-Philosophie minimiert Entscheidungsaufwände bei der Wahl von Bibliotheken. Dieser Komfort wird jedoch durch eine sehr steile Lernkurve (RxJS) und einen spürbaren Overhead bei der Bundle-Größe erkaufte. Für die Lernplattform bietet es die sicherste Basis für langfristige Wartbarkeit, erfordert aber Disziplin bei der Performance-Optimierung.

2.2.2 Flutter



Abbildung 9: Logo von Flutter

Historischer Kontext: Vom Experiment zur plattformübergreifenden Revolution Um die Entstehung von Flutter zu verstehen, muss man die technologische Sackgasse betrachten, in der sich die mobile App-Entwicklung Mitte der 2010er Jahre befand. Entwickler mussten sich zwischen performanten, aber teuren nativen Apps (Swift/Java) oder plattformübergreifenden Web-Wrappern (Cordova/PhoneGap) entscheiden, die jedoch oft träge reagierten und keine hochwertige User Experience boten.

Google stellte Flutter erstmals 2015 unter dem Codenamen „Sky“ auf der Dart Developer Summit vor. Die ursprüngliche Motivation war radikal: Man wollte testen, ob eine Rendering-Engine 120 Bilder pro Sekunde erreichen kann, wenn man den Ballast der traditionellen Web-Technologien (wie das DOM-Modell) komplett abwirft. Flutter entwickelte sich schnell von einem internen Google-Experiment zu einem ernsthaften Herausforderer für Frameworks wie React Native. Während Letzteres weiterhin auf eine „Bridge“ (Brücke) angewiesen war, um JavaScript-Befehle in native System-Aufrufe

zu übersetzen – was oft zu Performance-Flaschenhälsen führte –, implementierte das Flutter-Team eine eigene Grafik-Engine.

Mit der Veröffentlichung von Flutter 1.0 im Jahr 2018 vollzog Google den Schritt zum universellen UI-Toolkit. Die Vision weitete sich: Flutter sollte nicht mehr nur mobile Apps bedienen, sondern als „Portable UI Framework“ überall dort funktionieren, wo Pixel gezeichnet werden können. Dieser Weg gipfelte 2021 in der stabilen Unterstützung für das Web. Für die Lernplattform bedeutet dies den Einsatz einer Technologie, die nicht aus der Notwendigkeit heraus entstand, Webseiten dynamischer zu machen (wie Angular), sondern mit dem Ziel, die performanteste grafische Darstellung über alle Systemgrenzen hinweg zu standardisieren.

Technologischer Ansatz: Everything is a Widget Flutter bricht mit der traditionellen Web-Entwicklung, die auf HTML-Tags und CSS-Regeln basiert. Stattdessen folgt es einem rein deklarativen Ansatz, bei dem die gesamte Benutzeroberfläche aus verschachtelten Objekten, sogenannten *Widgets*, besteht. Für die Lernplattform bedeutet dies eine radikale Vereinfachung des UI-Codes: Ein Button, eine Statistik-Karte oder ein ganzer Fragenkatalog sind strukturell identisch aufgebaut. Diese Konsistenz erlaubt es, Layouts extrem schnell zu prototypisieren, da Flutter eine riesige Bibliothek an vorgefertigten „Material Design“- und „Cupertino“-Widgets mitliefert, die pixelgenau auf allen Endgeräten identisch aussehen.

Rendering-Paradigma: Die Skia-Engine im Web Der entscheidende technische Unterschied zu Frameworks wie Angular liegt im Rendering-Prozess. Während Angular den DOM (Document Object Model) des Browsers manipuliert, nutzt Flutter im Web standardmäßig das *CanvasKit* (basierend auf der Skia-Engine).

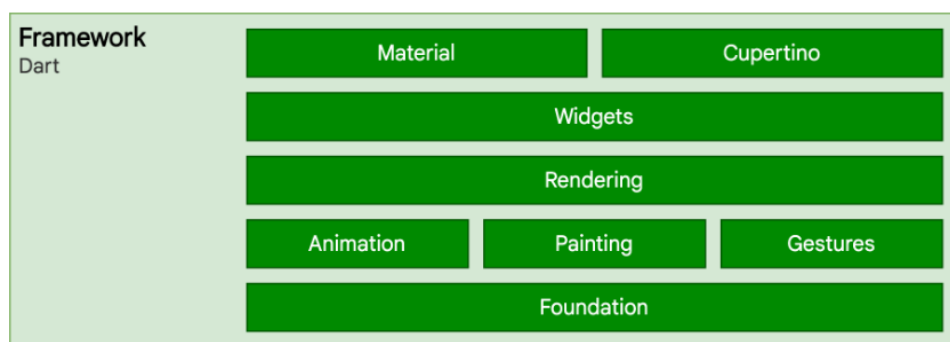


Abbildung 10: Architektur-Ebenen und Rendering-Pipeline von Flutter

Abbildung 10 verdeutlicht den Schichtaufbau von Flutter, der diesen Rendering-Prozess ermöglicht. Das *Framework* (oben) basiert auf der Sprache Dart und stellt die Reaktivität sowie die Widget-Bibliotheken bereit. Darunter liegt die *Engine*, die mithilfe von Skia das eigentliche Zeichnen übernimmt. Der *Embedder* fungiert schließlich als Bindeglied zur jeweiligen Plattform (in diesem Fall dem Browser). Dieser Aufbau hat direkte Auswirkungen auf die Performance der Lernplattform:

1. **Grafische Freiheit vs. Overhead:** Da Flutter die UI selbst auf ein HTML-Canvas „zeichnet“, können hochkomplexe Animationen (z. B. flüssige Übergänge bei der Auswertung von Lernerfolgen) ohne Rücksicht auf Browser-Inkonsistenzen realisiert werden. Der Preis dafür ist jedoch der Download der Engine: Der Browser muss beim ersten Start mehrere Megabyte an WebAssembly-Code laden, bevor die erste Interaktion möglich ist.
2. **SEO und Barrierefreiheit:** Da der Inhalt der Lernplattform für den Browser lediglich ein grafisches Element (Canvas) darstellt, ist die automatische Indexierung durch Suchmaschinen oder das Auslesen durch Screenreader deutlich erschwert. Für eine geschlossene Lernumgebung mag dies zweitrangig sein, für die allgemeine Auffindbarkeit von Lerninhalten ist es jedoch ein struktureller Nachteil.

Zustandsverwaltung: BLoC und Provider im Vergleich Um die Interaktivität der Lernmodi (z. B. das Umschalten zwischen Übung und Statistik) zu steuern, bietet Flutter verschiedene Muster für das State-Management an. Besonders relevant für die Lernplattform sind der *Provider* und das *BLoC*-Muster (Business Logic Component).

- **BLoC (Business Logic Component):** Trennt die UI strikt von der Logik mittels *Streams*. Ein Klick auf eine Antwort sendet ein Event an den BLoC, dieser verarbeitet die Antwort (z. B. Validierung gegen das Backend) und gibt einen neuen Status an die UI zurück. Dies bietet eine exzellente Testbarkeit, erhöht aber die Komplexität des Codes durch viel „Boilerplate“-Code massiv.
- **Provider:** Ein leichtgewichtigerer Ansatz, der Daten über den Komponentenbaum hinweg zur Verfügung stellt. Für die schnelle Implementierung von Materialverwaltungen ist dies oft effizienter, stößt aber bei sehr komplexen, ineinander verschachtelten Zuständen an seine Grenzen.

Entwicklerproduktivität: Hot Reload und Dart Ein wesentlicher Faktor für die Umsetzungsgeschwindigkeit der Diplomarbeit ist das *Hot Reload*-Feature. Änderungen am UI-Code werden in Millisekunden in der laufenden Anwendung sichtbar, ohne dass der aktuelle Lernfortschritt oder die Position in einem Fragenkatalog verloren geht. Dies verkürzt die Iterationszyklen im Vergleich zu Angular oder klassischen Java-Build-Prozessen im Backend drastisch.

Die zugrunde liegende Sprache *Dart* bietet zudem eine moderne Syntax, die Konzepte von Java und JavaScript vereint. Die starke Typisierung hilft dabei, die Datenmodelle des Backends (z. B. Fragen-Objekte) sicher im Frontend abzubilden, auch wenn die Tooling-Landschaft für Dart im Web-Bereich noch nicht die Reife von TypeScript erreicht hat.

Schnittstellen und Web-Integration Obwohl Flutter als „Multi-Platform“-Toolkit beworben wird, zeigt sich in der Praxis eine Optimierung auf mobile Endgeräte. Der Zugriff auf Web-spezifische Funktionen (wie fortgeschrittene Browser-APIs oder die Integration spezieller JavaScript-Bibliotheken für komplexe Diagramme) ist oft umständlicher als bei nativen Web-Frameworks. Für die Lernplattform bedeutet dies, dass bei der Einbindung von Drittanbieter-Tools (z. B. für PDF-Viewer oder spezielle mathematische Formeln) oft aufwendige „Platform Channels“ oder unfertige Community-Plugins zurückgegriffen werden muss.

Fazit Flutter glänzt durch seine enorme Entwicklungsgeschwindigkeit bei der UI-Erstellung und die garantierte visuelle Konsistenz. Das „Everything is a Widget“-Konzept ist intuitiv und das Hot-Reload-Feature spart wertvolle Zeit in der Implementierungsphase. Jedoch erkaufte man sich diese Vorteile im Web-Kontext durch eine schlechtere initiale Performance (Bundle-Größe) und eine schwächere Integration in das native Web-Ökosystem. Für eine Anwendung, deren Fokus primär auf einer schnellen, interaktiven Bedienung liegt und die eventuell später als mobile App veröffentlicht werden soll, bietet Flutter eine starke Basis – sofern die Nachteile beim initialen Laden akzeptabel sind.

2.2.3 React



Abbildung 11: Logo von React

Historischer Kontext: Paradigmenwechsel durch Facebook React wurde 2011 intern bei Facebook entwickelt, um die massiven Herausforderungen bei der Synchronisierung von Daten in der komplexen Facebook-Timeline zu bewältigen. Der eigentliche Auslöser war eine „Zustands-Hölle“ innerhalb der Facebook-Chat- und Benachrichtigungs-Funktion: Oft zeigte das System eine ungelesene Nachricht an, obwohl diese bereits im Chat-Fenster geöffnet war. Die damaligen Ansätze des *Imperative Programming* und das *Two-Way-Binding* machten es fast unmöglich, den Status der UI über verschiedene komplexe Widgets hinweg konsistent zu halten.

Jordan Walke, ein Softwareingenieur bei Facebook, entwickelte daraufhin *FaxJS* (den Vorläufer von React), beeinflusst von funktionalen Programmierkonzepten. Das Ziel war es, die UI einfach als eine Projektion des aktuellen Zustands zu betrachten: Ändert sich der Zustand, wird die UI „einfach neu gerendert“. Zu einer Zeit, als die meisten Frameworks (wie das frühe AngularJS) versuchten, Logik und Darstellung durch komplexes Two-Way-Binding zu verbinden, führte React 2013 auf der *JSConf US* das Konzept des „One-Way Data Flow“ und den „Virtual DOM“ ein. Die anfängliche Skepsis der Entwicklergemeinschaft gegenüber *JSX* (HTML in JavaScript) wich schnell der Erkenntnis, dass dieser Paradigmenwechsel – weg von der direkten Manipulation des Browsers hin zu einem rein zustandsgesteuerten UI-Modell – die moderne Webentwicklung effizienter und vorhersehbarer macht. Für die Entwicklung von LearnHub bedeutet der Einsatz von React den Zugriff auf eine Technologie, die über ein Jahrzehnt hinweg für extrem datenintensive Anwendungen optimiert wurde und die größte Community im Web-Bereich vorweisen kann.

Philosophie: Minimalismus und „Library first“

Im Gegensatz zu Angular oder Flutter ist React keine vollständige Framework-Lösung, sondern eine Bibliothek, die sich primär auf die Darstellungsschicht (View) konzentriert. Dieser minimalistische Ansatz bietet maximale Freiheit bei der Wahl zusätzlicher Werk-

zeuge für Routing oder API-Anbindungen. Für die Lernplattform bedeutet dies jedoch auch einen höheren Entscheidungsaufwand: Da zwei Teams am Projekt arbeiten, müssen Konventionen für die Ordnerstruktur und externe Bibliotheken explizit vereinbart werden, da React selbst – anders als Spring Boot im Backend – kaum Vorgaben macht („unopinionated“).

Technologischer Kern: Der Virtual DOM

Der entscheidende Performance-Vorteil von React liegt in der Verwendung eines *Virtual DOM*. Anstatt bei jeder Datenänderung (z. B. wenn ein Nutzer eine Antwortoption anklickt) die gesamte HTML-Seite im Browser neu zu berechnen, erstellt React eine virtuelle Kopie der Struktur im Arbeitsspeicher.

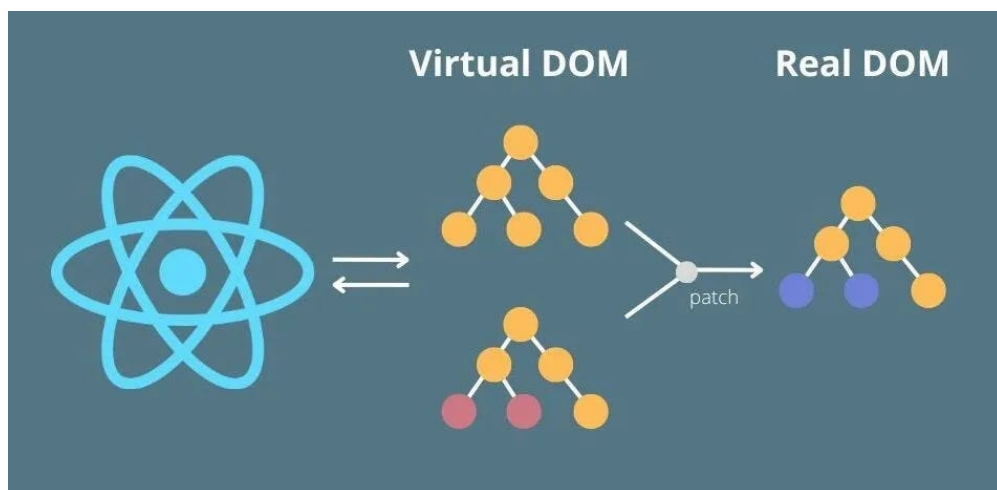


Abbildung 12: Visualisierung des Virtual DOM Diffing-Prozesses

Abbildung 12 illustriert diesen Kernprozess: Sobald sich der Zustand der Anwendung ändert, wird ein neuer *Virtual DOM* erstellt. React vergleicht diesen Snapshot mit der vorherigen Version (Diffing). Wie in der Grafik durch die farblichen Markierungen angedeutet, identifiziert das Framework präzise die veränderten Knotenpunkte. Nur diese minimalen Differenzen werden in einem finalen Schritt an den echten DOM des Browsers übertragen. Dieser Prozess wirkt sich direkt auf die Benutzererfahrung der Lernumgebung aus:

1. **Effizientes Diffing-Verfahren:** React vergleicht den virtuellen Zustand mit dem tatsächlichen Browser-Zustand und aktualisiert nur die minimal notwendigen Elemente (z. B. nur die Farbe des ausgewählten Buttons). Dies sorgt für eine extrem hohe Reaktivität, selbst auf leistungsschwachen mobilen Browsern.

2. **Deklarative Programmierung:** Entwickler beschreiben lediglich, wie die UI in einem bestimmten Zustand (z. B. „Frage beantwortet“) aussehen soll. React kümmert sich um die effiziente Umsetzung der Übergänge, was die Fehleranfälligkeit bei komplexen UI-Interaktionen reduziert.

Zustandsmanagement: Hooks und das Ein-Weg-Datenfluss-Prinzip

Ein zentrales Kriterium der Analyse ist das State-Management. React nutzt ein striktes „Unidirectional Data Flow“-Prinzip: Daten fließen immer von oben (Eltern-Komponente) nach unten (Kind-Komponente).

- **React Hooks (`useState`, `useEffect`):** Diese ermöglichen es, Logik und Zustand direkt in funktionalen Komponenten zu kapseln. Für die Lernplattform erlaubt dies eine sehr modulare Bauweise – so kann beispielsweise die Zeitmessung eines Prüfungsmodus als eigene Logik-Einheit (*Custom Hook*) geschrieben und überall wiederverwendet werden.
- **Prop-Drilling-Problem:** Bei tief verschachtelten Oberflächen (z. B. Dashboard -> Kursliste -> Themenpool -> Einzelfrage) müssen Daten oft durch viele Ebenen gereicht werden. Um dies zu vermeiden, ist der Einsatz der *Context-API* oder externer State-Manager wie *Redux* oder *Zustand* notwendig, was die Architektur komplexer macht.

Entwicklerproduktivität: JSX und das Ökosystem

React verwendet *JSX*, eine Syntaxerweiterung, die es erlaubt, HTML-Strukturen direkt im JavaScript-Code zu schreiben. Dies führt zu einer sehr engen Kopplung von Logik und Design, was die Lesbarkeit für erfahrene Entwickler erhöht, aber die Trennung der Verantwortlichkeiten aufbricht.

Das Ökosystem rund um React ist das größte im Frontend-Bereich. Für die Lernplattform bedeutet das den Zugriff auf tausende fertige Lösungen für Diagramme (z. B. *Recharts* für Statistiken) oder UI-Kits (z. B. *Tailwind CSS* oder *MUI*). Die Gefahr besteht jedoch in einer „Abhängigkeits-Hölle“: Da React für fast alles externe Bibliotheken benötigt, steigt das Risiko für inkompatible Versionen und Sicherheitslücken im Projektverlauf.

Rendering-Performance: Re-Rendering-Falle

Trotz der Schnelligkeit des Virtual DOM gibt es Ineffizienzen. Ohne explizite Optimierung (z. B. durch `React.memo` oder `useMemo`) rendert React oft Komponenten neu, obwohl sich deren relevante Daten gar nicht geändert haben. In einer datenintensiven Anwendung wie der Lernplattform, die viele Fortschrittsbalken und Statistiken gleichzeitig anzeigt, kann dies zu spürbarem „Lag“ führen, wenn der Abhängigkeits-Graph zu groß wird.

Fazit

React überzeugt durch seine Leichtgewichtigkeit und die enorme Flexibilität. Es bietet die beste Performance bei der Manipulation von Oberflächenelementen und verfügt über das reichhaltigste Ökosystem. Für das Projekt bietet es die Chance auf eine sehr maßgeschneiderte Lösung, erfordert aber aufgrund der fehlenden internen Standards (im Vergleich zu Angular) eine sehr disziplinierte Arbeitsweise und klare Absprachen zwischen den Teams, um eine konsistente Code-Basis zu erhalten.

2.2.4 Fazit und finale Technologieentscheidung

Die Gegenüberstellung der Frontend-Technologien zeigt, dass jedes Framework spezifische Schwerpunkte setzt, die die Entwicklung der Lernplattform unterschiedlich beeinflussen würden.

- **React** bietet zwar eine exzellente Rendering-Performance und maximale Flexibilität, erfordert jedoch aufgrund der fehlenden internen Standards (Library-Ansatz) einen hohen Abstimmungsaufwand zwischen den Teams. Das Risiko, in eine „Abhängigkeits-Hölle“ durch zu viele externe Bibliotheken zu geraten, wurde als kritisch für die Wartbarkeit eingestuft.
- **Flutter** besticht durch seine grafische Konsistenz und die schnelle UI-Entwicklung. Die Nachteile im Web-Kontext – insbesondere die hohen initialen Ladezeiten durch das Canvas-Rendering und die eingeschränkte SEO-Tauglichkeit – machen es jedoch zu einer weniger optimalen Wahl für eine primär browserbasierte Lernplattform.
- **Angular** erweist sich als die geeignetste Lösung für dieses Projekt. Die „Batteries Included“-Philosophie bietet eine stabile und vorstrukturierte Basis, die besonders bei der parallelen Arbeit an zwei Teilprojekten für die nötige Code-Konsistenz sorgt. Die tiefe Integration von TypeScript und RxJS ermöglicht zudem eine robuste

Handhabung komplexer asynchroner Datenströme, wie sie für die interaktiven Lernmodi erforderlich sind.

Neben den rein technischen Aspekten spielte die Praxiserfahrung des Projektteams eine entscheidende Rolle bei der Wahl. Da Angular bereits fester Bestandteil des Lehrplans im Unterrichtsfach „Softwareentwicklung“ an der HTL Leonding war, sind fundierte Vorkenntnisse in der Architektur und den Konzepten des Frameworks vorhanden.

Diese Vertrautheit mit dem Werkzeug minimiert die Einarbeitungszeit erheblich und ermöglicht es dem Team, sich von Beginn an auf die Umsetzung der komplexen Logik und der User Experience zu konzentrieren, anstatt grundlegende Framework-Konzepte neu erlernen zu müssen. Die Entscheidung für Angular ist somit eine strategische Wahl, um die begrenzte Zeit der Diplomarbeit effizient zu nutzen und gleichzeitig ein professionelles, typsicheres und skalierbares Endprodukt zu garantieren.

3 Praktische Umsetzung

3.1 Logische Programmstruktur und Use Cases

Der User Flow (Abbildung 13) visualisiert den typischen Interaktionsablauf der Schüler und Schülerinnen mit der Lernplattform. Dabei ist wichtig, welche Schritte der User von der Anmeldung bis zur Lernaktivität durchläuft und welche Handlungsmöglichkeiten ihm dabei zur Verfügung stehen.

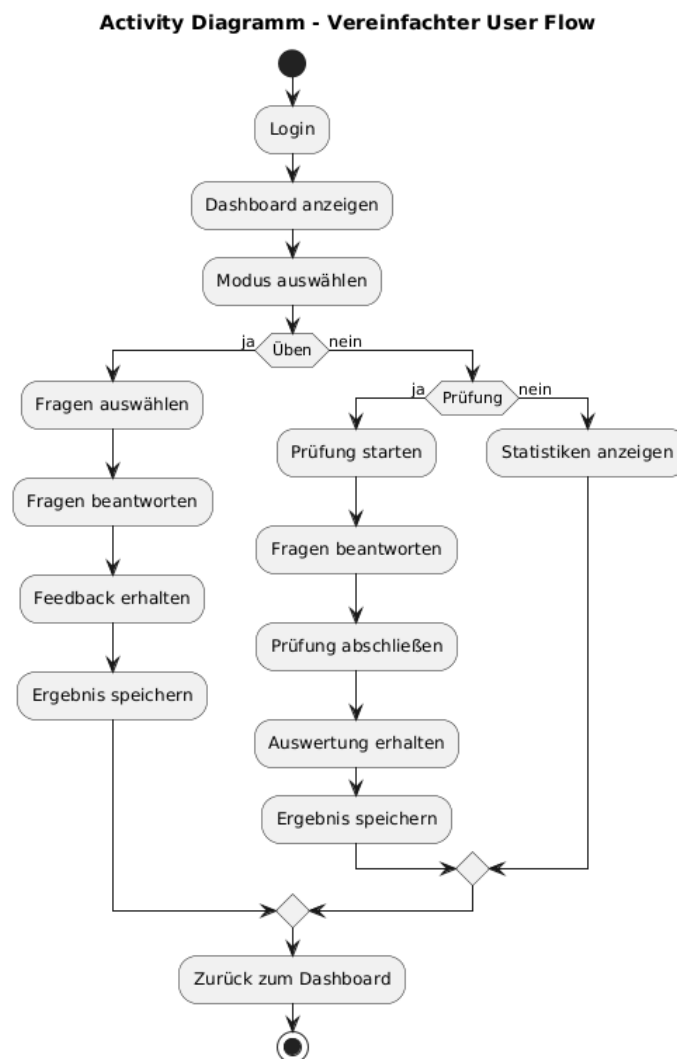


Abbildung 13: Ablaufdiagramm eines Users
[27]

Aus diesem User Flow lassen sich 3 zentrale Abläufe erkennen, die den Kernfunktionen der Lernplattform entsprechen:

- **Übungsmodus:**

Der Übungsmodus beginnt beim gezielten Stöbern in Themenbereichen und der Aufnahme von Fragen in einen persönlichen Fragenpool für systemgestützte Wiederholungen. Wichtig dabei ist der Lernprozess durch unmittelbares Feedback nach jeder Antwort, was die in Kapitel 1.4 untersuchte Lernstrategie durch die Persistierung des Wissenstands effektiv umsetzt.

- **Prüfungssimulationen:**

Bei den Prüfungssimulationen geht es um die Leistungsüberprüfung unter realitätsnahen Bedingungen, bei welchem der aktuelle Wissenstand auf die Probe gestellt werden kann. Nutzer und Nutzerinnen können hierbei frei die zu überprüfenden Themenbereiche, Fragenanzahl sowie das Zeitlimit konfigurieren. Im Gegensatz zum Übungsmodus erfolgt die Auswertung aber nicht nach jeder Antwort, sondern erst nach finaler Abgabe, um authentische Prüfungssimulationen zu gewährleisten.

- **Fortschrittsübersicht:**

In der Fortschrittsübersicht steht die Transparenz und Motivationssteigerung im Vordergrund. Dies wird durch die Visualisierung von täglichen Streaks, den Übungsstatus spezifischer Themenbereiche sowie den chronologischen Prüfungsverlauf erzielt. Dadurch wird eine objektive Einschätzung der Prüfungsreife ermöglicht und eine langfristige Lernbereitschaft durch die grafische Aufbereitung gefördert.

3.2 Startansicht

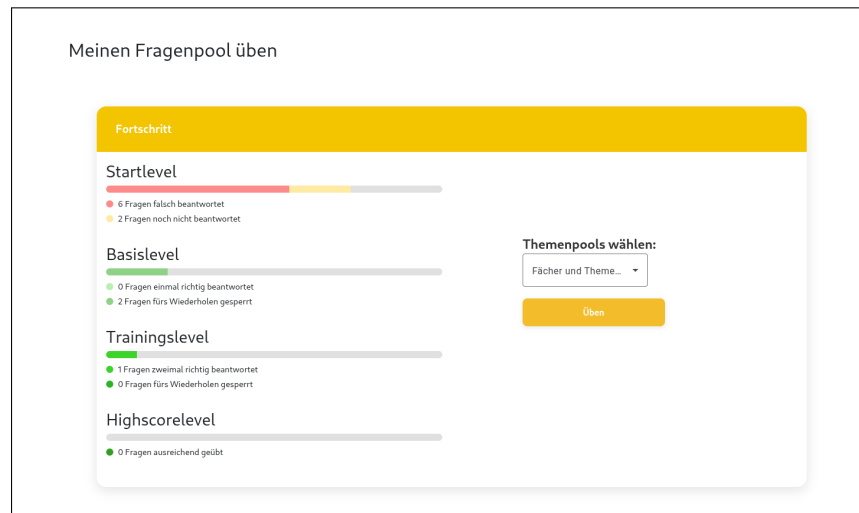


Abbildung 14: Visualisierung der Lernlevel auf der Startseite

Zum Einstieg in den Maturatrainer ist das zentrale Dashboard der Anwendung zu sehen. Sie bietet den Nutzer:innen einen direkten Einstieg in die Lernsession und visualisiert den aktuellen Wissensstand auf einen Blick.

3.2.1 Aufbau und Funktionen der Startseite

Die Startseite ist so gestaltet, dass der persönliche Lernfortschritt motivierend dargestellt wird. Die zentrale Komponente ist dabei die Anzeige der verschiedenen Lernstufen, die den Weg von einer neuen Frage bis hin zum gefestigten Wissen abbilden.

In Abbildung 14 ist die grafische Aufbereitung des Leitner-Systems zu sehen. Die Fragen des Nutzers werden dabei in vier aufeinanderfolgende Level unterteilt:

- **Startlevel:** Hier finden sich alle Fragen, die noch nicht bearbeitet wurden oder zuletzt falsch beantwortet wurden. Diese Fragen stehen jederzeit zum Üben bereit.
- **Basislevel:** In diesem Level befinden sich Fragen, die bereits einmal korrekt beantwortet wurden. Die Fragen die hier liegen, werden für einen Tag gesperrt. Dadurch wird verhindert, dass Nutzer:innen dieselben Fragen zu häufig hintereinander üben, was die Effektivität des Lernens beeinträchtigen könnte.
- **Trainingslevel:** In diesem Bereich befinden sich Fragen, die bereits zweimal korrekt beantwortet wurden. Diese Fragen werden für drei Tage gesperrt, um eine gezielte Wiederholung zu ermöglichen.

- **Highscorelevel:** Fragen in diesem Bereich wurden bereits mehrfach erfolgreich wiederholt. Sie gelten als „ausgelernt“ und signalisieren den Nutzer:innen, dass dieser Stoff sicher beherrscht wird.



Abbildung 15: Visualisierung der Sperrzeiten für Fragen im Basislevel

Interaktionsmöglichkeiten

Die Startansicht dient als Steuerungselement für den weiteren Lernweg:

- **Fortschrittskontrolle:** Über farbige Fortschrittsbalken (*Progressbars*) wird visualisiert, wie viel der Fragen sich in den jeweiligen Leveln befinden. Je mehr Fragen im Highscorelevel liegen, desto höher ist der Fortschritt. Die Anzeige unter den Balken zeigt die genaue Anzahl der Fragen in jedem Level an. Ebenfalls ist ersichtlich, wie viele davon bereit zum Wiederholen sind und welche momentan gesperrt sind.
- **Lernsession starten:** Durch einen Klick auf die „Üben“-Schaltfläche gelangen die Nutzer:innen direkt in den Übungsmodus. Die Auswahl der Fragen basiert auf der Auswahl seines eigenen Fragenpools und den Filteroptionen die sich auf die ausgewählten Themenpools beziehen.

Durch diese strukturierte Darstellung wird der Lernprozess greifbar gemacht. Die Nutzer:innen sehen ihren eigenen Zuwachs an Wissen durch die Visualisierung der Level, was den Lernprozess übersichtlich und motivierend gestaltet. Hier ist eine deutlich erweiterte und vertiefte Fassung des Kapitels 3.2.1. Ich habe die strategische Bedeutung der Vorbereitung, die psychologische Nutzerführung sowie die funktionalen Details detaillierter ausgearbeitet, um den Text gehaltvoller zu gestalten.

3.3 Fragenpool erstellen und verwalten

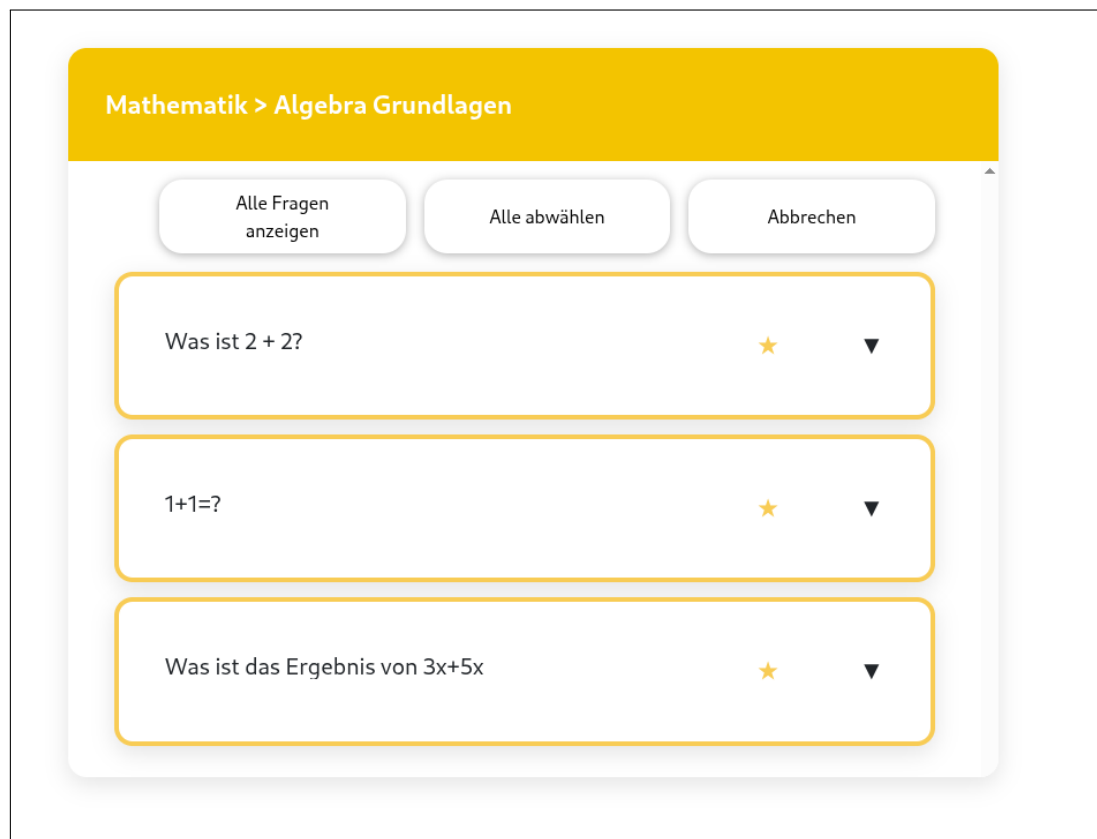


Abbildung 16: Zentrale Oberfläche zur individuellen Zusammenstellung des Fragenpools

Bevor der eigentliche Lernprozess initiiert werden kann, steht die strategische Konfiguration des persönlichen Lernbereichs im Vordergrund. Wenn der/die Nutzer:in im Partnerprojekt bereits selbst Fragen erstellt hat wurden diese schon automatisch in den Fragenpool übernommen.

3.3.1 Die „Fragen browsen“-Ansicht als digitaler Katalog

Die „Fragen browsen“-Ansicht fungiert als das Herzstück der Inhaltsverwaltung. Sie ist als interaktiver, digitaler Katalog konzipiert, der den Zugriff auf die umfangreiche Fragenauswahl der Plattform ermöglicht. Bei der Gestaltung dieser Ansicht wurde besonderer Wert auf eine klare visuelle Trennung zwischen Navigation und Inhaltspräsentation gelegt, um eine effiziente Exploration des Stoffes zu gewährleisten.

Das Interface ist für eine intuitive Bedienung in zwei funktionale Hauptbereiche unterteilt:

- **Strukturierte Navigations-Seitenleiste (Links):** Um die Komplexität der verschiedenen Lehrpläne abzubilden, nutzt die Seitenleiste eine hierarchische

Baumstruktur. Nutzer:innen finden hier eine geordnete Auflistung aller verfügbaren Hauptfächer. Durch ein interaktives Dropdown-System lassen sich diese Fächer in ihre jeweiligen Themengebiete und Unterkategorien aufgliedern. Diese Struktur erlaubt es, innerhalb weniger Klicks von einer globalen Fachansicht (z. B. Mathematik) zu einer spezifischen Problemstellung (z. B. Differenzialrechnung) zu navigieren. Die optische Hervorhebung des aktuell gewählten Pfades dient dabei als Orientierungshilfe.

- **Interaktiver Hauptbereich mit Vorschaufunktion (Rechts):** Im Fokus des Hauptfensters stehen die Fragenkarten. Jede Karte ist so gestaltet, dass sie bereits in der Kompaktansicht die wesentlichen Informationen vermittelt. Neben dem prägnanten Fragentext werden auch mathematische Formeln oder Grafiken direkt gerendert. Ein wesentliches Merkmal für die Qualitätssicherung der Auswahl ist die integrierte **Schnellvorschau**: Jede Fragekarte kann durch einen Klick erweitert werden, wodurch zusätzliche Details und ein „Üben“-Button erscheinen. Dies ermöglicht es den Nutzer:innen, eine Frage probenhalber zu bearbeiten, ohne den Kontext der Verwaltung verlassen zu müssen. So kann unmittelbar beurteilt werden, ob der Schwierigkeitsgrad oder der Inhalt der Frage der aktuellen Lernphase entspricht.

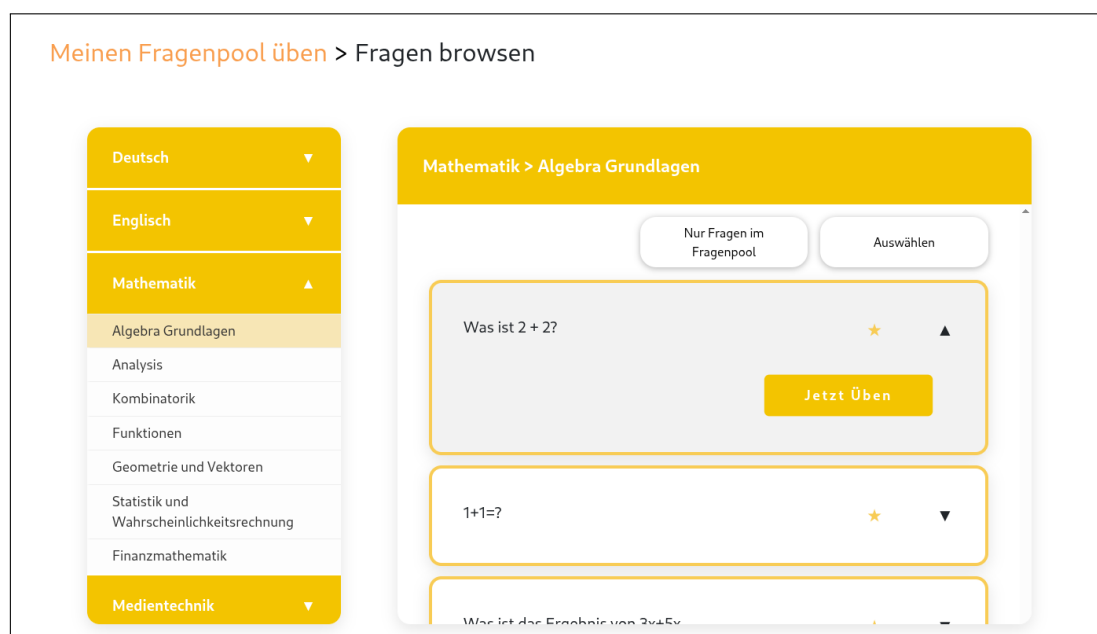


Abbildung 17: Oberfläche zum direkten Testen einer Frage aus der Browsing-Ansicht

3.3.2 Auswahl und individuelle Anpassung des Fragenpools

Die Architektur des Systems unterstützt verschiedene Lernansätze – von der punktuellen Ergänzung spezifischer Schwachstellen bis hin zum Aufbau eines kompletten Semesterstoffs. Hierfür wurden Mechanismen implementiert, die sowohl die Präzision als auch die Geschwindigkeit bei der Pool-Verwaltung optimieren.

Dynamisches Hinzufügen

Das primäre Werkzeug zur Individualisierung ist das **Stern-System**. In Anlehnung an bekannte Favoriten-Logiken aus modernen Web-Applikationen können Nutzer:innen durch das Anklicken des Sternsymbols eine Frage instantan ihrem aktiven Lernpool hinzufügen. Diese Entscheidung wird visuell sofort durch eine Farbänderung des Symbols bestätigt. Dieser Prozess ist bewusst niederschwellig gestaltet, um den „Flow“ beim Durchsichten der Kataloge nicht zu unterbrechen. Ebenso einfach lassen sich Fragen wieder entfernen, was eine dynamische Anpassung des Pools ermöglicht: Beherrscht ein Schüler ein Thema nach einiger Zeit perfekt, kann er es aus dem aktiven Pool nehmen, um Platz für neue Herausforderungen zu schaffen.

Ansichtssteuerung: „Alle“ / „Mein Pool“

Um die Übersichtlichkeit auch bei hunderten verfügbaren Fragen zu wahren, verfügt die Ansicht über einen umschaltbaren Filtermodus. Dieser ist entscheidend für die Selbstorganisation:

- **Explorations-Modus („Alle“)**: In dieser Ansicht wird das gesamte Potenzial der Datenbank sichtbar. Es dient der Akquise neuer Lerninhalte und zeigt auch jene Fragen an, die noch nicht Teil der persönlichen Sammlung sind. **Fokus-Modus („Bereits im Fragenpool“)**: Dieser Modus wirkt wie ein digitaler Schreibtisch, auf dem nur die aktuell relevanten Unterlagen liegen. Nutzer:innen können hier prüfen, wie umfangreich ihr persönlicher Katalog in einem bestimmten Fach bereits ist, und gezielte Bereinigungen vornehmen, ohne durch externe, noch nicht gewählte Fragen abgelenkt zu werden.

Effizienzsteigerung durch Batch-Operationen

Da die Vorbereitung auf die Matura oft unter Zeitdruck geschieht, bietet das System Funktionen zur Massenverarbeitung. Anstatt jede der teilweise über 50 Fragen eines Themengebiets einzeln bewerten zu müssen, erlaubt der **„Alle auswählen“-Button** eine pauschale Übernahme ganzer Module. Dies ist besonders wertvoll für Nutzer:innen,

die sich zu Beginn eines Lernzyklus ein komplettes Fachgebiet erschließen wollen. In Kombination mit der darauffolgenden selektiven Abwahl einzelner Fragen ermöglicht dieser „Top-Down-Ansatz“ eine extrem schnelle und dennoch präzise Konfiguration des Lernumfelds.

Durch diese differenzierte Form der Verwaltung wird sichergestellt, dass die nachfolgenden Module wie die Startansicht (Kapitel 3.2) und der Übungsmodus (Kapitel 3.4.1) eine hochgradig personalisierte Umgebung vorfinden. Die Nutzer:innen behalten zu jedem Zeitpunkt die volle Souveränität über ihre Lerninhalte, was die Eigenverantwortung und das Engagement im Lernprozess maßgeblich fördert.

3.4 Use Case: Üben

Der Kern der Anwendung ist der Übungsmodus, welcher Schüler:innen ermöglicht, gezielt Maturafragen zu trainieren und ihr Wissen Stück für Stück auszubauen. Anders als beim Prüfungsmodus steht hier nicht der Leistungsdruck im Vordergrund, sondern das aktive Lernen mit direktem Feedback. Nutzer:innen soll es ermöglicht werden, ein Fundament für ihr Wissen festigen zu können.

Funktionen umfassen das Auswählen von bestimmten Themen, das Zusammenstellen von einem eigenen Fragenpool und das Beantworten von Fragen ohne Stress. Nutzer:innen können hierbei also genau die Themen lernen, die ihnen Probleme bereiten, und das durch eigene Fragen oder Fragen anderer Nutzer:innen.

Die Plattform unterstützt dabei verschiedene Fragetypen, um das Lernerlebnis flexibler zu gestalten. Um nicht einfach nur ein Ergebnis von „richtig“ und „falsch“ zu liefern, können Nutzer:innen auch Lösungswege einsehen, um das Verständnis nachhaltig zu fördern.

Durch diese wiederholbare und flexible Übungsumgebung eignet sich die Anwendung sowohl für den schnellen Gebrauch als Wissenscheck als auch für die langfristige Vorbereitung auf Tests, Schularbeiten oder die Matura.

3.4.1 Übungsmodus mit direktem Feedback

Beim Aufruf des Übungsmodus landet der User zunächst auf einer Übersichtsseite, auf der eine Zusammenfassung des aktuellen Lernstands zu finden ist und die einen Einstieg in den nächsten Übungsdurchlauf ermöglicht. Diese Ansicht dient dazu, einen schnellen

Überblick darüber zu geben, wie gut einzelne Fragen bereits beherrscht werden und welche Bereiche noch wiederholt werden sollten.

Meinen Fragenpool üben

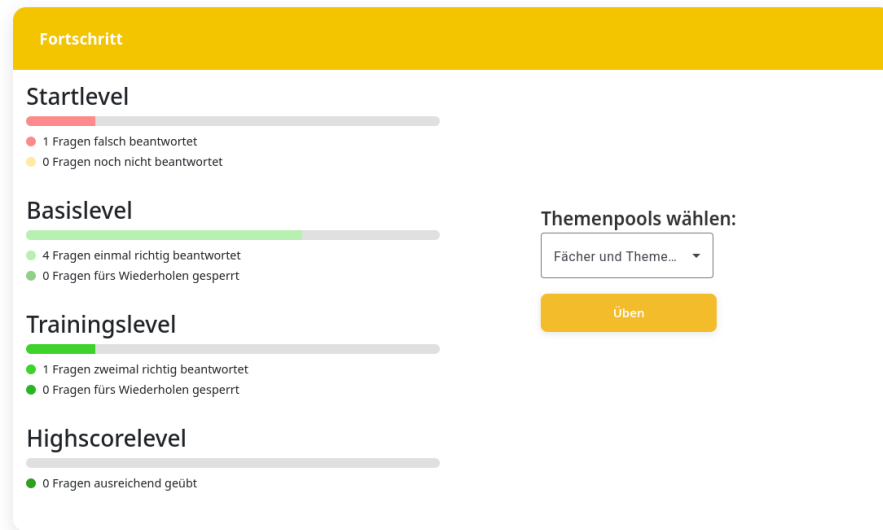


Abbildung 18: Fortschrittsübersicht des Übungsmodus

Rechts neben der Statusübersicht können die Themenbereiche gewählt werden, auf die sich sowohl die Fortschrittsübersicht als auch die nächste Übungssitzung beschränken soll. So kann sich der Übungsmodus auf bestimmte Fächer oder Themenbereiche eingrenzen lassen. Wird aber bei dieser Auswahl „Alle Fragen“ angegeben, so wird der Übungsmodus auch auf alle Themenbereiche im eigenen Fragenpool ausgeweitet.

Meinen Fragenpool üben

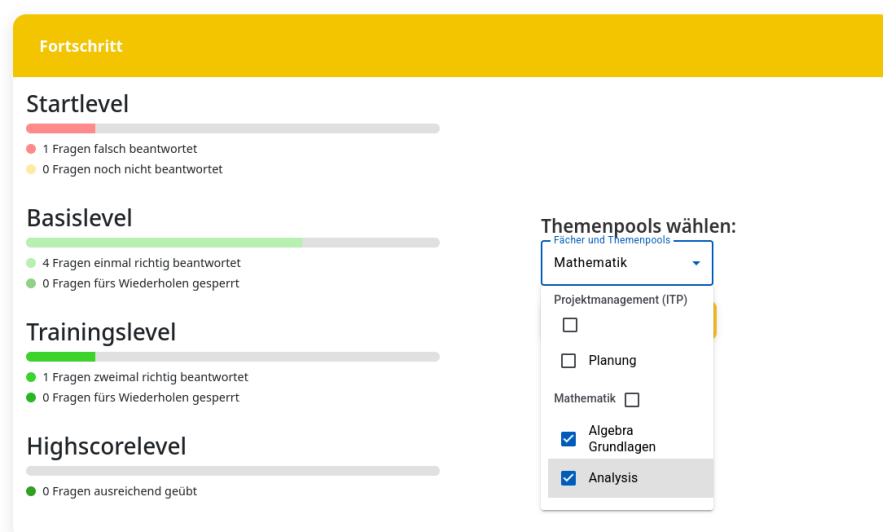


Abbildung 19: Auswahl der Themenbereiche für den nächsten Übungsdurchlauf

Nachdem die gewünschten Themenbereiche ausgewählt wurden, kann der Übungsdurchlauf über den "ÜbenButton direkt unter der Auswahl der Themenbereiche gestartet werden. Das System wechselt automatisch in den Question Runner und lädt die nächste passende Frage aus den ausgewählten Themenbereichen, je nachdem bei welchem Fragenstatus aktuell am meisten Übungsbedarf besteht. Auf diese Weise kann die Übungssitzung direkt an dem Punkt starten, an dem es zu diesem Zeitpunkt am wichtigsten ist.

Bearbeiten der Fragen im Question Runner

Beim Start in den Übungsmodus gelangt der/die Nutzer:in, wie in Abbildung 20 dargestellt, in den Question-Runner, also der zentralen Ansicht, bei der Fragen beantwortet und ausgewertet werden können. Diese Komponente ist eine dynamische Ansicht, da sie je nach gewähltem Modus (Üben, Prüfen oder Review) verschiedene Steuerungselemente bereitstellt.

Benutzeroberfläche des Question Runners

Üben > Fragen beantworten

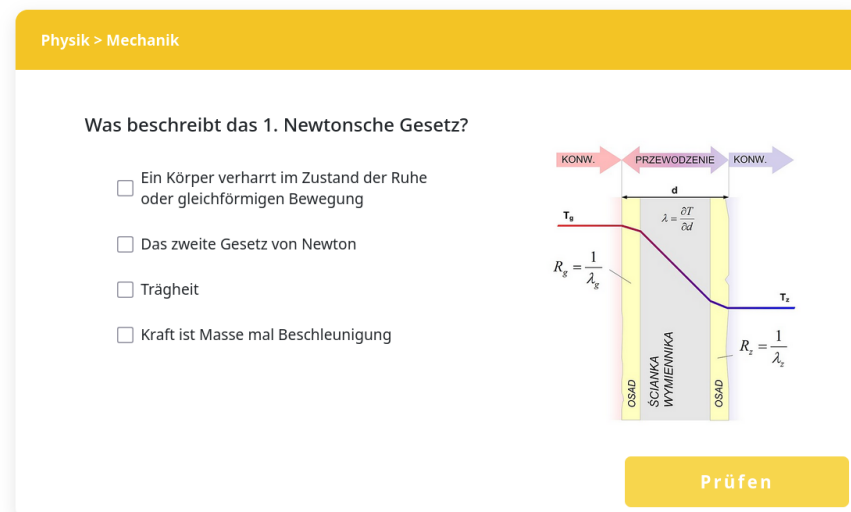


Abbildung 20: Beispielhafte Ansicht einer Frage im Question Runner

Die Ansicht ist hierarchisch aufgebaut, um für eine schnelle Orientierung zu sorgen:

- **Informationsbereich:**

Am oberen Rand wird das aktuelle Fach sowie der spezifische Themenpool angezeigt.

- **Inhaltsbereich:**

Die Fragestellung wird auffällig als Überschrift präsentiert und bei Bedarf auf der rechten Seite durch eine illustrative Grafik ergänzt.

- **Interaktionsbereich:**

Wie nachfolgend beschrieben erscheinen hier je nach Fragetyp Auswahlfelder (Multiple-Choice) oder ein Texteingabefeld (Freitext).

- **Aktionsleiste:**

Im unteren Bereich befinden sich die Navigationselemente. Im Übungsmodus ist dies primär der Button „Prüfen“ zur sofortigen Auswertung, während in der Prüfungssimulation (Kapitel 3.5.2) Navigationsbuttons zum Vor- und Zurückblättern dominieren.

Unterstützung verschiedener Fragetypen

Neben der Unterscheidung der Modi werden im Question Runner die verschiedenen Fragetypen berücksichtigt. Aktuell werden zwei Haupttypen unterstützt:

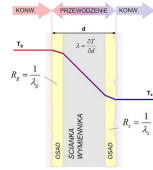
- **Multiple-Choice-Fragen:** Nutzer:innen wählen eine oder mehrere richtige Antworten aus einer Liste vorgegebener Auswahlmöglichkeiten aus. Multiple-Choice-Fragen sind besonders geeignet, um Faktenwissen abzufragen oder schnelle Wissenskontrollen durchzuführen. Das System wertet die Antwort automatisch aus und kann im Übungsmodus direkt Feedback geben. Die Darstellung erfolgt als übersichtliche Auswahlfelder, die klar zwischen ausgewählt und nicht ausgewählt unterscheiden. (siehe Abbildung 21a)
- **Freitext-Fragen:** Nutzer:innen geben ihre Antwort in ein Texteingabefeld ein. Dieser Fragetyp eignet sich für offene Aufgabenstellungen, bei denen die Antwort nicht auf eine vorgegebene Auswahl beschränkt ist. Für die Eingabe wird ein größeres Eingabefeld bereitgestellt, um längere Antworten komfortabel angeben zu können. Die Eingabe wird mit einer Menge an korrekten Antworten abgeglichen. Entspricht die Eingabe einer dieser Antwortmöglichkeiten, wird die Antwort als korrekt bewertet, andernfalls als falsch. (siehe Abbildung 21b).

Üben > Fragen beantworten

Physik > Mechanik

Was beschreibt das 1. Newtonsche Gesetz?

- ☐ Ein Körper verharrt im Zustand der Ruhe oder gleichförmigen Bewegung
- ☐ Das zweite Gesetz von Newton
- ☐ Trägheit
- ☐ Kraft ist Masse mal Beschleunigung



Prüfen

(a) Multiple-Choice-Frage

Üben > Fragen beantworten

Mathematik > Algebra Grundlagen

Was ist $2 + 2$?

Bitte Antwort eingeben...

Prüfen

(b) Freitext-Frage

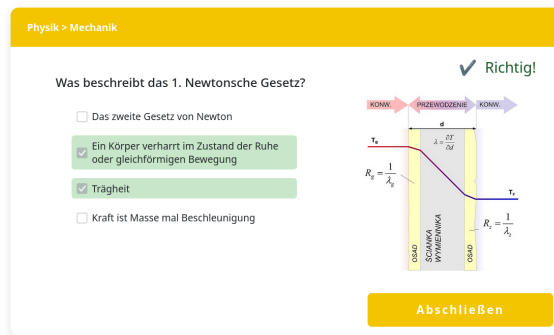
Abbildung 21: Question-Runner: Beispiele für verschiedene Fragetypen

Durch diese Gestaltung unterstützt der Question Runner den/die Nutzer:in auf effektive Weise beim Lernen, beim Prüfen des eigenen Wissens und beim Überprüfen bereits bearbeiteter Aufgaben. Die modulare Darstellung der Fragen, der direkte Zugriff auf Feedback im Übungsmodus und die klare Trennung der Modi tragen dazu bei, dass sich der/die Nutzer:in jederzeit auf das Lernen konzentrieren kann, ohne von technischen Details abgelenkt zu werden.

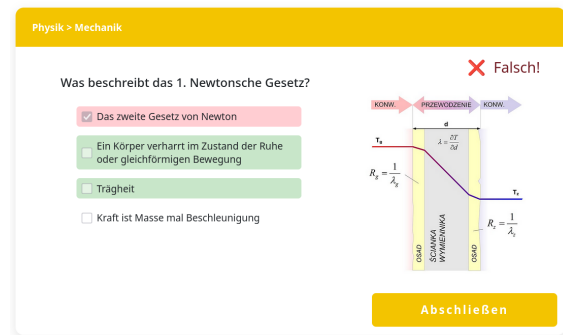
Direktes Feedback im Übungsmodus

Im Übungsmodus liegt das Stressfreie Lernen im Fokus. Der/Die Nutzer:in steht hier also nicht unter Zeitdruck und kann die Frage in Ruhe lesen, überdenken und schließlich beantworten. Erst nachdem eine Antwort ausgewählt beziehungsweise eingegeben wurde, kann diese über den „Prüfen“-Button bestätigt werden. Anschließend wird die Antwort ausgewertet und das Ergebnis unmittelbar im Question Runner angezeigt.

Das direkte Feedback ist so gestaltet, dass es den Lernprozess aktiv unterstützt. Nach der Auswertung wird die Frage deshalb visuell markiert und Antworten werden entsprechend farbig markiert. Korrekte Antworten werden mit einem positiven Grünton, falsche Antworten rot hervorgehoben (siehe Abbildung 22). Zusätzlich wird die richtige Lösung und ein möglicher Lösungsweg (siehe Kapitel 3.4.2) angezeigt. So können Fehler sofort nachvollzogen und Missverständnisse direkt geklärt werden.



(a) Korrekt beantwortete Frage



(b) Falsch beantwortete Frage

Abbildung 22: Question-Runner: Richtig und Falsch beantwortete Frage

Durch diese Form der Rückmeldung wird das Gelernte mit der jeweils bearbeiteten Frage verknüpft. So bleibt der Lernprozess nicht abstrakt, sondern findet direkt am Beispiel der Aufgabe statt. Zugleich ermöglicht die strukturierte Darstellung, dass auch falsch beantwortete Fragen nicht nur negativ und als Fehler wahrgenommen, sondern auch als positiver Bestandteil des Lernprozesses genutzt werden.

Nachdem die Auswertung erfolgt ist, wechselt der "Prüfen"-Button zum "Nächste Frage"-Button. Dieser Button bringt den/die Nutzer:in direkt zur nächsten passenden Frage der ausgewählten Themenbereiche, wobei die Auswahl der nächsten Frage der in Kapitel 1.4 beschriebenen Lernstrategie folgt. Der Übungsmodus führt die Nutzer:innen somit schrittweise durch die Inhalte, die für den weiteren Lernfortschritt am relevantesten sind. Dabei muss der/die Nutzer:in nicht selbstständig die nächste Frage auswählen, sondern kann sich einfach vom System leiten lassen.

Wiederholungslogik und erneute Präsentation von Fragen

Nach der Bearbeitung einer Frage wird deren Ergebnis gespeichert. Richtig beantwortete Fragen gelten zunächst als *zum Wiederholen gesperrt* und sind erst nach einer definierten Wartezeit wieder im Übungsmodus vorzufinden. So wird vermieden, dass dieselbe Frage unmittelbar hintereinander erneut gestellt wird, sondern erst zu einem späteren Zeitpunkt, wenn der Lerneffekt größer ist. Falsch beantwortete Fragen bleiben im Übungsmodus, bis sie korrekt beantwortet wurden, sodass der/die Nutzer:in direkt das Verständnis der gemachten Fehler überprüfen kann. Das soll verhindern, dass Fehler ein weiteres Mal gemacht werden und es zu einem endlosen Kreislauf wird, da man den Fehler nie wirklich verstehen konnte.

Fragen, die mehrfach korrekt beantwortet wurden, werden nach und nach als gefestigt eingestuft. Wurde eine Frage bereits dreimal hintereinander richtig beantwortet, so gilt sie als ausgelernt und wird dauerhaft aus dem aktiven Fragenpool für den Übungsmodus entfernt. Sie erscheint also nicht mehr im Übungsmodus, da davon ausgegangen werden kann, dass der Inhalt ausreichend gefestigt ist.

Verhalten bei Sitzungsfortsetzung und Abschluss des Übungsdurchlaufs

Das explizite Abbrechen des Übungsdurchlaufs ist nicht vorgesehen, verlässt der/die Nutzer:in den Question Runner während der Bearbeitung dennoch, geht der aktuelle Bearbeitungsstand verloren, die bereits beantworteten Fragen bleiben aber gespeichert. Deshalb kehrt der/die Nutzer:in beim erneuten Aufrufen des Übungsmodus also auch automatisch zur zuletzt offenen Frage zurück.

Sind für den ausgewählten Themenbereich vorübergehend keine weiteren Fragen mehr verfügbar, weil alle Fragen entweder ausgelernt oder zur Wiederholung gesperrt sind, so wird der "Prüfen-Button zum Abschließen"-Button. Durch diesen gelangt man zurück zur vorherigen Ansicht, also der Fortschrittsübersicht.

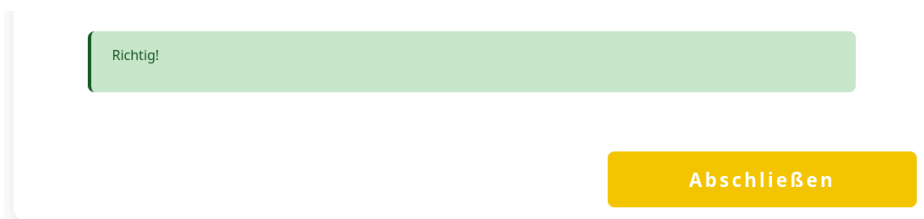


Abbildung 23: "PrüfenButton wurde zum AbschließenButton

3.4.2 Lösungswege ansehen und erstellen

Ein wesentlicher Bestandteil des Übungsmodus ist, dass Nutzer:innen nicht nur erfahren, ob eine Antwort korrekt war, sondern auch warum (nicht). Um ein tiefgreifendes Verständnis der Materie zu fördern, bietet die Anwendung nach jeder beantworteten Frage die Möglichkeit, detaillierte Lösungswege und Erklärungen einzusehen oder bei richtiger Antwort einen eigenen Lösungsweg zu erstellen.

Einsehen von Lösungswegen

Sobald eine Frage im Übungsmodus über den „Prüfen“-Button ausgewertet wurde, erscheint unterhalb der Interaktionselemente der Bereich für die Lösungen verschiedener

Schüler. Ein Lösungsweg besteht dabei aus einer ausführlichen Erklärung, die den Lösungsprozess Schritt für Schritt erläutert.

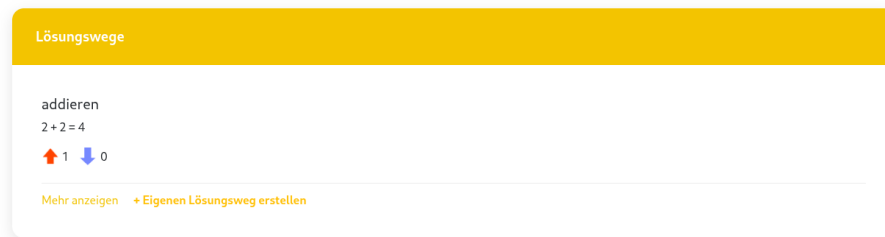


Abbildung 24: Anzeige eines detaillierten Lösungsweges

Aktive Mitgestaltung: Lösungen erstellen

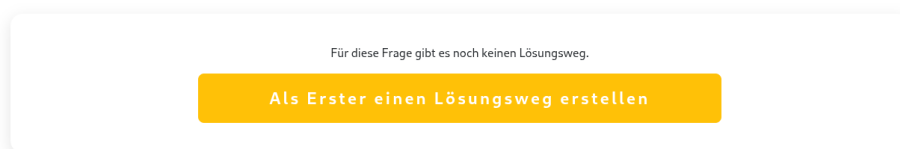


Abbildung 25: Anzeige einer Frage ohne vorhandenen Lösungsweg

Die Lernplattform lebt von der Interaktion und dem Wissensaustausch innerhalb der Community. Sollte zu einer Frage noch kein Lösungsweg existieren oder ein/e Nutzer:in eine verständlichere Erklärung beisteuern wollen, bietet das System die Funktion, eigene Lösungswege zu verfassen.

[Meinen Fragenpool üben](#) > Fragen beantworten

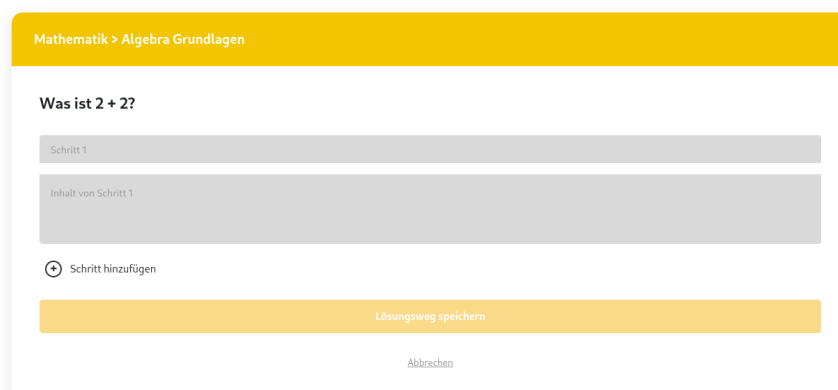


Abbildung 26: Editor zum Erstellen eines neuen Lösungsweges

Über einen Editor-Modus können Nutzer:innen Texte verfassen. Durch das Erstellen eigener Erklärungen findet ein zusätzlicher Lerneffekt statt, da die aktive Auseinandersetzung mit dem Stoff („Lernen durch Lehren“) die eigenen Kenntnisse festigt.

Feedbackmechanismen und Community-Interaktion

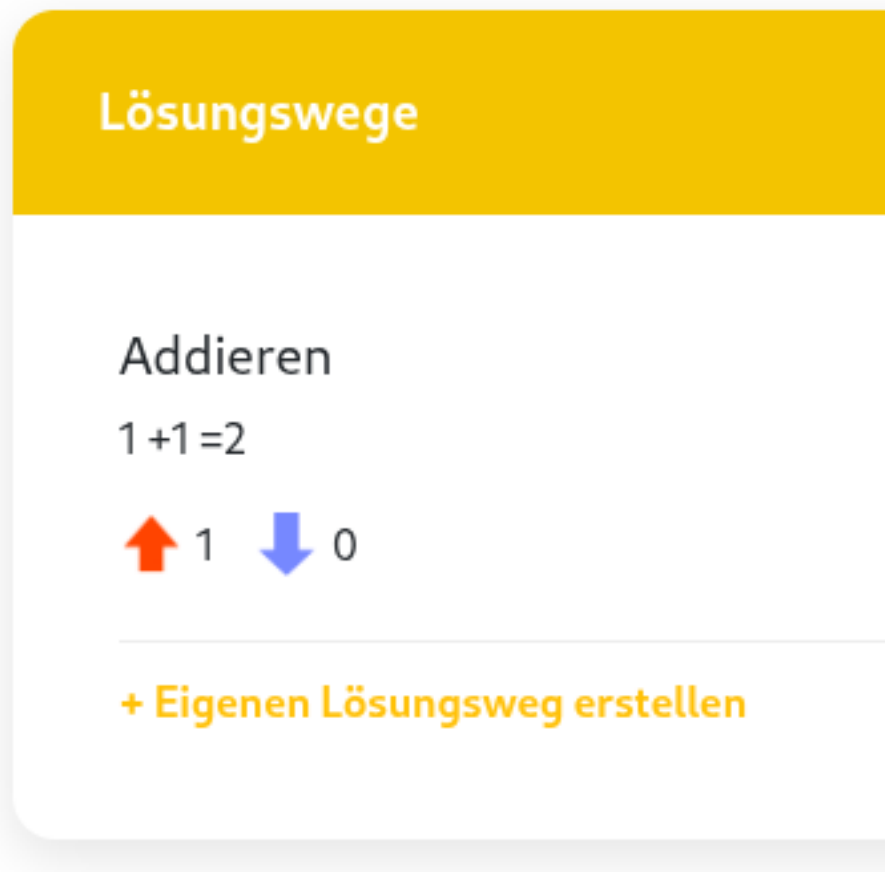


Abbildung 27: Darstellung des Feedback-Systems für Lösungswege

Um die Qualität der bereitgestellten Inhalte sicherzustellen, verfügt jeder Lösungsweg über ein Feedback-System. Nutzer:innen können hilfreiche Erklärungen positiv markieren oder bei unklaren Lösungen negativ bewerten.

Bewertungssystem: Hilfreiche Lösungen werden durch die Community nach oben priorisiert, sodass immer die qualitativ hochwertigste oder am besten zu verstehenden Erklärung zuerst angezeigt wird.

3.5 Use Case: Prüfungssimulation

Im Prüfungsmodus können Nutzer:innen ihr Wissen, das sie im Übungsmodus (3.4.1) erlangt haben, auf die Probe stellen. Er richtet sich also an Schüler:innen, die bereits gelernt und ihr Wissen gefestigt haben, und nun eine Prüfungssituation simulieren möchten, um ihren aktuellen Wissensstand einzuschätzen.

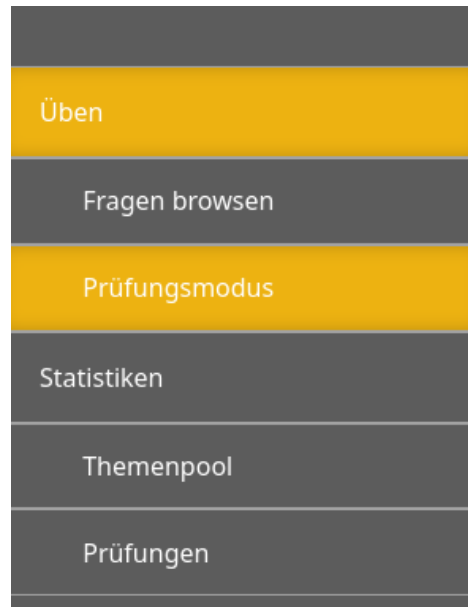


Abbildung 28: Seitennavigationsleiste bei ausgewähltem Prüfungsmodus

Wie in der obenstehenden Abbildung erkenntlich, ist der Prüfungsmodus über die Seitennavigation der Lernplattform zugänglich. Er dient ausschließlich zur Überprüfung des Wissensstands, nicht aber des aktiven Lernens. Die Kernfunktionen umfassen die Bearbeitung einer Reihe an Fragen aus ausgewählten Themenbereichen unter Zeitlimit, die Navigation zwischen Fragen während der Prüfung sowie die Auswertung der Prüfung am Ende mit einer Übersicht über richtig und falsch beantworteter Fragen.

Nutzer:innen können realistische Prüfungssituationen erleben, ohne dass dies Auswirkungen auf ihren Fortschritt im Übungsmodus hat. So bleibt Üben und Prüfen klar voneinander getrennt, während gleichzeitig ein Eindruck über den eigenen Lernfortschritt gewonnen wird. Dabei kann auch ein Einblick über die eigene Performance unter Zeitdruck helfen, neue Baustellen aufzudecken.

3.5.1 Konfiguration einer Prüfung

Die erste Ansicht, wenn der/die Nutzer:in zum Prüfungsmodus navigiert, ist die Konfigurationsansicht. Vor dem Start der Prüfung kann der/die Nutzer:in die Prüfung

individuell konfigurieren. Die Optionen werden dafür übersichtlich auf der Ansicht dargestellt, sodass der/die Nutzer:in gezielt entscheiden kann, welche Themenbereich getestet werden sollen, wie viele Fragen die Prüfung beinhalten soll und welches Zeitlimit dafür zur Verfügung stehen soll.

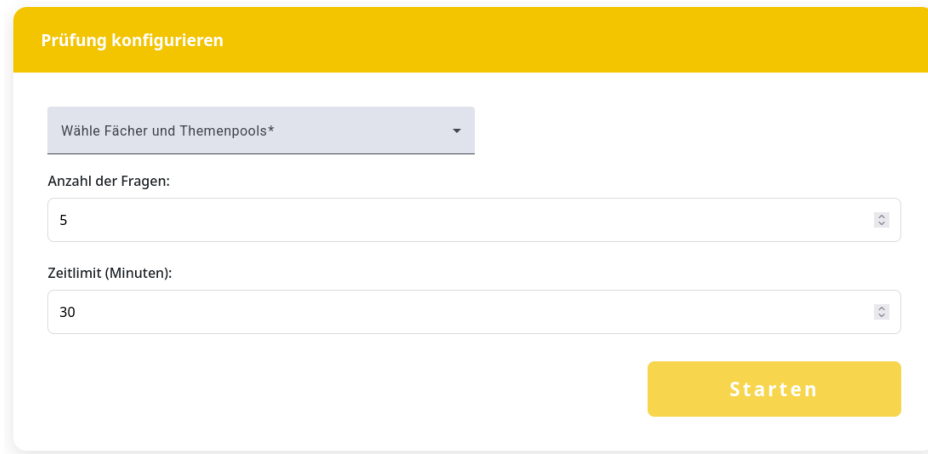


Abbildung 29: Beispielhafte Ansicht der Prüfungs-Konfiguration

Folgende Optionen sind dabei vorhanden:

- **Themenbereich auswählen:**

Nutzer:innen können einzelne Fächer und Themenbereiche auswählen. Dabei werden nur jene ausgewählt, von welchen Fragen auch im eigenen Fragenpool vorhanden sind.

- **Anzahl der Fragen:**

Es kann festgelegt werden, wie viele Fragen insgesamt in der Prüfung enthalten sein sollen. Dies ermöglicht sowohl kurze Tests als auch längere Prüfungssimulationen. Die Prüfung besteht dann aus zufälligen Fragen aus dem Fragenpool über die ausgewählten Bereiche, wobei zu jedem Themenbereich mindestens eine Frage gestellt wird, wenn die Anzahl der Fragen dafür ausreicht.

- **Zeitlimit:**

Das gewünschte Zeitlimit für die gesamte Prüfung kann eingestellt werden, um die Situation einer realen Prüfung nachzubilden. Läuft die Zeit aus, so wird die Prüfung automatisch mit den aktuell eingegebenen Antworten abgeschickt und ausgewertet.

Sobald die Konfiguration abgeschlossen ist, kann der/die Nutzer:in die Prüfung über den *Starten*-Button beginnen. Die Plattform wechselt automatisch in den Prüfungs-

modus im Question Runner, wobei die zuvor festgelegten Konfigurationen angewendet werden.

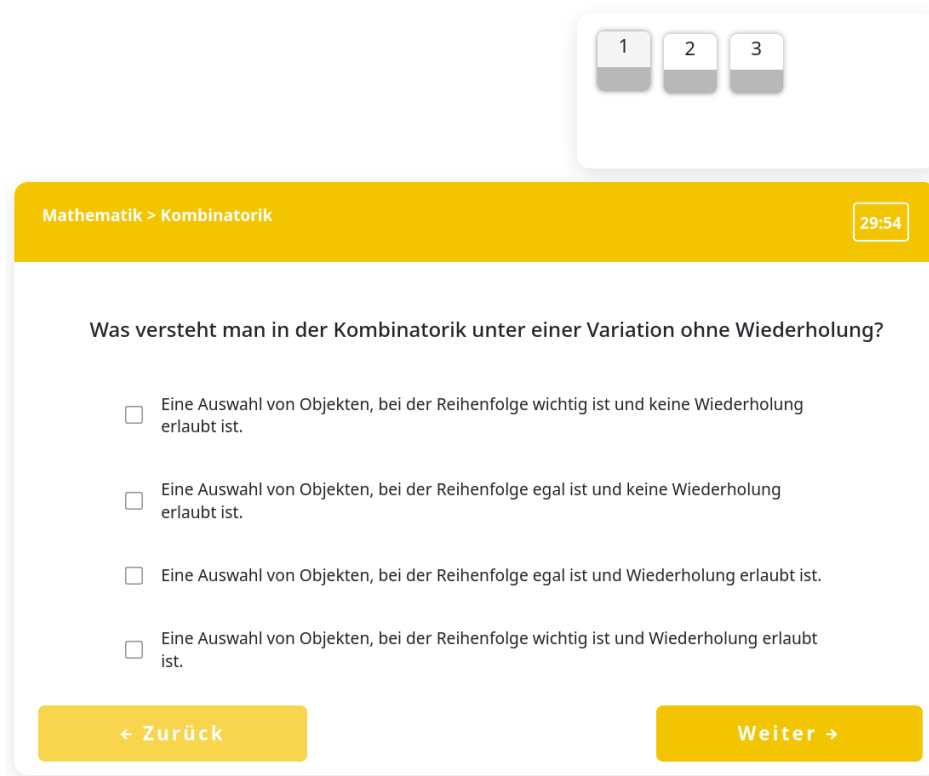


Abbildung 30: Start der Prüfung nach Konfiguration

3.5.2 Prüfungsmodus mit Zeitlimit

Ist die Prüfung gestartet, so gelangt der/die Nutzer:in in den Prüfungsmodus des Question Runners. Dieser Modus soll möglichst authentisch eine realistische Prüfungssituation nachbilden und sich somit klar vom Übungsmodus unterscheiden. Der Fokus soll hierbei nicht mehr auf dem Lernen liegen, sondern auf der Überprüfung des aktuellen Leistungsstands unter Zeitdruck.

Wie in der oberen Abbildung 30 dargestellt wird in der oberen Zeile nun zusätzlich zum Fach und Themenpool wie im Übungsmodus auch permanent die verbleibende Zeit angezeigt. Durch diese Darstellung soll der zeitliche Rahmen der Prüfung verdeutlicht werden, ohne dabei zu stark abzulenken. Die aktuelle Frage wird darunter angezeigt inklusive Antwortfeldern und gegebenenfalls das zugehörige Bild, analog zur Darstellung im Übungsmodus.

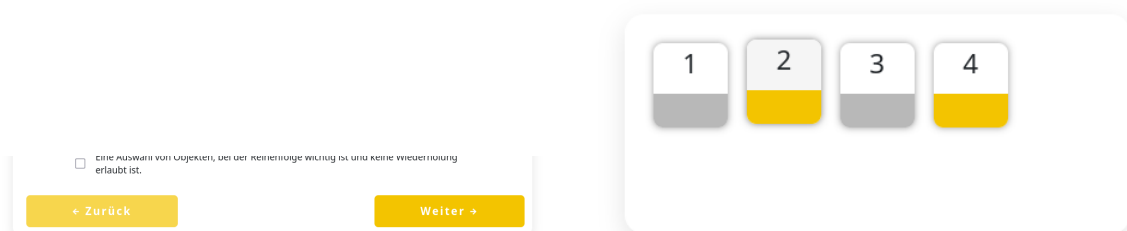
Der zentrale Unterschied zum Übungsmodus ist das verzögerte Feedback. Im Gegensatz zum Übungsmodus erhält der/die Nutzer:in während der Prüfung keinerlei Rückmeldung darüber, ob eine Frage korrekt oder falsch beantwortet wurde. Es kann lediglich zwischen

den einzelnen Fragen beliebig oft hin und her navigiert werden, während sich das System die eingegebenen Antworten merkt. Dadurch kommt der Prüfungsmodus einer realen Prüfung besonders nahe, da man bis zum Ende der Prüfung seine Antworten beliebig oft verändern kann, doch nur die letzte wirklich zählt.

Navigation durch die Prüfung

Der/Die Nutzer:in kann während der Prüfung beliebig oft zwischen Fragen hin und her navigieren und dabei seine Antworten beliebig oft ändern. Das Navigieren erfolgt über 2 Arten:

- **„Zurück“- und „Weiter“-Buttons** (siehe Abbildung 31a)
Ermöglichen es, zur nächsten oder zur letzten Frage zu wechseln.
- **Hauptnavigationsleiste über dem Question Runner** (siehe Abbildung 31b)
Ermöglicht es, zu einer beliebigen Frage in der Prüfung zu springen.



(a) Weiter- und Zurück-Buttons im Prüfungsmodus

(b) Hauptnavigationsleiste des Prüfungsmodus

Abbildung 31: Prüfungsmodus: Navigationsarten

Die „Zurück“- und „Weiter“-Buttons navigieren den/die Nutzer:in zur vorherigen beziehungsweise nächsten Frage. Befindet sich der/die Nutzer:in bereits auf der ersten Frage der Prüfung, so ist der „Zurück“-Button ausgegraut und es ist nicht möglich, eine weitere Frage zurück zu springen. Erreicht der/die Nutzer:in allerdings die letzte Frage, so wird der „Weiter“-Button zum „Fertigstellen“-Button, mit welchem die Prüfung abgegeben werden kann.



Abbildung 32: Prüfungsmodus: Letzte Frage - Fertigstellen-Button

Alternativ zu den beiden Navigations-Buttons gibt es auch die Prüfungsnavigationsleiste über dem Question Runner. Diese ermöglicht es, jederzeit zu jeder beliebigen Frage zu

springen. Wie in Abbildung 31b zu erkennen, repräsentiert jeder dieser Boxen eine eigene Frage, welche entweder grau oder orange markiert ist. Diese Farben haben folgende Bedeutungen:

- **Grau:** Diese Frage wurde noch nicht beantwortet
- **Orange:** Für diese Frage wurde bereits eine Antwort gespeichert. Diese Markierung gibt allerdings keine Auskunft darüber, ob die abgegebene Antwort korrekt oder falsch ist.

Abgabe der Prüfung

Läuft das vorgegebene Zeitlimit ab, so wird die Prüfung automatisch abgegeben. In diesem Fall werden alle bis dahin gespeicherten Antworten zur als richtig oder falsch ausgewertet. Alle Fragen, die bis zum Ablauf der Zeit nicht beantwortet wurden, gelten dementsprechend auch als unbeantwortet und werden automatisch als falsch gewertet. Der/Die Nutzer:in kann die Prüfung somit nicht über die konfigurierte Dauer hinaus fortsetzen.

Neben der automatischen Abgabe durch Zeitablauf kann die Prüfung auch jederzeit manuell abgeschlossen werden. Wie im oberen Abschnitt bereits angeführt, erfolgt dies über den „Fertigstellen“-Button, der beim Erreichen der letzten Frage angezeigt wird. Durch das manuelle Abgeben wird die Prüfung sofort beendet und alle gespeicherten Antworten werden zur Auswertung weitergegeben. Auch hier ist eine nachträgliche Änderung der Antworten ab diesem Zeitpunkt nicht mehr möglich.

Während der gesamten Prüfung werden die eingegebenen Antworten regelmäßig zwischengespeichert, was das Erhalten des Bearbeitungsstands auch beim Navigieren ermöglicht. Ein explizites Pausieren oder Abbrechen der Prüfung ist allerdings nicht vorgesehen. Wird die Prüfung vorzeitig verlassen, so gilt die Prüfung als nicht ordnungsgemäß abgeschlossen und kann nicht fortgesetzt werden. Dies sorgt für einen authentischeren Umgang mit Prüfungssituationen, welche nicht einfach abgebrochen und neu gestartet werden können.

3.5.3 Bewertung und Ergebnisausgabe

Nach dem Abschluss einer Prüfung durch Zeitablauf oder manueller Abgabe wird die Prüfung automatisch ausgewertet. Der/Die Nutzer:in gelangt im Anschluss zur

Ergebnisansicht, in der eine zusammenfassende Auswertung der eben abgelegten Prüfung dargestellt wird.

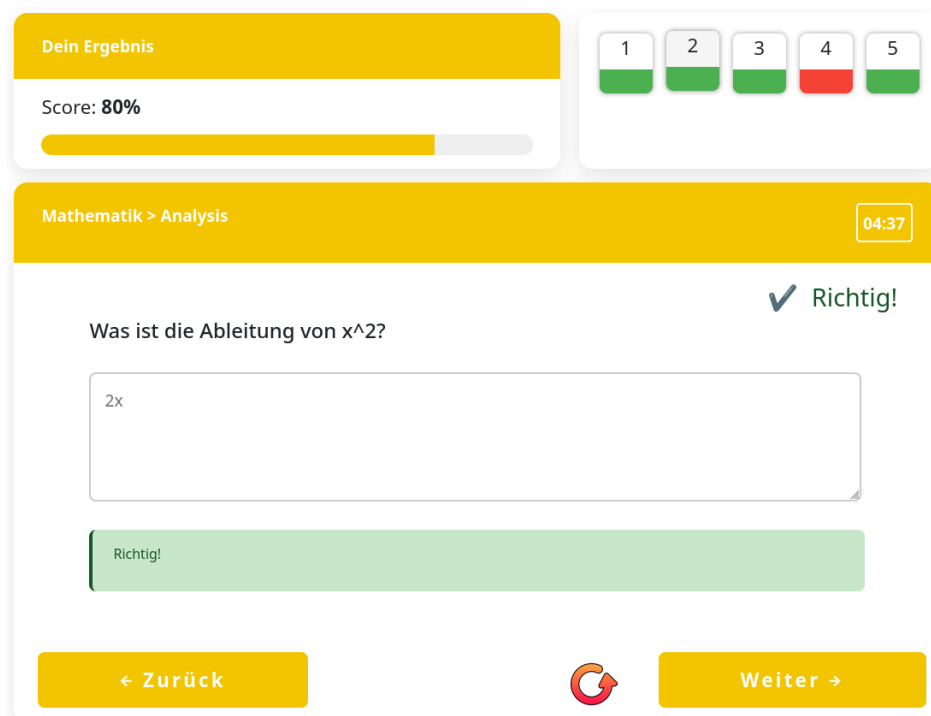


Abbildung 33: Ergebnisübersicht nach Abschluss der Prüfung

In dieser Übersicht kann auf den ersten Blick erkannt werden, wie viele Fragen korrekt beziehungsweise falsch ausgewertet wurden. Zusätzlich wird der erreichte Anteil korrekt beantworteter Fragen in Prozent dargestellt. Auf Basis dieser Anzeige kann der/die Nutzer:in direkt grob einschätzen, wie sehr die ausgewählten Themenbereiche unter Zeitdruck beherrscht werden.

Neben der Gesamtauswertung bietet die Ergebnisansicht auch einen detaillierten Überblick über alle einzelnen Fragen der Prüfung. Die Navigation zwischen den Fragen erfolgt hierbei analog zum Prüfungsmodus. Es ist also sowohl über die „Zurück“- und „Weiter“-Buttons als auch über die Navigationsleiste über dem Question Runner möglich, schnell zwischen Fragen zu springen. Die Navigationsleiste unterscheidet sich allerdings darin, dass bei dieser sofort sichtbar ist, ob die jeweilige Frage korrekt oder falsch beantwortet wurde. Dafür ist die Box für die zugehörige Frage grün beziehungsweise rot markiert.

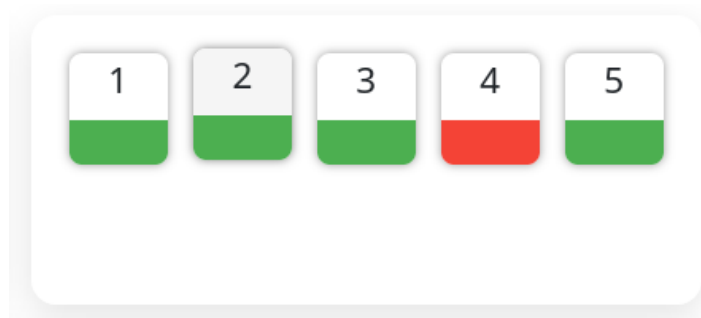


Abbildung 34: Navigationsleiste nach Auswertung der Prüfung

Beim Aufruf einer einzelnen Frage innerhalb der Ergebnisansicht wird diese im Question Runner angezeigt. Die Darstellung der Frage und des Feedbacks entspricht dabei bewusst der des Übungsmodus (siehe Kapitel 3.4.1). Korrekte und falsche Antworten werden also entsprechend farblich hervorgehoben, die richtige Lösung sowie zugehörige Lösungswege werden angezeigt.

Durch die identische Darstellung des Feedbacks in Übungs- sowie Reviewmodus müssen sich Nutzer:innen nicht an eine neue Oberfläche gewöhnen. Dadurch wird die Orientierung erleichtert und die Prüfung besser als Lerneffekt fungieren, da Fehler direkt im bekannten Kontext nachvollzogen werden können.

Ist der/die Nutzer:in mit dem Analysieren der Fehler fertig, so kann der Reviewmodus jederzeit verlassen werden. Befindet sich der/die Nutzer:in auf der letzten Frage der Prüfung, so wird wie beim Prüfungsmodus der „Weiter“-Button durch einen „Fertigstellen“-Button ersetzt. Durch das Betätigen dieses Buttons wird der Reviewmodus beendet und der/die Nutzer:in zurück zur Prüfungskonfiguration geführt.

Alternativ kann diese Prüfung von hier aus auch neu gestartet werden. Dabei wird eine exakte Kopie der Prüfung mit den selben Fragen und dem selben Zeitlimit erstellt. Der/Die Nutzer:in kann somit die Prüfung beliebig oft neu starten, wenn das Verständnis für die eben gemachten Fehler direkt geprüft werden soll. Für diese Funktion ist direkt neben dem „Weiter“-Button ein Neustart-Symbol zu finden.



Abbildung 35: Neustart-Button im Reviewmodus

Damit ist der jeweilige Prüfungsdurchlauf vollständig abgeschlossen. Alle Ergebnisse bleiben gespeichert und können später erneut im Reviewmodus eingesehen werden,

ohne den Übungsmodus oder den Lernfortschritt zu beeinflussen. Wird eine Prüfung über die Neustart-Funktion erneut begonnen, so entsteht ein neuer, unabhängiger Prüfungsdurchlauf, während die Ergebnisse vorheriger Durchläufe weiterhin erhalten bleiben.

3.6 Use Case: Fortschritt einsehen und analysieren

Ein zentrales Element für den Lernerfolg in der Prüfungsvorbereitung ist die kontinuierliche Selbstreflexion. Die Lernplattform stellt hierfür ein umfassendes Modul zur Fortschrittseinsicht bereit, das weit über eine bloße Auflistung von Ergebnissen hinausgeht. Das Ziel ist es, den Nutzer:innen ein Werkzeug an die Hand zu geben, mit dem sie ihren aktuellen Wissensstand objektiv bewerten und ihre Lernstrategie datengestützt anpassen können. Durch die visuelle Aufbereitung komplexer Leistungsdaten wird der Lernprozess greifbar und die Motivation durch sichtbare Erfolge langfristig gesteigert.

3.6.1 Prüfungsverlauf und tiefergehende Detailanalyse

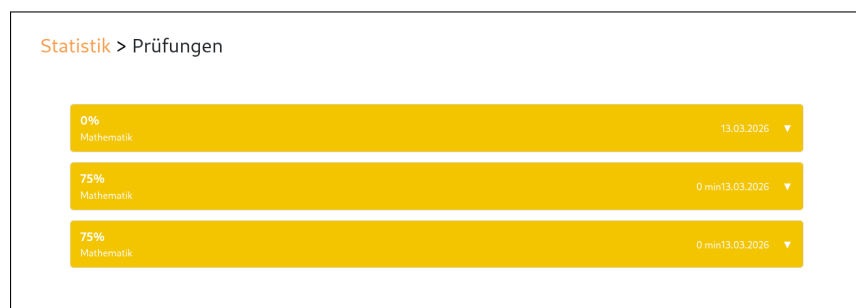


Abbildung 36: Übersicht des Prüfungsverlaufs mit chronologischer Historie

Um die individuelle Entwicklung über einen längeren Zeitraum hinweg nachvollziehen zu können, bietet die Anwendung eine lückenlose Historie aller absolvierten Prüfungen. Diese Übersicht dient als „Logbuch“ des Lernprozesses. Jede Zeile der Liste liefert sofort die entscheidenden Parameter: Das Datum dokumentiert die Lernfrequenz, während die Fachangabe und die erreichte Prozentzahl den direkten Leistungsvergleich ermöglichen. Besonders die Anzeige der benötigten Zeit bietet einen Mehrwert, da sie Aufschluss darüber gibt, ob die Nutzer:innen bereits unter realen Zeitbedingungen sicher agieren oder ob die Bearbeitungsgeschwindigkeit noch gesteigert werden muss.

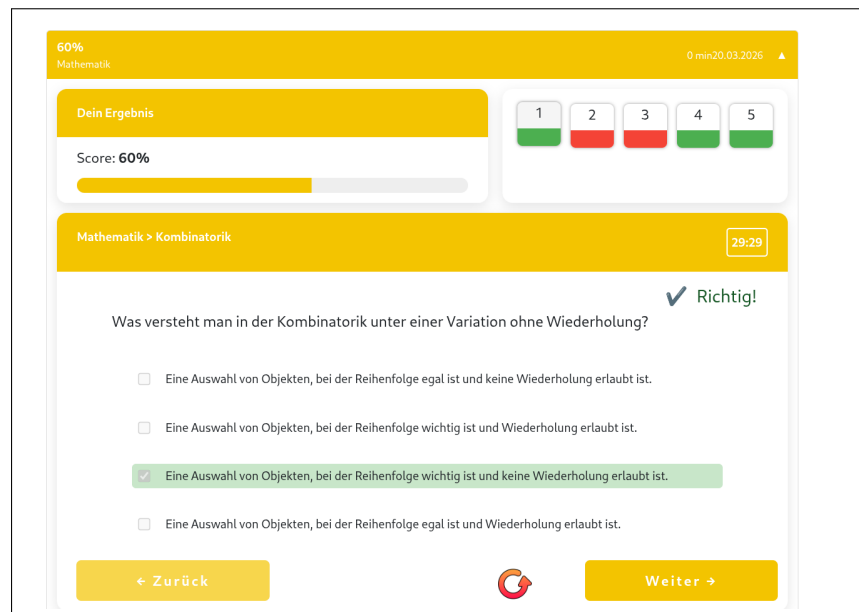


Abbildung 37: Detailansicht zur Identifikation spezifischer Wissenslücken

Die wahre Tiefe der Analyse offenbart sich jedoch in der Detailansicht, die durch ein einfaches Aufklappen des jeweiligen Prüfungseintrags aktiviert wird. Hier wird die Prüfung in ihre Einzelteile zerlegt: Jede gestellte Frage wird inklusive der vom Nutzer gewählten Antwortmöglichkeit und der korrekten Lösung eingeblendet. Diese Transparenz ist entscheidend für die Fehlerkultur: Nutzer:innen können direkt prüfen, ob ein Fehler aus Flüchtigkeit geschah oder ob ein tieferliegendes Verständnisproblem vorliegt.

Ein besonderes Feature zur Erfolgskontrolle ist die Option, eine bereits abgeschlossene Prüfung im identischen Format erneut zu starten. Dieser „Re-Test“-Ansatz ist pädagogisch wertvoll, um zu verifizieren, ob aus Fehlern der Vergangenheit gelernt wurde. Es entsteht ein direkter Vorher-Nachher-Vergleich, der die Lernkurve innerhalb eines spezifischen Fragen-Sets steil ansteigen lässt und so das Selbstvertrauen für die echte Matura stärkt.

3.6.2 Zentrale Lernstatistiken und Motivationsmechanismen

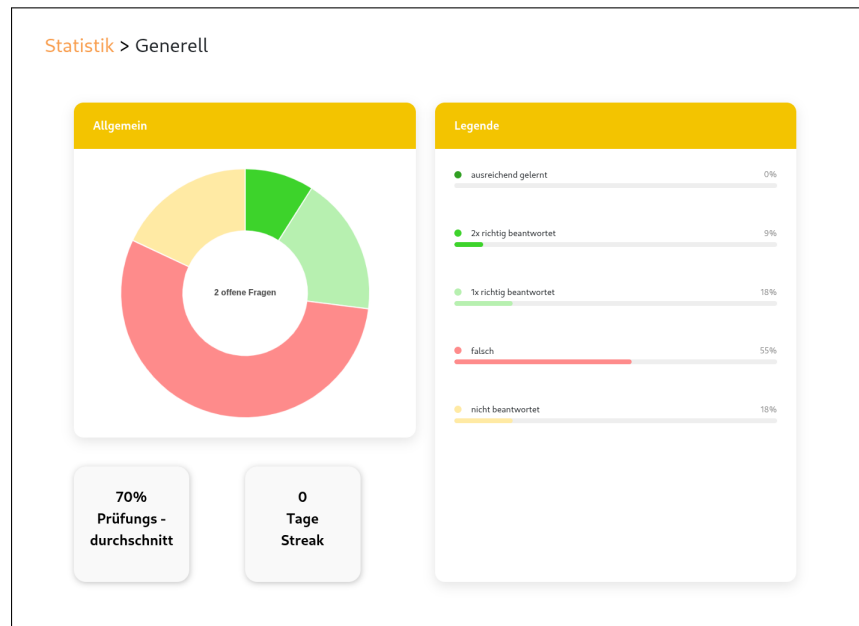


Abbildung 38: Zentrales Dashboard für den Gesamtfortschritt

Das allgemeine Statistik-Dashboard fungiert als optischer Ankerpunkt der Anwendung. Das prominente Ringdiagramm bietet eine intuitive Kategorisierung des gesamten Fragenpools. Anstatt nur „Richtig“ oder „Falsch“ anzuzeigen, wird hier die Nachhaltigkeit des Wissens abgebildet. Fragen wandern durch verschiedene Stadien – von der ersten korrekten Beantwortung bis hin zum Status „langfristig gefestigt“, der erst nach mehrfacher korrekter Bearbeitung erreicht wird. Diese Differenzierung verhindert eine Selbstüberschätzung und motiviert dazu, Inhalte nicht nur einmalig zu verstehen, sondern dauerhaft abzuspeichern.

Zusätzlich zur grafischen Auswertung integriert die Plattform spielerische Elemente (Gamification), um die Nutzerbindung zu erhöhen:

- **Streak-Anzeige:** Durch die Visualisierung der aufeinanderfolgenden Lerntage wird die psychologische Hemmschwelle, das Training zu unterbrechen, erhöht. Die „Streak“ belohnt die Disziplin und macht Beständigkeit zu einem sichtbaren Erfolgserlebnis.
- **Globaler Prüfungsdurchschnitt:** Dieses Widget fungiert als objektives Barometer. Es aggregiert alle bisherigen Leistungen und gibt eine realistische Einschätzung darüber ab, wie die aktuelle Leistung im Verhältnis zur angestrebten positiven Matura steht. Schwankungen werden geglättet, und ein stabiler Aufwärtstrend wird sofort erkennbar.

3.6.3 Selektive Auswertung nach Themenpools

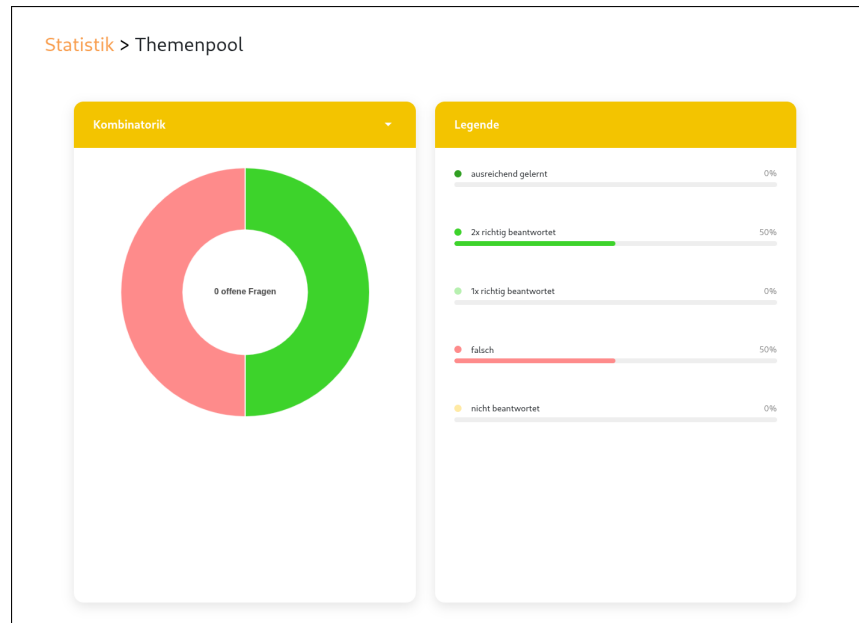


Abbildung 39: Spezifische Analyse mittels Themenpool-Filter

Da die Anforderungen der Matura sehr breit gefächert sind, ist eine isolierte Betrachtung einzelner Fachbereiche für eine effiziente Vorbereitung unerlässlich. Die themenspezifische Auswertung ermöglicht es, die statistische Ansicht gezielt nach Themenpools (z.B. „Vektorrechnung“, „Analysis“ oder „Finanzmathematik“) zu filtern.

Diese Granularität in der Darstellung ist ein wesentlicher Vorteil für das Zeitmanagement. Viele Lernende neigen dazu, Themen zu wiederholen, die sie ohnehin schon beherrschen, da dies ein falsches Gefühl von Sicherheit vermittelt. Die Filterfunktion deckt schonungslos auf, in welchen Pools die „unberührten“ oder „falsch beantworteten“ Fragen dominieren.

Durch diese gezielte Rückmeldung können Nutzer:innen ihre Lernsessions personalisieren. Wenn die Statistik beispielsweise zeigt, dass im Bereich „Stochastik“ noch 80

3.7 Implementierungsdetails

3.7.1 Datenmodell

Das Datenmodell bildet die grundlegende Struktur der gesamten Anwendung und wurde als gemeinsames Schema von beiden Entwicklungsteams entworfen. Es umfasst alle zentralen Entitäten zur Verwaltung von Nutzer:innen, Fragen, Themenbereichen, Übungsfortschritt und Prüfungssimulation.

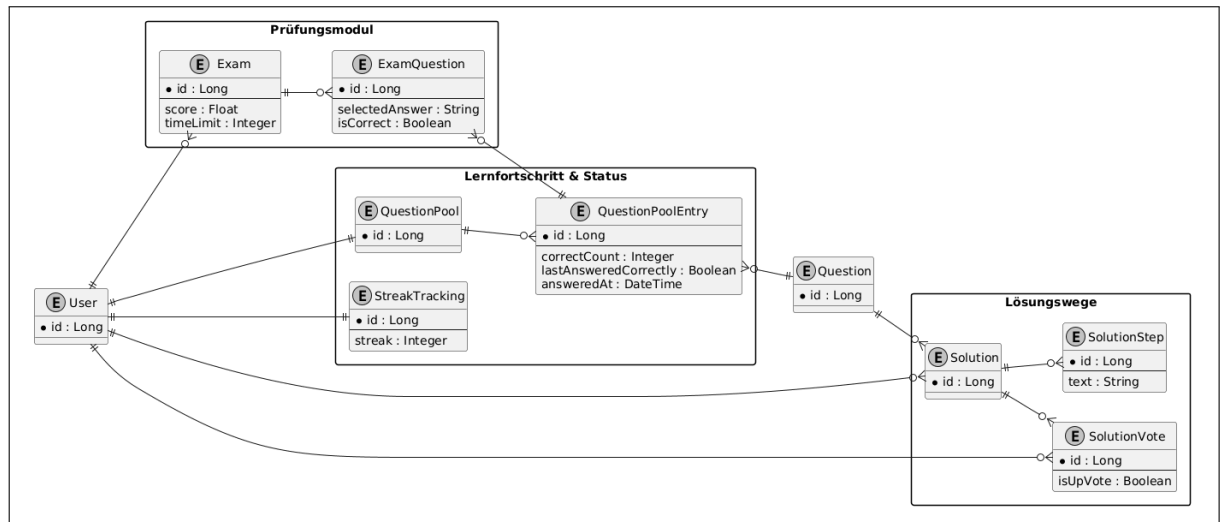


Abbildung 40: Datenbankmodell der Anwendung

Abbildung 40 zeigt das vollständige relationale Datenbankmodell der Anwendung. Nicht alle dargestellten Tabellen werden in jedem beschriebenen Use Case dieser Arbeit verwendet, da einzelne Komponenten von unterschiedlichen Teams umgesetzt wurden. Für die folgenden Abschnitte werden daher nur jene Entitäten näher betrachtet, die für die Umsetzung der Übungs- und Prüfungsfunktionen relevant sind.

Zentrale Entitäten für den Übungsmodus

Für den Übungsmodus sind insbesondere die Entitäten *Question*, *Answer*, *QuestionPool*, *QuestionPoolEntry* und *User* relevant. Sie bilden die Grundlage, um Fragen darzustellen, Antworten zu erfassen und den Lernfortschritt der Nutzer:innen nachzuvollziehen.

Question

Die Entität *Question* repräsentiert eine einzelne Übungsfrage. Sie enthält den Fragetext und den Fragetyp (*MULTIPLE CHOICE*, *TEXT*). Jede Frage kann mehrere Antworten besitzen, die in der Entität *Answer* hinterlegt sind.

Answer

Answer-Objekte beinhalten den Antworttext und einen Eintrag, ob die Antwort korrekt ist (*isCorrect*). Unabhängig vom Fragetypen werden alle richtigen und falschen Antworten auf eine Frage gespeichert. Bei Multiple-Choice-Fragen müssen alle richtigen und keine falschen Antworten angegeben werden, um als korrekt gewertet zu werden. Für Freitext-Fragen werden nur richtige Antworten angelegt. Die eingegebene Antwort muss hierbei einer der Einträge gleichen, um korrekt gewertet zu werden.

QuestionPool und QuestionPoolEntry

Der individuelle Fragenpool der Nutzer:innen wird von einem *Questionpool*-Objekt repräsentiert. Die Entität *QuestionPoolEntry* verbindet eine Frage mit einem Pool und speichert den individuellen Fortschritt des Nutzers/der Nutzerin. Dazu zählen unter anderem:

- *correctCount*: Anzahl der aufeinanderfolgend korrekt geantworten Male für diese Frage
- *lastAnsweredCorrectly*: ob die Frage zuletzt richtig beantwortet wurde
- *answeredAt*: Zeitpunkt, zu dem die Frage zuletzt beantwortet wurde

Diese Tabelle ermöglicht die Umsetzung der Wiederholungslogik im Übungsmodus, indem Fragen je nach Bearbeitungsstatus und Lernerfolg wiederholt oder vorübergehend gesperrt werden.

User

Die Entität *User* enthält die grundlegenden Informationen über die Lernenden. Sie ist mit den *QuestionPool*-Objekten verbunden, um den individuellen Lernfortschritt zu speichern, sowie mit *Question* und *Solution*, um erstellte Fragen und Lösungswege zu verwalten.

Funktionale Zusammenhänge

In Abbildung 40 sind die Beziehungen zwischen diesen Entitäten dargestellt. Zusammen ermöglichen sie:

- Die flexible Auswahl von Fragen aus einem Pool für eine Übungssitzung
- Das Tracking des individuellen Lernfortschritts pro Frage
- Die Umsetzung von Wiederholungs- und Sperrlogiken im Übungsmodus

Diese Struktur ermöglicht, dass im Übungsmodus flexibel auf die Bedürfnisse der Nutzer:innen eingegangen werden kann, während gleichzeitig eine saubere Trennung zwischen Fragen, Fragenpools der Nutzer:innen und individuellen Fortschritten besteht.

Zentrale Entitäten für Lösungswege

Für die Darstellung von Lösungswegen im Übungsmodus sind vor allem die Entitäten *Solution*, *SolutionStep* und *SolutionVote* relevant. Durch diese können Nutzer:innen

nicht nur die richtige Lösung sehen, sondern auch die einzelnen Schritte nachvollziehen und Feedback von anderen Nutzer:innen erhalten. Zudem können individuelle Lösungswege hochgeladen werden, um sich und anderen Nutzer:innen in der Zukunft zu unterstützen.

Solution

Die Entität *Solution* repräsentiert einen Lösungsweg zu einer Frage. Jede Lösung gehört zu genau einer Frage (*Question*) und einem erstellenden Nutzer (*User*) und stellt sich aus mehreren *SolutionStep*-Objekten zusammen

SolutionStep

SolutionStep-Objekte enthalten die einzelnen Schritte einer Lösung. Jeder Schritt besitzt einen Titel zur Zusammenfassung und einen Text zur Erklärung und Veranschaulichung des nächsten Schritts. Durch die Schritt-für-Schritt-Darstellung kann der/die Nutzer:in den Lösungsweg nachvollziehen und gezielt lernen.

SolutionVote

Diese Entität ermöglicht es anderen Nutzer:innen, eine Lösung positiv oder negativ zu bewerten (*isUpVote*). So werden Lösungswege hochwertige Lösungswege mit mehr positiven Bewertungen hervorgehoben und die Community kann gegenseitig Feedback geben.

Funktionale Zusammenhänge

- Jede Frage kann mehrere von Nutzer:innen erstellte Lösungen besitzen.
- Jede Lösung besteht aus mehreren Schritten.
- Nutzer:innen können Lösungswege bewerten, wodurch die Qualität sichtbar wird.
- Die Anzeige der Lösungen erfolgt im Übungsmodus direkt nach der Beantwortung, wodurch der Lernprozess unterstützt wird.

Datenmodell für Prüfungssimulationen

Für die Umsetzung des Prüfungsmodus sind relevante Entitäten vor allem *Exam* und *ExamQuestion*. Durch diese sind die Verwaltung von Prüfungen, die Speicherung der Antworten und die nachträgliche Auswertung möglich.

Exam

Die Entität *Exam* stellt eine einzelne Prüfung dar. Sie enthält Informationen über das Zeitlimit, den Beginnzeitpunkt, den Endzeitpunkt sowie dem Gesamtergebnis. Über die Beziehung zu *User* ist klar, welcher Nutzer/welche Nutzerin diese Prüfung abgelegt hat. Jede Prüfung kann mehrere Fragen enthalten, die über die Entität *ExamQuestion* abgebildet werden.

ExamQuestion

ExamQuestion speichert die gegebene Antworten auf die einzelne Frage innerhalb einer Prüfung und ist mit dem dementsprechendem Fragenpoolobjekt verknüpft. Sie speichert nicht nur die ausgewählte Antwort, sondern auch ob diese Frage korrekt beantwortet wurde. So können alle Fragen einer Prüfung individuell ausgewertet werden und im Reviewmodus angezeigt werden.

Funktionale Zusammenhänge

Durch die Beziehung zwischen *Exam* und *ExamQuestion* wird folgendes ermöglicht:

- Speicherung von Prüfungen für einzelne Nutzer:innen
- Zuordnung der ausgewählten Antworten zu den jeweiligen Fragen
- Berechnung des Gesamtergebnisses und der prozentual korrekt beantworteten Fragen
- Nachträgliche Ansicht der Prüfung im Reviewmodus, inklusive der Anzeige von korrekt und falsch beantworteten Fragen

Diese Struktur macht eine realistische Simulation einer Prüfungssituation möglich. Die Nutzer:innen bearbeiten Fragen unter Zeitdruck, ihre Antworten werden erfasst und persitiert und eine nachträgliche Auswertung ist jederzeit möglich.

3.7.2 Überblick über Komponenten**Backend / API**

Das auf Quarkus basierende Backend stellt über REST-Endpunkte die zentrale Schnittstelle für das Frontend bereit. Dabei werden alle Daten über JSON-Format übertragen, wobei klassische HTTP-Methoden wie *GET* und *POST* verwendet werden.

Die Kernaufgaben des Backends umfassen:

- **Fragenmanagement:**

Laden von Fragen, Abrufen von Antwortoptionen und Speichern der Nutzerantworten. Die Logik zur Bewertung der Antworten wird durch Services wie *AnswerService* umgesetzt.

- **Prüfungslogik:**

Erstellung und Verwaltung von Prüfungen über den *ExamService*. Dieser Service wählt Fragen aus den individuellen Fragenpools aus, setzt Zeitlimits und berechnet den Endpunktestand. Auch Funktionen wie das Kopieren bereits abgeschlossener Prüfungen werden hier ermöglicht.

- **Fortschritts- und Statistikfunktionen:**

Der *StatsService* liefert alle Daten für die Fortschritts- und Statistikübersichten im Frontend. Dazu gehören globale Statistiken, Fortschritte in einzelnen Themenbereichen sowie die Umsetzung der Sperrlogik für Übungsfragen, um Wiederholungen strukturiert zu steuern.

- **Kommunikation mit dem Frontend:**

Das Backend stellt alle relevanten Daten für den *Question Runner*, die Fortschrittsübersicht und die Ergebnisanzeige der Prüfungen bereit. Eingehende Antworten werden validiert, ausgewertet und in der Datenbank persistiert.

- **Datenzugriff:**

Services greifen auf die Daten über dedizierte Repository-Klassen (wie zum Beispiel *QuestionRepository* oder *ExamRepository*) zu. Das führt zu einer klaren Trennung zwischen Geschäftslogik und Datenzugriff.

Das Backend ist unter anderem für besondere Funktionen, wie die Wiederholungslogik für Übungsfragen, die Sicherstellung der Prüfungskonsistenz sowie die Kopie von abgeschlossenen Prüfungen, verantwortlich. Dadurch bleibt die Trennung zwischen Übungs- und Prüfungsmodus sauber erhalten und im Frontend kann die Darstellung vollständig im Fokus stehen.

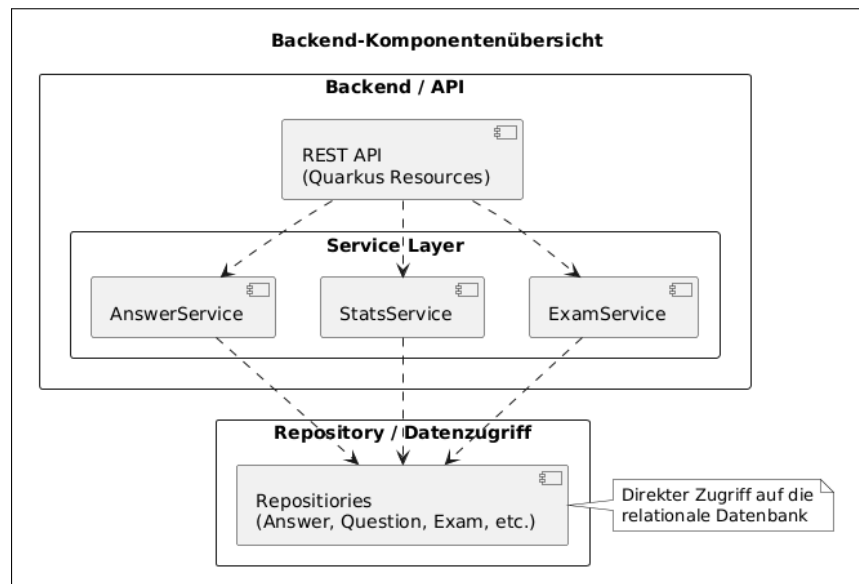


Abbildung 41: Komponentendiagramm des Backends

Frontend-Komponenten

Das Frontend der Anwendung ist sowohl für die Darstellung der Inhalte als auch für die Interaktion der Nutzer:innen mit dem Übungs- und Prüfungsmodus verantwortlich. Dabei ist eine klare Struktur und Wiederverwendung zentraler Komponenten Grundlage für die Gewährleistung von konsistenten Nutzererlebnissen über alle Modi hinweg.

Eine zentrale Rolle nimmt dafür der **Question Runner** ein. Diese Komponente wird sowohl im Übungs-, Prüfungs- als auch im Reviewmodus verwendet und ist für die Darstellung einzelner Fragen, der zugehörigen Antwortmöglichkeiten sowie der Navigation zwischen Fragen verantwortlich. Je nachdem, welcher Modus aktiv ist, verändert sich dabei das Verhalten der Komponente, in Bezug auf Feedback, Darstellung des Zeitlimits oder Bearbeitbarkeit von Antworten, während die grundlegende Oberfläche unverändert bleibt.

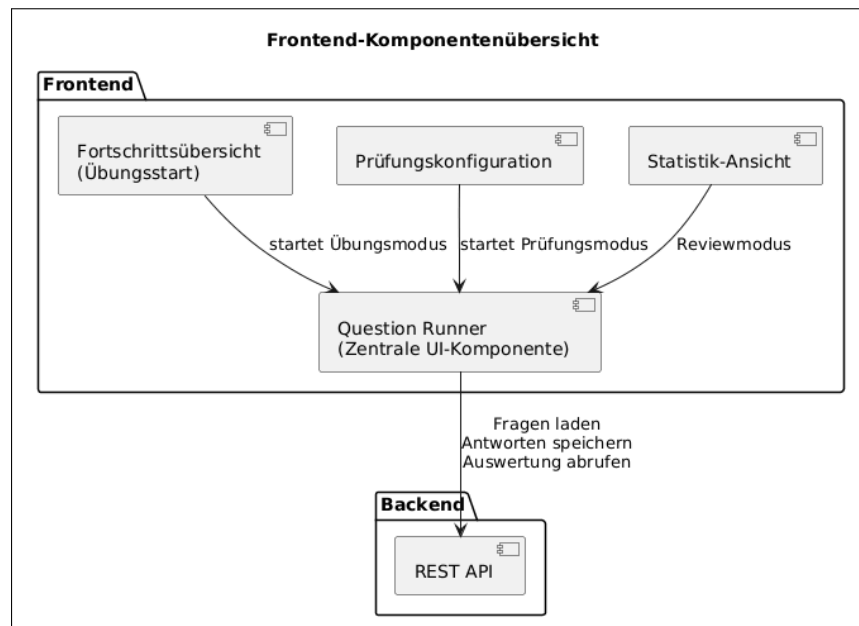


Abbildung 42: Komponentendiagramm des Frontends

In die jeweiligen Modi kann über eigene Views eingestiegen werden, wie etwa die Fortschrittsübersicht im Übungsmodus oder die Prüfungsconfiguration im Prüfungsmodus. Diese Komponenten sind vor allem für die Auswahl von Themenbereichen und den Start eines Durchlaufs zuständig und übergeben lediglich die notwendigen Parameter an den Question Runner.

Diese Aufteilung gewährleistet eine klare Trennung zwischen Darstellungslogik und fachlicher Logik. Zur gleichen Zeit ermöglicht dieser modulare Aufbau eine einfache Erweiterbarkeit für weitere Modi, ohne bestehende Komponenten grundlegend anpassen zu müssen.

3.7.3 Custom CSS-Klassen und eingebundene Bibliotheken

Die Implementierung der Lernplattform erfordert sowohl die Integration externer Bibliotheken zur Datenvisualisierung als auch die Entwicklung wiederverwendbarer CSS-Komponenten. Diese Kombination gewährleistet eine effiziente Entwicklung bei gleichzeitig konsistentem Erscheinungsbild der Anwendung.

Visualisierung von Statistiken mit Chart.js

Um den Lernfortschritt der Nutzer:innen übersichtlich und ansprechend darzustellen, wird im Frontend die JavaScript-Bibliothek Chart.js verwendet. Diese Open-Source-Lösung hat sich als Industriestandard für webbasierte Datenvisualisierungen etabliert

und bietet eine Vielzahl an Diagrammtypen, die individuell angepasst und mit dynamischen Daten befüllt werden können.

Die Wahl von Chart.js begründet sich durch mehrere technische und praktische Vorteile: Die Bibliothek ist weit verbreitet, gut dokumentiert und lässt sich nahtlos in das Angular-Framework integrieren. Zudem bietet sie eine reaktive API, die eine einfache Aktualisierung von Diagrammen bei Datenänderungen ermöglicht. In der entwickelten Lernplattform werden hauptsächlich Donut- und Balkendiagramme verwendet, um verschiedene Aspekte der Lernkurve aufzuzeigen (siehe ??).

Implementierung von Donutdiagrammen

Donutdiagramme eignen sich besonders zur Darstellung prozentualer Verteilungen, beispielsweise des Bearbeitungsstatus von Lernkarten oder der Verteilung von Fachgebieten im Lernfortschritt. Die ringförmige Darstellung mit zentralem Freiraum ermöglicht zusätzlich die Platzierung von Gesamtstatistiken oder Schlüsselkennzahlen im Zentrum des Diagramms.

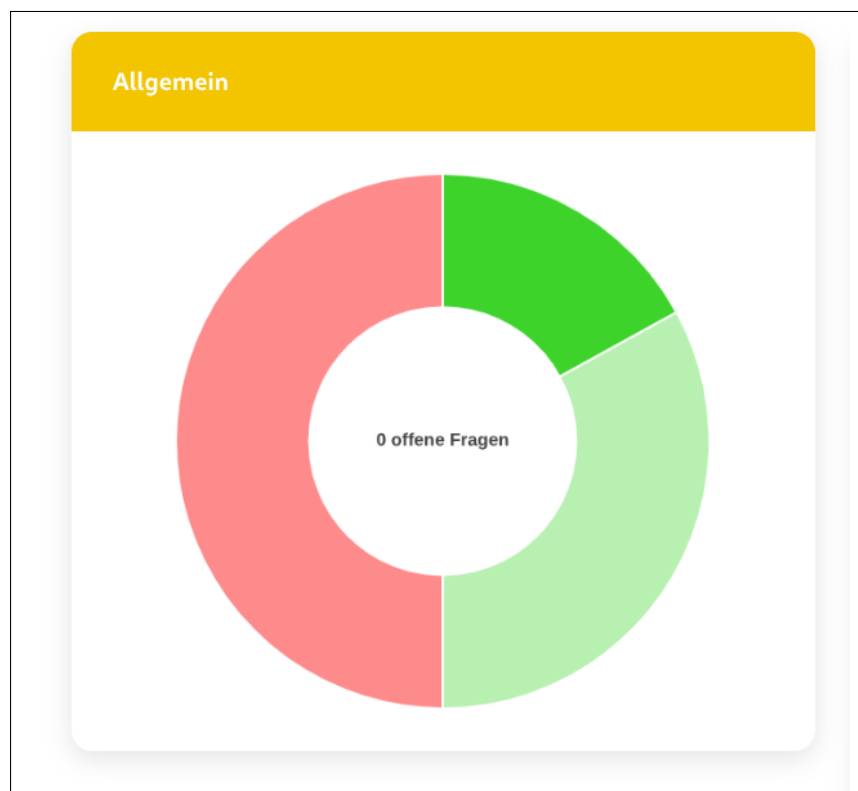


Abbildung 43: Beispielhafte Implementierung von Chart.js zur Visualisierung des Lernfortschritts

Technische Aspekte der Chart.js-Integration

- **Datenaktualisierung:** Die dynamische Natur der Lernplattform erfordert häufige Aktualisierungen der Diagramme, wenn neue Lerndaten generiert werden. Chart.js stellt hierfür die Methode `update()` bereit, welche eine effiziente Neuberechnung und Neudarstellung des Diagramms ermöglicht, ohne dass die gesamte Seite neu geladen werden muss. Dies verbessert die User Experience erheblich, da Übergänge flüssig animiert werden und der Anwendungszustand erhalten bleibt.

Bei der Implementierung wird darauf geachtet, dass nur die tatsächlich geänderten Datenpunkte aktualisiert werden, was die Performance bei größeren Datensätzen optimiert. Die Bibliothek übernimmt dabei automatisch die Interpolation zwischen alten und neuen Werten und erzeugt so eine visuell ansprechende Übergangsanimation.

- **Anpassung des Designs:** Chart.js bietet umfangreiche Möglichkeiten zur Anpassung des visuellen Designs der Diagramme. Farben, Schriftarten, Abstände und andere Stilelemente können so konfiguriert werden, dass sie zum Gesamtdesign der Anwendung passen und ein kohärentes Erscheinungsbild gewährleisten.

In der Lernplattform werden die Farben der Diagramme an das definierte Farbschema der Anwendung angepasst, wobei unterschiedliche Lernstatus durch verschiedene Farbtöne repräsentiert werden. Beispielsweise werden erfolgreich absolvierte Lerneinheiten in Grüntönen, noch zu bearbeitende Inhalte in neutralen Grautönen und fehlerhafte Versuche in Rottönen dargestellt. Diese konsistente Farbkodierung unterstützt die intuitive Erfassung des Lernstatus.

- **Benutzerdefinierte Plugins:** Für spezielle Anforderungen, die über die Standardfunktionalität von Chart.js hinausgehen, können benutzerdefinierte Plugins entwickelt werden. Diese ermöglichen die Implementierung zusätzlicher Funktionalitäten, wie beispielsweise:
 - Anzeige von benutzerdefinierten Tooltips mit erweiterten Informationen beim Hovern über Datenpunkte
 - Integration von Schwellwertlinien zur Markierung von Lernzielen oder Mindestanforderungen
 - Implementierung von Exportfunktionen zur Speicherung von Diagrammen als Bilddateien
 - Hinzufügen von interaktiven Legenden mit erweiterten Filterfunktionen

Die Plugin-Architektur von Chart.js ist so konzipiert, dass eigene Erweiterungen nahtlos in den Rendering-Prozess integriert werden können, ohne die Kernfunktionalität der Bibliothek zu beeinträchtigen.

Wiederverwendbare CSS-Klassen

Die Anwendung nutzt ein System wiederverwendbarer CSS-Klassen, um ein konsistentes Design und Layout über alle Komponenten hinweg zu gewährleisten. Diese Klassen sind in einer zentralen, geteilten CSS-Datei definiert und können komponentenübergreifend verwendet werden. Dieser Ansatz folgt dem DRY-Prinzip (Don't Repeat Yourself) und bringt mehrere Vorteile mit sich:

- **Konsistenz:** Ein einheitliches Erscheinungsbild der Anwendung wird durch die Verwendung derselben Stilklassen in verschiedenen Komponenten sichergestellt.
- **Wartbarkeit:** Änderungen am Design müssen nur an einer zentralen Stelle vorgenommen werden und wirken sich automatisch auf alle verwendenden Komponenten aus.
- **Teamarbeit:** Die gemeinsame CSS-Datei vereinfacht die Zusammenarbeit im Team, da alle Entwickler:innen auf denselben Stil-Definitionen aufbauen.
- **Reduzierung von Redundanz:** Durch die Wiederverwendung werden doppelte Code-Definitionen vermieden, was die Dateigröße reduziert und die Ladezeit der Anwendung optimiert.

Beispiele häufig verwendeter CSS-Klassen

Die folgenden Klassen bilden das Grundgerüst des visuellen Designs der Lernplattform:

- **.widget-wrapper:** Diese Klasse definiert den Grundcontainer für die Widget-Darstellung der Anwendung. Widgets sind modulare UI-Komponenten, die Informationen kompakt und übersichtlich präsentieren, wie beispielsweise Statistikkarten, Fortschrittsanzeigen oder Lernziel-Übersichten.

Die `.widget-wrapper`-Klasse sorgt für ein einheitliches Layout aller Widgets durch standardisierte Abstände, Padding-Werte, Border-Radius und Schatten-Effekte. Dies gewährleistet, dass verschiedene Widget-Typen visuell harmonisieren und als zusammengehörige Elemente wahrgenommen werden. Zusätzlich implementiert die

Klasse responsive Breakpoints, sodass Widgets auf verschiedenen Bildschirmgrößen optimal dargestellt werden.

- **.header:** Die Header-Klasse fügt, wo benötigt, einen visuell hervorgehobenen Kopfbereich hinzu, der mit der primären Farbe des Anwendungsdesigns gestaltet ist. Diese Klasse wird typischerweise für Überschriften von Widgets, Sektionen oder modalen Dialogen verwendet.

Die primäre Designfarbe schafft dabei eine klare visuelle Hierarchie, sodass Nutzer:innen wichtige Bereiche der Anwendung schnell erkennen können. Neben der Hintergrundfarbe legt die Klasse auch Textfarbe, Schriftgröße, Schriftgewicht und vertikale Abstände fest, damit alle Header-Elemente einheitlich dargestellt werden.

- **button.submit-btn:** Diese Klasse stellt einen standardisierten Button-Stil für Interaktionen bereit und kommt vor allem bei Übungsaufgaben, Formular-Submissions und anderen Bestätigungsaktionen zum Einsatz.

Der `.submit-btn` zeigt visuelles Feedback für unterschiedliche Zustände: normale Darstellung, Hover-Effekt, aktiver Zustand beim Klicken und deaktivierter Zustand. Nutzer:innen können dadurch schnell erkennen, welche Schaltflächen primäre Aktionen auslösen. Übergänge und Animationen sorgen dafür, dass Interaktionen flüssiger wirken.

Die Button-Klasse berücksichtigt auch Accessibility-Aspekte wie ausreichende Größe für Touch-Bedienung, gute Farbkontraste und sichtbare Fokus-Indikatoren bei Tastatur-Navigation.

CSS-Architektur und Best Practices

Die CSS-Klassen sind modular strukturiert und unterscheiden zwischen globalen Basis-klassen, komponentenspezifischen Stilen und Utility-Klassen. So lassen sich einzelne Komponenten flexibel anpassen, ohne dass globale Stile beeinflusst werden. Außerdem treten dadurch weniger Spezifität-Konflikte auf, was die Wartbarkeit des Codes verbessert.

3.8 Zukunftsaussichten und Erweiterungsmöglichkeiten

Im Rahmen dieser Arbeit wurde eine Plattform entwickelt, die ein funktionales Fundament für die digitale Prüfungsvorbereitung darstellt. Darauf basierend lassen sich verschiedene Richtungen für zukünftige Erweiterungen identifizieren, mit welchen die Lerneffizienz weiter gesteigert werden kann.

3.8.1 KI-gestützte Inhaltsgenerierung und Feedback

Ein wesentliches Potenzial lässt sich in der Integration von Künstlicher Intelligenz erkennen, um einige Schritte des Lernzyklus zu automatisieren. Dabei ist einer der zentralen Schritte die Automatisierung der Fragerstellung. Aktuelle Technologien wie *Google NotebookLM* [28] zeigen bereits, wie KI-Modelle auf spezifische Quellen wie individuelle Mitschriften spezialisiert werden können, um daraus kontextbezogene Lerninhalte zu generieren, ohne dabei auf externes, potenziell ungesichertes Wissen zurückgreifen zu müssen.

Für LearnHub würde dies zwei wesentliche Vorteile bieten:

Automatisierte Fragergenerierung

Durch die Implementierung eines sogenannten *Retrieval-Augmented Generation (RAG)* [29] Workflows wäre es möglich, den Prozess des Fragerstellens massiv zu beschleunigen. Dabei würden Nutzer:innen ihre Mitschriften als PDF-Dokumente wie gehabt hochladen, während die integrierte KI-Schnittstelle diese Dokumente analysieren und daraus automatisiert Fragen erstellen würde.

- **Multiple-Choice-Extraktion:**

Die KI erkennt aus dem Textabschnitt nicht nur die korrekte Antwort, sondern auch falsche Antwortmöglichkeiten, die durchaus plausibel sind.

- **Kontextbezogene Erklärungen:**

Zu jeder generierten Frage könnte automatisch ein *SolutionStep* erstellt werden, der direkt auf die entsprechende Stelle im Quelldokument verweist.

Dadurch fällt die Hürde aus, dass Nutzer:innen manuell Frage nach Frage in das System einpflegen müssen. Sie müssten lediglich überprüfen, ob die KI mit den Antwortmöglichkeiten keine Fehler gemacht hat und gegebenenfalls anpassen. Dennoch wäre der

Zeitaufwand verglichen zum Manuellen Erstellen jeder Frage durch eine KI-Integration in die Lernplattform deutlich verkürzt.

Intelligente Freitext-Bewertung und Feedback

Eine der größten Hürden bei der digitalen Lernplattform ist die Korrektur von Freitextantworten, die über einen einfachen String-Abgleich hinausgeht. Durch die Einbindung von *Large Language Models* könnte Learnhub die Freitextantworten nicht einfach nur auf exakte Übereinstimmung, sondern auf ihre inhaltliche Korrektheit prüfen.

- **Semantische Analyse:**

Die KI gleicht die Eingabe der Nutzer:innen mit der Musterlösung ab und erkennt korrekte Lösungen auch bei abweichender Formulierung.

- **Personalisiertes Feedback:**

Statt einer binären „Richtig/Falsch“-Wertung könnte das System detailliertes Feedback geben, welche die fehlenden Aspekte einer Antwort beschreiben.

Auf diese Weise würde sich **Learnhub** von einer rein reaktiven Abfrageplattform zu einer aktiven, KI-gestützten Lernbegleitung entwickeln, mit welchem der administrative Aufwand für Inhaltserstellung minimiert und die Qualität des Feedbacks maximiert wird.

3.8.2 Mobile Applikation (Cross-Platform)

Eine Entwicklung einer nativen App für iOS und Android wäre der nächste logische Schritt, um dem Lernverhalten heutiger Schüler:innen gerecht zu werden. Eine native App würde zusätzliche Funktionen wie Push-Benachrichtigungen etwa zur Erinnerung für „Lern-Streaks“ oder Fähigkeit, offline Fragen zu beantworten, ermöglichen. Vor allem das Lernen in kürzeren Zeitfenstern wie zum Beispiel während des Pendelns würde dies deutlich fördern.

4 Zusammenfassung

Diese Diplomarbeit befasst sich mit der Entwicklung der webbasierten Lernplattform **LearnHub**, die als Partnerprojekt für die HTL Leonding konzipiert wurde. Dabei standen die methodische Trennung von Lern- und Testphasen, sowie die Implementierung effizienter Lernstrategien im Vordergrund.

Im Rahmen der Projektentwicklung konnten wir uns besonders im technischen Bereich neues Know-how aneignen. Durch die Arbeit mit modernen Frameworks im Fullstack-Bereich sowie die Anbindung einer zentralen REST-Schnittstelle konnten wir unser bereits bestehendes Wissen vertiefen. Im Laufe des Entwicklungsprozesses haben wir diese Fertigkeiten kontinuierlich ausgebaut. Auch die technische Umsetzung der Lernstrategien, wie die Kombination aus dem Leitner-System und Spaced Repetition, erforderte einiges an Planung, um eine automatisierte Wiederholungslogik zu ermöglichen. Rückblickend sind wir sehr zufrieden mit dem Endergebnis von LearnHub, da der Übungsmodus durch direktes Feedback und der Prüfungsmodus durch eine realitätsnahe Simulation überzeugen.

Trotzdem gibt es Aspekte, die wir im Nachhinein anders angegangen wären. Besonders bei den Usability-Tests besteht Verbesserungspotenzial. Aufgrund von Zeitmangel konnten diese nicht im geplanten Ausmaß durchgeführt werden. Eine frühzeitige Einbindung von Nutzertests hätte geholfen, den Userflow noch flüssiger zu gestalten und die Bedienbarkeit der Plattform weiter zu optimieren.

In der Zukunft bietet LearnHub ein funktionales Fundament für weitere Erweiterungen.

Literaturverzeichnis

- [1] Bundesministerium Bildung, „Standardisierte, kompetenzorientierte Reife- und Diplomprüfung an BHS,” letzter Zugriff am 026.03.2026. Online verfügbar: https://www.bmb.gv.at/Themen/schule/schulpraxis/zentralmatura/srdp_bhs.html
- [2] Ajay Kumar S, „Evolution of Spring Boot and Microservices,” letzter Zugriff am 01.03.2026. Online verfügbar: <https://medium.com/javarevisited/evolution-of-spring-boot-and-microservices-4d1109b5a4d3>
- [3] Michael Vitz, „Java and its annotations,” letzter Zugriff am 02.03.2026. Online verfügbar: <https://www.innoq.com/en/articles/2024/09/java-annotations/>
- [4] —, „Was ist die Magie von Spring Boot?” letzter Zugriff am 01.03.2026. Online verfügbar: <https://www.innoq.com/de/articles/2020/02/spring-boot-magie/>
- [5] Viplav Fauzdar, „Quarkus vs Spring Boot — The Cloud-Native Java Showdown for 2025,” letzter Zugriff am 02.03.2026. Online verfügbar: <https://medium.com/@viplav.fauzdar/quarkus-vs-spring-boot-the-cloud-native-java-showdown-for-2025-deb8bd990894>
- [6] Baeldung, „Spring Boot with Multiple SQL Import Files,” letzter Zugriff am 02.03.2026. Online verfügbar: <https://www.baeldung.com/spring-boot-sql-import-files>
- [7] Ian Holton, „Node.js - Die fünf wichtigsten Fakten,” letzter Zugriff am 02.03.2026. Online verfügbar: https://www.esono.de/blog/softwareentwicklung/node-js-die-fuenf-wichtigsten-fakten_aid_1127.html
- [8] OpenJS Foundation, „The Node.js Event Loop,” letzter Zugriff am 09.03.2026. Online verfügbar: <https://nodejs.org/en/learn/asynchronous-work/event-loop-timers-and-nexttick>
- [9] Muhammad Sufiyan, „Improving Performance in Node.js: Blocking the Event Loop,” letzter Zugriff am 09.03.2026. Online verfügbar: <https://innosufiyan.hashnode.dev/improving-performance-in-nodejs-blocking-the-event-loop>
- [10] ElAmit Mansour, „The Node.js “Single-Threaded” Myth: The 5-Thread Truth Hiding in Plain Sight,” letzter Zugriff am 09.03.2026. Online verfügbar: <https://elamir.medium.com/the-node-js-single-threaded-myth-the-5-thread-truth-hiding-in-plain-sight-db4b9be3c099>
- [11] OpenJS Foundation, „Don’t Block the Event Loop (or the Worker Pool,” letzter Zugriff am 09.03.2026. Online verfügbar: <https://nodejs.org/en/learn/asynchronous-work/dont-block-the-event-loop>
- [12] —, „About Node.js,” letzter Zugriff am 09.03.2026. Online verfügbar: <https://nodejs.org/en/about>

- [13] Prisma Data Inc., „What is Prisma ORM?” letzter Zugriff am 09.03.2026. Online verfügbar: <https://www.prisma.io/docs/orm>
- [14] TypeORM, „TypeORM - Code with Confidence. Query with Power.” letzter Zugriff am 09.03.2026. Online verfügbar: <https://typeorm.io/>
- [15] Ashley Davis, „A complete guide to full-stack live reload,” letzter Zugriff am 09.03.2026. Online verfügbar: <https://blog.logrocket.com/complete-guide-full-stack-live-reload/>
- [16] Yatin Batra, „How to Create Database Seed Scripts in Node.js,” letzter Zugriff am 09.03.2026. Online verfügbar: <https://www.javacodegeeks.com/how-to-create-database-seed-scripts-in-node-js.html>
- [17] ClearPeeks, „Configuration Management in Node.js,” letzter Zugriff am 09.03.2026. Online verfügbar: <https://www.clearpeaks.com/configuration-management-in-node-js/>
- [18] IBM, „The JIT compiler,” letzter Zugriff am 14.03.2026. Online verfügbar: <https://www.ibm.com/docs/en/sdk-java-technology/8?topic=reference-jit-compiler>
- [19] Red Hat, „Quarkus — Supersonic Subatomic Java,” letzter Zugriff am 03.01.2026. Online verfügbar: <https://quarkus.io>
- [20] ERP Solutions oodles, „The Pros and Cons of Quarkus vs Spring Boot,” letzter Zugriff am 14.03.2026. Online verfügbar: <https://erpsolutionsoodles.medium.com/the-pros-and-cons-of-quarkus-vs-spring-boot-a38e8b8df479>
- [21] Red Hat, „Quarkus Guide – Re-augment a Quarkus Application,” letzter Zugriff am 03.01.2026. Online verfügbar: <https://quarkus.io/guides/reaugmentation>
- [22] —, „Quarkus - Extensions,” letzter Zugriff am 03.01.2026. Online verfügbar: <https://quarkus.io/extensions/>
- [23] —, „Dev Services Overview,” letzter Zugriff am 14.03.2026. Online verfügbar: <https://quarkus.io/guides/dev-services>
- [24] Daniel Oh, „Simplify Java persistence using Quarkus and Hibernate Reactive,” letzter Zugriff am 14.03.2026. Online verfügbar: <https://developers.redhat.com/articles/2022/01/06/simplify-java-persistence-using-quarkus-and-hibernate-reactive>
- [25] David, „Speed up development and save time with Live Reload and Testing in Quarkus 2.0,” letzter Zugriff am 14.03.2026. Online verfügbar: <https://medium.com/geekculture/speed-up-development-and-save-time-with-live-reload-and-testing-in-quarkus-2-0-53af6ff65edb>
- [26] Red Hat, „How dev mode differs from a production application,” letzter Zugriff am 14.03.2026. Online verfügbar: <https://quarkus.io/guides/dev-mode-differences>
- [27] Arnaud Roques, „PlantUML,” letzter Zugriff am 05.01.2026. Online verfügbar: <https://www.plantuml.com/plantuml/uml/>
- [28] Google, „NotebookLM - Lernen Sie, was Sie möchten,” letzter Zugriff am 19.03.2026. Online verfügbar: <https://notebooklm.google/students?hl=de>
- [29] —, „Was ist Retrieval-Augmented Generation (RAG)?” letzter Zugriff am 19.03.2026. Online verfügbar: <https://cloud.google.com/use-cases/retrieval-augmented-generation>

Abbildungsverzeichnis

1	Das 3-Säulen-Modell der Reife- und Diplomprüfung ([1])	2
2	Die Ebbinghaus'sche Vergessenskurve und die Wirkung von Spaced Repetition	7
3	Das Leitner-System	8
4	Logo von Spring Boot	11
5	Logo von Node.js	15
6	Logo von Quarkus	20
7	Logo von Angular	28
8	Funktionsweise von RxJS-Observables in der Datenverarbeitung	29
9	Logo von Flutter	31
10	Architektur-Ebenen und Rendering-Pipeline von Flutter	32
11	Logo von React	35
12	Visualisierung des Virtual DOM Diffing-Prozesses	36
13	Ablaufdiagramm eines Users	40
14	Visualisierung der Lernlevel auf der Startseite	42
15	Visualisierung der Sperrzeiten für Fragen im Basislevel	43
16	Zentrale Oberfläche zur individuellen Zusammenstellung des Fragenpools	44
17	Oberfläche zum direkten Testen einer Frage aus der Browsing-Ansicht .	45
18	Fortschrittsübersicht des Übungsmodus	48
19	Auswahl der Themenbereiche für den nächsten Übungsdurchlauf	48
20	Beispielhafte Ansicht einer Frage im Question Runner	49
21	Question-Runner: Beispiele für verschiedene Fragetypen	51
22	Question-Runner: Richtig und Falsch beantwortete Frage	52
23	"PrüfenButton wurde zum AbschließenButton	53
24	Anzeige eines detaillierten Lösungsweges	54
25	Anzeige einer Frage ohne vorhandenen Lösungsweg	54
26	Editor zum Erstellen eines neuen Lösungsweges	54
27	Darstellung des Feedback-Systems für Lösungswege	55
28	Seitennavigationsleiste bei ausgewähltem Prüfungsmodus	56
29	Beispielhafte Ansicht der Prüfungs-Konfiguration	57
30	Start der Prüfung nach Konfiguration	58
31	Prüfungsmodus: Navigationsarten	59
32	Prüfungsmodus: Letzte Frage - Fertigstellen-Button	59
33	Ergebnisübersicht nach Abschluss der Prüfung	61
34	Navigationsleiste nach Auswertung der Prüfung	62
35	Neustart-Button im Reviewmodus	62
36	Übersicht des Prüfungsverlaufs mit chronologischer Historie	63
37	Detailansicht zur Identifikation spezifischer Wissenslücken	64
38	Zentrales Dashboard für den Gesamtfortschritt	65
39	Spezifische Analyse mittels Themenpool-Filter	66
40	Datenbankmodell der Anwendung	67
41	Komponentendiagramm des Backends	72

42	Komponentendiagramm des Frontends	73
43	Beispielhafte Implementierung von Chart.js zur Visualisierung des Lernfortschritts	74

Tabellenverzeichnis